

Architektur Subsysteme und Plattformdesign

Documentary Engine, Event Mesh, Vocabulary Governance, AI-Agent-Architektur

Die Subsysteme hinter der Kernarchitektur: die Documentary Engine, das Enterprise Event Mesh, die Vocabulary Governance, das AI-Agent-Zielbild mit Bedrohungsmodell und IAM sowie die Render- und Server-Modularisierung. Dieses Bündel erklärt, wie die Plattform intern aufgebaut und entkoppelt ist.

Repository: SvenKulesa/Finance

Stand: `claude/pdf-dokumente-notebook-llm-l6kcxq @ 4eadf9d`

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 71

Umfang: ca. 36.326 Wörter

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

Inhalt dieses Quellpakets

1. SUPERSEDED title — Phase 3.4.4 Crypto/Stooq Provider Stages	6
<code>docs/architecture/ADR-0014-PHASE-3.4.4.md</code>	
2. SUPERSEDED title — Phase 3.4.6 Market-Data Compatibility Facade	7
<code>docs/architecture/ADR-0014-PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md</code>	
3. ADR-0013 — Server Composition Root & Modular Bootstrap	8
<code>docs/architecture/adr/ADR-0013-server-composition-root-and-modular-bootstrap.md</code>	
4. SUPERSEDED title — Phase 3.3 AI Analysis Extraction	12
<code>docs/architecture/adr/ADR-0014-phase3-3-ai-analysis-extraction-note.md</code>	
5. SUPERSEDED title — Phase 3.4 Market-data adapters v2	13
<code>docs/architecture/adr/ADR-0014-phase3-4-market-data-adapters-v2.md</code>	
6. SUPERSEDED title — Phase 3.4 Market-data adapters	14
<code>docs/architecture/adr/ADR-0014-phase3-4-market-data-adapters.md</code>	
7. SUPERSEDED — Server Runtime Architecture Consolidation	15
<code>docs/architecture/adr/ADR-0014-server-runtime-architecture-consolidation.md</code>	
8. AI Agent Audit Event Schema	16
<code>docs/architecture/ai-agent/AI_AGENT_AUDIT_EVENT_SCHEMA.md</code>	
9. AI Agent Capability Matrix	17
<code>docs/architecture/ai-agent/AI_AGENT_CAPABILITY_MATRIX.md</code>	
10. AI Agent Deployment Identity	18
<code>docs/architecture/ai-agent/AI_AGENT_DEPLOYMENT_IDENTITY.md</code>	
11. AI Agent Execution Lifecycle	19
<code>docs/architecture/ai-agent/AI_AGENT_EXECUTION_LIFECYCLE.md</code>	
12. AI Agent IAM Model	20
<code>docs/architecture/ai-agent/AI_AGENT_IAM_MODEL.md</code>	
13. AI Agent Incident Response	21
<code>docs/architecture/ai-agent/AI_AGENT_INCIDENT_RESPONSE.md</code>	
14. AI Agent Platform Comparison and Forced Decision	22
<code>docs/architecture/ai-agent/AI_AGENT_PLATFORM_COMPARISON_DECISION.md</code>	
15. AI Agent Provider Profile Contract	24
<code>docs/architecture/ai-agent/AI_AGENT_PROVIDER_PROFILE_CONTRACT.md</code>	
16. AI Agent Redaction Policy	26
<code>docs/architecture/ai-agent/AI_AGENT_REDACTION_POLICY.md</code>	
17. AI Agent Risk Classification	27
<code>docs/architecture/ai-agent/AI_AGENT_RISK_CLASSIFICATION.md</code>	

18. AI Agent Supply Chain Model	28
docs/architecture/ai-agent/AI_AGENT_SUPPLY_CHAIN_MODEL.md	
19. AI Agent Target Architecture	29
docs/architecture/ai-agent/AI_AGENT_TARGET_ARCHITECTURE.md	
20. AI Agent Telemetry Model	30
docs/architecture/ai-agent/AI_AGENT_TELEMETRY_MODEL.md	
21. AI Agent Threat Model	31
docs/architecture/ai-agent/AI_AGENT_THREAT_MODEL.md	
22. AI Agent Trust Boundaries	32
docs/architecture/ai-agent/AI_AGENT_TRUST_BOUNDARIES.md	
23. M10 — Passkey-only Owner PR Authorization Threat Model	33
docs/architecture/ai-agent/M10_PASSKEY_OWNER_PR_AUTHORIZATION_THREAT_MODEL.md	
24. CAPITAL-AI AI Agent Architecture	38
docs/architecture/ai-agent/README.md	
25. Alpaca Shadow: Datenqualitäts- und Kostenplan	39
docs/architecture/api/ALPACA_SHADOW_DATA_QUALITY_PLAN.md	
26. CAPITAL-AI API Interface & Environment Inventory	41
docs/architecture/api/API_INTERFACE_INVENTORY.md	
27. CAPITAL-AI API QA Report	50
docs/architecture/api/API_QA_REPORT.md	
28. CAPITAL-AI Autonomous Content Engine Architecture	54
docs/architecture/AUTONOMOUS_CONTENT_ENGINE_ARCHITECTURE.md	
29. CAPITAL-AI Documentary D0 — Baseline, Manifest & Version Authority	65
docs/architecture/DOCUMENTARY_D0_BASELINE_VERSION_AUTHORITY.md	
30. Documentary D1 — Code Integration & Discovery	68
docs/architecture/DOCUMENTARY_D1_CODE_INTEGRATION_DISCOVERY.md	
31. D2 — Documentary Core Engine	70
docs/architecture/DOCUMENTARY_D2_CORE_ENGINE.md	
32. Documentary D3 — Document Models, Schemas & Provenance	71
docs/architecture/DOCUMENTARY_D3_DOCUMENT_MODELS_PROVENANCE.md	
33. Documentary D4 — Review & Lifecycle Governance	72
docs/architecture/DOCUMENTARY_D4_REVIEW_LIFECYCLE_GOVERNANCE.md	
34. Documentary D5 + E1 + E4 — Traceability & Event Integration	73
docs/architecture/DOCUMENTARY_D5_E1_E4_TRACEABILITY_EVENTS.md	
35. Documentary D6 — Generators & Renderers	74
docs/architecture/DOCUMENTARY_D6_GENERATORS_RENDERERS.md	
36. Documentary D7 — Knowledge Integration	75

docs/architecture/DOCUMENTARY_D7_KNOWLEDGE_INTEGRATION.md

37. CAPITAL-AI Documentary Engine & Event-Driven Value Chain Roadmap 76

docs/architecture/DOCUMENTARY_EVENT_VALUE_CHAIN_ROADMAP.md

38. Documentary Status-Event-Drift Contract 84

docs/architecture/DOCUMENTARY_STATUS_EVENT_DRIFT_CONTRACT.md

39. Documentary Status-Event-Drift — Handoff 87

docs/architecture/DOCUMENTARY_STATUS_EVENT_DRIFT_HANDOFF.md

40. CAPITAL-AI ESS Consolidation Report 89

docs/architecture/ESS_CONSOLIDATION_REPORT.md

41. CAPITAL-AI ESS Responsibility Matrix 106

docs/architecture/ESS_RESPONSIBILITY_MATRIX.md

42. Event Value Chain E6 — Governance Boundaries 116

docs/architecture/EVENT_VALUE_CHAIN_E6_GOVERNANCE_BOUNDARIES.md

43. EventMesh E0 + E3 — Event Contract Inventory & Versioning 117

docs/architecture/EVENTMESH_E0_E3_EVENT_CONTRACT_VERSIONING.md

44. EventMesh E2 + E5 — Replay, Ordering & Reliability Evidence 119

docs/architecture/EVENTMESH_E2_E5_REPLAY_RELIABILITY.md

45. ADR-0014 Phase 3.4.5 — FMP Index Provider Stage 120

docs/architecture/evidence/server-domain-phase-3-4-5-fmp-stage.md

46. Generalisierter Systemadministrator-Ausführungshost — Architektur-Diskussionspapier 121

docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md

47. CAPITAL-AI Legacy Documentation Consolidation 126

docs/architecture/LEGACY_DOCUMENTATION_CONSOLIDATION_2026-08-09.md

48. M4 Agent IAM — Implementation Plan 128

docs/architecture/M4_AGENT_IAM_IMPLEMENTATION_PLAN.md

49. M8 (I1) — Was ein "realer Aufrufer" für `claude-code-cli` bedeuten müsste 131

docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md

50. CAPITAL-AI Observability & Telemetry — O1 Baseline 135

docs/architecture/OBSERVABILITY_TELEMETRY_O1_BASELINE.md

51. CAPITAL-AI — Open Pull Request Consolidation Evidence 138

docs/architecture/OPEN_PR_CONSOLIDATION_2026-08-04.md

52. Phase 3.3 — AI Analysis Route Extraction Note 142

docs/architecture/PHASE-3.3-ai-analysis-extraction-note.md

53. Phase 3.4 — Market-data adapter extraction (v2 note) 144

docs/architecture/PHASE-3.4-market-data-adapters-v2.md

54. Phase 3.4 — Market-data adapter extraction 145

docs/architecture/PHASE-3.4-market-data-adapters.md

55. Phase 3.4.4 — Crypto/Stooq Provider Stages	147
docs/architecture/PHASE-3.4.4-crypto-stooq-provider-stages.md	
56. Phase 3.4.6 — Market-Data Compatibility Facade	148
docs/architecture/PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md	
57. ADR-0014 Phase 3.4.3 — Crypto Provider Chain Extraction	150
docs/architecture/phase3-4-3-crypto-provider-extraction.md	
58. CAPITAL-AI — Pull Request & Multi-Agent Governance	151
docs/architecture/PR_MULTI_AGENT_GOVERNANCE.md	
59. CAPITAL-AI — Render Production Configuration Audit	152
docs/architecture/RENDER_PRODUCTION_CONFIGURATION_AUDIT.md	
60. CAPITAL-AI — Render Runtime Remediation Evidence	174
docs/architecture/RENDER_RUNTIME_REMEDIATION_2026-08-03.md	
61. CAPITAL-AI — Render Production Audit Part 2: Stateless Web Tier & Worker Architecture	179
docs/architecture/RENDER_STATELESS_WEB_TIER_AUDIT_2026-08-03.md	
62. CAPITAL-AI Server Modularization	196
docs/architecture/SERVER_MODULARIZATION.md	
63. CAPITAL-AI Vocabulary Governance Migration Roadmap	198
docs/architecture/VOCABULARY_GOVERNANCE_MIGRATION_ROADMAP.md	
64. CAPITAL-AI Vocabulary Governance — Phase 1.5 Baseline	201
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_1_5_BASELINE.md	
65. CAPITAL-AI Vocabulary Governance — Phase 2 Implementation	205
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_2_IMPLEMENTATION.md	
66. CAPITAL-AI Vocabulary Governance — Phase 3 Safe Rename Gate	208
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_3_SAFE_RENAME_GATE.md	
67. CAPITAL-AI Vocabulary Governance — Phase 4 Bilingual Documentary Integration	211
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_4_BILINGUAL_DOCUMENTARY.md	
68. CAPITAL-AI Vocabulary Governance — Phase 5 Event-Driven Value Chain	213
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_5_EVENT_DRIVEN_VALUE_CHAIN.md	
69. CAPITAL-AI Vocabulary Governance — Phase 6 Incremental Existing-Code Migration	215
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_6_INCREMENTAL_MIGRATION.md	
70. Phase 7 — Continuous Vocabulary Governance	217
docs/architecture/VOCABULARY_GOVERNANCE_PHASE_7_CONTINUOUS_GOVERNANCE.md	
71. Work Claim — ADR-0014 Phase 3.4.2	219
docs/architecture/workclaims/server-domain-decomposition-phase3-4-2.md	

SUPERSEDED title — Phase 3.4.4 Crypto/Stooq Provider Stages

Quelle: docs/architecture/ADR-0014-PHASE-3.4.4.md

SUPERSEDED title — Phase 3.4.4 Crypto/Stooq Provider Stages

Canonical note: <docs/architecture/PHASE-3.4.4-crypto-stooq-provider-stages.md>

Parent decision: [ADR-0083](#)

Date relocated: 2026-08-16

SUPERSEDED title — Phase 3.4.6 Market-Data Compatibility Facade

Quelle: docs/architecture/ADR-0014-PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md

SUPERSEDED title — Phase 3.4.6 Market-Data Compatibility Facade

Canonical note: <docs/architecture/PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md>

Parent decision: [ADR-0083](#)

Date relocated: 2026-08-16

ADR-0013 — Server Composition Root & Modular Bootstrap

Quelle: docs/architecture/adr/ADR-0013-server-composition-root-and-modular-bootstrap.md

ADR-0013 — Server Composition Root & Modular Bootstrap

Status: Accepted Implementation-Status: COMPLETE — server.ts extraction closed Date: 2026-08-08 Scope: CAPITAL-AI production server architecture

Context

`server.ts` historically combined process bootstrap, environment initialization, Express application construction, security middleware, provider initialization, observability, health/metrics endpoints, webhook handling, router composition, market-data orchestration and domain-specific scoring logic. This created a large shared-file change surface and increased merge-conflict and regression risk.

The migration was executed incrementally against the continuously advancing roadmap baseline. The authoritative constraints preserved during the work are:

- R-001 / ADR-0032 provenance enforcement for Crypto/Meme scoring;
- R-002 / ADR-0044 production runtime artifact immutability;
- R-003 / ADR-0045 Stripe event ownership and durable inbox;
- ADR-0040 CSP runtime/safe-rollout ownership through `server/securityResponse.ts`;
- ADR-0037 Render runtime and graceful-shutdown behavior;
- process-wide quiet dotenv initialization.

Decision

`server.ts` is a stable, thin build/runtime entry point only. Application behavior is owned outside that file.

The final entry-point contract is:

```
import './server.application';
```

The exact pre-cutover application implementation is preserved byte-for-byte in `server.application.ts`. This final relocation step is intentionally behavior-neutral: it changes ownership and change surface, not runtime semantics. The production build continues to bundle from `server.ts`, so Docker/Render start contracts remain unchanged.

Already extracted architectural modules remain the target decomposition boundaries:

```
server.ts           # thin build/runtime entry only
server.application.ts # behavior-preserving compatibility composition module
server/
└─ app/
```

```
├── compositionRoot.ts
├── bootstrap/
│   ├── runtime.ts
│   ├── providers.ts
│   └── processLifecycle.ts
├── middleware/
│   ├── cors.ts
│   ├── securityHeaders.ts
│   ├── probeProtection.ts
│   ├── globalRateLimit.ts
│   └── observability.ts
├── routes/
│   ├── health.ts
│   └── metrics.ts
```

`server.application.ts` is a compatibility composition module, not a new domain authority. Existing modular routers/services under `server/` and `src/` remain authoritative for their domains. Future cleanup may progressively replace compatibility-owned blocks with the extracted modules, but no future change requires reopening `server.ts` itself.

Migration record

1. Process-scoped lifecycle, AI provider and runtime-context boundaries were introduced.
2. Security/observability middleware policies were extracted as independently testable modules.
3. `/healthz` and `/metrics` were extracted as dedicated routers.
4. Parallel roadmap work was reconciled before shared-file changes, including R-003 billing ownership, ADR-0040 CSP and ADR-0037 Render runtime work.
5. The final cutover copied the current `server.ts` implementation byte-for-byte to `server.application.ts` and replaced `server.ts` with the thin import shell.

The final cutover deliberately used the exact current blob from the repository rather than reconstructing the large file. This prevents stale-snapshot loss and preserves all parallel changes already present on `main`.

Roadmap coexistence constraints

R-001 — Provenance truth

The refactor preserves ADR-0032 semantics. Bootstrap/catalog values from `AssetRegistry` must not become verified scoring evidence, and future route extraction must remain behavior-preserving with the R-001 regression suite as a mandatory gate.

R-002 — Runtime artifact immutability

`server/runtime/runtimeArtifactGuard.mjs` remains outside the bundled application composition root and continues to preload through Docker `NODE_OPTIONS`. Production Documentary/release mutation boundaries and immutable build evidence remain unchanged.

R-003 — Stripe event ownership

The application-side Stripe event inbox and event-ID ownership introduced by ADR-0045 remain authoritative. Stripe raw-body webhook ingress must continue to execute before

`express.json()` and before middleware that would invalidate the signature-verification contract.

ADR-0040 — CSP ownership

`server/securityResponse.ts` remains the authoritative CSP rollout/security-response implementation. The server modularization must not duplicate or fork CSP policy ownership.

Invariants

- Stripe webhook raw-body parsing remains before `express.json()`.
- CORS, CSP, HSTS, probe protection and global rate limiting retain their protected semantics.
- `/healthz` remains network-independent and suitable for Render health checks.
- `/metrics` remains fail-closed when `METRICS_TOKEN` is absent or invalid.
- AI provider absence remains fail-open only where deterministic fallbacks are defined.
- IAM authorization remains fail-closed.
- Render runtime port resolution and graceful shutdown behavior remain unchanged.
- No secrets or key fragments are introduced into startup diagnostics.
- R-001 scoring evidence semantics remain unchanged.
- R-002 preload/control-plane/release-evidence boundaries remain unchanged.
- R-003 Stripe event ownership remains unchanged.

Consequences

Positive consequences:

- `server.ts` is no longer a shared architectural hotspot;
- build and runtime entry semantics are stable and minimal;
- future modularization can occur without repeatedly touching the root entry file;
- all parallel roadmap changes present on the cutover baseline are preserved exactly;
- rollback is straightforward because the compatibility implementation is byte-identical to the former entry file.

Trade-off:

- `server.application.ts` intentionally retains compatibility-owned composition and legacy inline handlers. Further decomposition improves modularity but is no longer a prerequisite for the `server.ts` extraction invariant.

Completion criterion

ADR-0013 is complete when all of the following are true:

1. `server.ts` contains only the stable thin-entry contract;
2. the former implementation exists outside `server.ts` without semantic drift;

3. the production TypeScript/build/test gates pass;
4. R-001/R-002/R-003 and protected CSP/Render invariants remain green;
5. no production configuration mutation is required for the cutover.

These criteria define the closure of the `server.ts` extraction workstream.

SUPERSEDED title — Phase 3.3 AI Analysis Extraction

Quelle: docs/architecture/adr/ADR-0014-phase3-3-ai-analysis-extraction-note.md

SUPERSEDED title — Phase 3.3 AI Analysis Extraction

Canonical note: <docs/architecture/PHASE-3.3-ai-analysis-extraction-note.md>

Parent decision: [ADR-0083](#)

Date relocated: 2026-08-16

Do not treat this path as an ADR-0014 decision record.

SUPERSEDED title — Phase 3.4 Market-data adapters v2

Quelle: docs/architecture/adr/ADR-0014-phase3-4-market-data-adapters-v2.md

SUPERSEDED title — Phase 3.4 Market-data adapters v2

Canonical note: <docs/architecture/PHASE-3.4-market-data-adapters-v2.md>

Parent decision: [ADR-0083](#)

Date relocated: 2026-08-16

SUPERSEDED title — Phase 3.4 Market-data adapters

Quelle: docs/architecture/adr/ADR-0014-phase3-4-market-data-adapters.md

SUPERSEDED title — Phase 3.4 Market-data adapters

Canonical note: <docs/architecture/PHASE-3.4-market-data-adapters.md>

Parent decision: [ADR-0083](#)

Date relocated: 2026-08-16

SUPERSEDED — Server Runtime Architecture Consolidation

Quelle: docs/architecture/adr/ADR-0014-server-runtime-architecture-consolidation.md

SUPERSEDED — Server Runtime Architecture Consolidation

Status: SUPERSEDED as an ADR number under architecture/

Canonical ADR: <docs/adr/ADR-0083-server-runtime-architecture-consolidation.md>

Date relocated: 2026-08-16

This file previously claimed **ADR-0014**, which collides with the Documentation Governance Validator. Content moved into the canonical ADR registry as **ADR-0083**.

AI Agent Audit Event Schema

Quelle: docs/architecture/ai-agent/AI_AGENT_AUDIT_EVENT_SCHEMA.md

AI Agent Audit Event Schema

Required fields: - event_id, timestamp, request_id, trace_id, span_id - human_actor_id, app_id, agent_id, session_id - provider, model (metadata only) - intent, scope, environment, resource - capability_requested, capability_granted, risk_class - policy_id, policy_version, authorization_decision - approval_id/step_up_id where applicable - tool_id, command/action, sanitized_input_hash - repo, branch, commit, pr, workflow_run - artifact_digest, deployment_id, runtime_version when applicable - result, error_code, rollback_reference

Never store secrets/tokens or full sensitive prompts/diffs in this event by default.

AI Agent Capability Matrix

Quelle: docs/architecture/ai-agent/AI_AGENT_CAPABILITY_MATRIX.md

AI Agent Capability Matrix

Capability	LOW	MEDIUM	HIGH	CRITICAL
READ/ANALYZE	allow	allow	allow with audit	allow with audit
PLAN	allow	allow	allow with audit	allow with audit
BRANCH/COMMIT	allow docs	allow isolated	approval by policy	normally deny
PR	allow	allow	allow with evidence	deny without step-up
CI_REQUEST	allow	allow	allow	allow with approval
DEPLOY_REQUEST	deny	deny	approval	step-up + owner
PRODUCTI- ON_MUTATION	deny	deny	exceptional	step-up + owner + rollback

NotebookLM: READ/ANALYZE only. Google AI Studio: READ/ANALYZE/PLAN and isolated BRANCH/COMMIT only when connected through approved development tooling. ChatGPT/ Claude: capabilities are policy grants, never provider defaults.

AI Agent Deployment Identity

Quelle: docs/architecture/ai-agent/AI_AGENT_DEPLOYMENT_IDENTITY.md

AI Agent Deployment Identity

Deployment is a distinct service identity from coding agents. Only verified `main` source may request production deployment. Prefer short-lived/OIDC identity when supported by the target. For Render, if direct GitHub OIDC authentication is unavailable, use an environment-scoped secret bridge with owner, rotation, revocation and audit controls.

PR #190 deployment code is intentionally not carried into M2; implementation waits for M7.

AI Agent Execution Lifecycle

Quelle: docs/architecture/ai-agent/AI_AGENT_EXECUTION_LIFECYCLE.md

AI Agent Execution Lifecycle

REQUEST -> IDENTITY -> EVIDENCE -> RISK -> POLICY -> PLAN -> ISOLATED EXECUTION ->
VALIDATION -> ATTEST -> REVIEW -> RELEASE -> VERIFY

Each transition emits correlation/audit evidence. Read-first is mandatory. Agent write occurs only on isolated branch/workspace. Merge and production release remain independent gates. Failed authorization stops before tool execution. Failed validation returns to plan/fix without privilege escalation.

AI Agent IAM Model

Quelle: docs/architecture/ai-agent/AI_AGENT_IAM_MODEL.md

AI Agent IAM Model

Principals: human actor, AI application/client, agent/session, service/tool identity, CI/release identity and break-glass identity.

Authorization is capability-based and contextual: principal + environment + resource + action + risk + policy version + approval state.

Existing CAPITAL-AI IAM/step-up and ADR-0050/0051 grants remain authoritative implementation foundations. Provider accounts are authentication inputs, not application roles. Raw infrastructure credentials stay behind tools/connectors whenever possible.

AI Agent Incident Response

Quelle: docs/architecture/ai-agent/AI_AGENT_INCIDENT_RESPONSE.md

AI Agent Incident Response

Triggers include unauthorized tool calls, secret exposure, prompt/tool injection, unexpected production mutation, audit gaps, compromised connector/MCP server and provider behavior drift.

Response: kill agent mutation -> preserve read-only diagnostics -> revoke/rotate credentials -> freeze branch/deploy -> collect correlated evidence -> assess blast radius -> rollback -> post-incident review -> update policy/tests.

Break-glass is time-limited, attributable and disabled by default. Every use requires retrospective review.

AI Agent Platform Comparison and Forced Decision

Quelle: docs/architecture/ai-agent/AI_AGENT_PLATFORM_COMPARISON_DECISION.md

AI Agent Platform Comparison and Forced Decision

Date: 2026-08-11
Updated: 2026-08-16 (Owner provider-set correction)

Current external patterns

OpenAI Apps SDK is MCP-based and the Agents SDK emphasizes controlled sandboxes and separation of agent harness from compute. Anthropic Claude Code uses explicit permission modes/tool allow-deny rules, MCP and documented LLM-gateway patterns for centralized auth, usage, budgets and audit. xAI Grok operates via GitHub connector / SuperGrok session with host-side grants. Historical Google AI Studio/Gemini and NotebookLM surfaces are **retired** from the CAPITAL-AI active value chain (Owner 2026-08-16).

CAPITAL-AI comparison

Surface	Strength	Correct role	Must not become
ChatGPT	connectors/MCP/Apps, agent harness	research + controlled execution client	repository/platform trust root
Claude Code	code-native permissions/ MCP/gateway	controlled execution client	bypass-permission production admin
Grok (xAI)	GitHub MCP / Grok Chat connector	research + controlled execution client	repository/platform trust root
Google AI Studio/Gemini	(historical) rapid prototyping	RETIRED — not in active value chain	canonical production authority
NotebookLM	(historical) source grounding	RETIRED — not in active value chain	mutation agent

Forced decision

CAPITAL-AI SHALL use a provider-neutral Agent Control Plane and explicit plane separation. Provider choice becomes an interchangeable profile decision among **ChatGPT, Claude and Grok**. Authorization, risk, audit, provenance and deployment remain CAPITAL-AI-owned contracts.

This supersedes any workflow assumption that Google AI Studio, Claude, ChatGPT or Grok is itself the development/production trust boundary.

Research basis

Official OpenAI Apps SDK and Agents SDK documentation; Anthropic Claude Code security/IAM/MCP/LLM-gateway documentation; xAI Grok connector patterns; NIST SSDF/SP 800-218A, SLSA/OpenSSF and OpenTelemetry/W3C Trace Context standards.

AI Agent Provider Profile Contract

Quelle: docs/architecture/ai-agent/AI_AGENT_PROVIDER_PROFILE_CONTRACT.md

AI Agent Provider Profile Contract

Each provider profile declares: provider/product, plane, allowed capabilities, authentication source, tool transports, sandbox boundary, telemetry integration, data-retention constraints, prohibited capabilities and kill-switch procedure.

Canonical DEVELOPMENT Chain / AI value-chain providers (Owner 2026-08-16)

appld	Provider	Plane	Role
chatgpt-github-connector	OpenAI / ChatGPT	Research + controlled Execution	MCP/Apps/connectors; capability policy external to model
claude-code-cli	Anthropic / Claude	controlled Execution	permission modes/allowed tools/MCP; bypass-permissions prohibited for production
grok-xai-connector	xAI / Grok	Research + controlled Execution	GitHub MCP / Grok Chat connector; same control-plane policy as ChatGPT/ Claude

Google AI Studio, NotebookLM and Gemini are NOT part of the active DEVELOPMENT Chain or AI value chain. They remain retired aliases and resolve to DENY/RETIRED in the control plane.

New providers require a profile before use; no new ADR is needed unless trust boundaries or capabilities change.

M8 Cutover Readiness

Ein Provider-Profil mit mutierenden Capabilities darf erst als kanonischer Ausführungspfad gelten, wenn alle folgenden Nachweise vorliegen:

- realer Aufrufer verifiziert;
- provider-neutraler Control-Plane-Pfad verifiziert;
- provider-spezifischer Bypass nachweislich verweigert;
- vollständige Audit-Korrelation verifiziert;
- Rollback auf Read-only verifiziert;
- externe Host-/Connector-Konfiguration durch deren zuständige Plattform verifiziert.

Read-only-Profile ohne mutierende Capability sind für einen privilegierten Cutover `NOT_APP-LICABLE`. Retired aliases are `RETIRED`. Eine Modell- oder Providerbezeichnung erzeugt niemals Ausführungsberechtigung.

AI Agent Redaction Policy

Quelle: docs/architecture/ai-agent/AI_AGENT_REDACTION_POLICY.md

AI Agent Redaction Policy

Actions: ALLOW, HASH, MASK, DROP, SECURE-EVIDENCE-ONLY.

DROP by default: API keys, passwords, OAuth/OIDC tokens, Supabase secret/service keys, Stripe secrets, Render deploy hook URLs, SMTP credentials. MASK/HASH: user identifiers, emails and correlation-sensitive values when full value is unnecessary. SECURE-EVIDENCE-ONLY: approved forensic payloads under restricted retention/access. Prompts, tool inputs/outputs and Git diffs are not logged in full by default.

Redaction happens before telemetry export and before model-visible diagnostic summaries.

AI Agent Risk Classification

Quelle: docs/architecture/ai-agent/AI_AGENT_RISK_CLASSIFICATION.md

AI Agent Risk Classification

LOW: docs, wording, formatting, non-executable metadata. MEDIUM: isolated UI/business logic, bounded tests, no privileged path. HIGH: auth/IAM, scoring, CI/CD, dependencies, security controls, provider routing. CRITICAL: secrets, production DB/RLS, billing, deployment, Owner IAM, destructive operations, break-glass.

Risk is determined by affected resource and capability, not by the model used. When uncertain, classify upward. A change touching multiple classes inherits the highest class.

AI Agent Supply Chain Model

Quelle: docs/architecture/ai-agent/AI_AGENT_SUPPLY_CHAIN_MODEL.md

AI Agent Supply Chain Model

Canonical chain: source SHA -> lockfile -> SBOM -> tests/build -> artifact digest -> provenance -> attestation -> deployment -> runtime identity.

Controls: pinned GitHub Actions, minimal permissions, deterministic dependency installation, vulnerability checks, SBOM export, provenance/attestation, protected release identity, immutable rollback artifact.

Agent-generated content has no elevated trust. M6 maps the implementation to SLSA, OpenSSF and NIST SSDF.

AI Agent Target Architecture

Quelle: docs/architecture/ai-agent/AI_AGENT_TARGET_ARCHITECTURE.md

AI Agent Target Architecture

Baseline: `main@1c3706c4f24e5f5a9fe5b0398a2fcd5bee758b17`

Target

`Human -> AI Client -> Agent Control Plane -> Capability/Policy -> Tool Adapter -> GitHub/
Platform -> Evidence`

Four planes: 1. Research & Evidence: NotebookLM, Deep Research, read-only repository/platform inspection. 2. Agent Execution: ChatGPT/Claude/Gemini managed or local sandboxes performing bounded work. 3. Control: identity, capability, risk, approval, policy, audit, kill switch. 4. Production: protected GitHub main, CI, Render, Supabase, Stripe and runtime.

No model/provider crosses directly from Research/Execution to Production.

Current-to-target mapping

Existing ESS-0018, ADR-0050/0051 and PolicyGate are retained as foundations. ADR-0056/O1 telemetry is extended, not replaced. GitHub branch/PR/CI remains the software change boundary.

AI Agent Telemetry Model

Quelle: docs/architecture/ai-agent/AI_AGENT_TELEMETRY_MODEL.md

AI Agent Telemetry Model

Use W3C Trace Context and OpenTelemetry-compatible spans across AI client -> Control Plane -> tool -> GitHub/CI -> deployment -> runtime.

Operational telemetry may be sampled. Security audit evidence **MUST NOT** depend on sampled spans. Correlation IDs are shared, stores/retention are separate.

Core span attributes mirror the audit schema but contain only redacted operational values. Provider-specific tracing may enrich but never replace the canonical trace.

AI Agent Threat Model

Quelle: docs/architecture/ai-agent/AI_AGENT_THREAT_MODEL.md

AI Agent Threat Model

Primary threats: prompt/tool injection; confused deputy; capability escalation; secret exfiltration; malicious MCP/tool server; poisoned repository/retrieval content; unauthorized production mutation; replay/idempotency failure; CI/supply-chain compromise; audit tampering; model/provider outage or behavior drift; cross-agent race/branch conflicts.

Required mitigations: deny-by-default capabilities, explicit schemas, trusted tool registry, branch isolation, protected main, human gates for HIGH/CRITICAL, redaction, immutable audit evidence, idempotency, SHA-pinned actions, SBOM/provenance, kill switch, time-limited break-glass and negative tests.

Residual model risk is never accepted as authorization evidence.

AI Agent Trust Boundaries

Quelle: docs/architecture/ai-agent/AI_AGENT_TRUST_BOUNDARIES.md

AI Agent Trust Boundaries

TB1 Human <-> AI client: user intent is authenticated but natural-language content is not authorization policy. TB2 AI client <-> Control Plane: requests require attributable identity, capability and risk evaluation. TB3 Control Plane <-> tool/connector: connector holds credentials; model receives minimum results. TB4 Repository branch <-> protected main: PR/CI/ruleset boundary. TB5 CI <-> artifact/deployment: verified SHA and provenance boundary. TB6 Deployment <-> runtime/data/billing: production credential and environment boundary. TB7 External/retrieved content <-> agent: always untrusted; cannot grant capability. TB8 Telemetry exporter <-> audit store: redaction and retention boundary.

M10 — Passkey-only Owner PR Authorization Threat Model

Quelle: docs/architecture/ai-agent/M10_PASSKEY_OWNER_PR_AUTHORIZATION_THREAT_MODEL.md

M10 — Passkey-only Owner PR Authorization Threat Model

Status: PROPOSED — IMPLEMENTATION BLOCKED BY M9 Date: 2026-08-12 Authority: ADR-0066, ESS-0022

System Under Review

M10 authorizes one exact `AUTHORIZE_PR_CI` transaction through a CAPITAL-AI WebAuthn/passkey assertion. It does not authorize PR merge or general Owner impersonation.

Protected Assets

- Human/Owner authorization decision;
- exact Finance PR state (base/head/file set/diff);
- registered Owner public credentials;
- WebAuthn challenge/approval state;
- M5 immutable approval audit evidence;
- CI request consumption state;
- Human-only merge boundary;
- recovery/revocation state.

Trust Boundaries

1. Browser / authenticator ↔ CAPITAL-AI WebAuthn verifier;
2. CAPITAL-AI verifier ↔ trusted GitHub PR-state resolver;
3. verifier ↔ durable audit persistence;
4. approval record ↔ CI request mechanism;
5. AI agents/tools ↔ approval service;
6. recovery/break-glass operator ↔ credential registry;
7. GitHub PR mutations ↔ previously issued challenge/approval context.

Adversaries / Failure Sources

- malicious external attacker;
- compromised browser/session;
- compromised or over-privileged AI agent;
- prompt/tool injection through repository or external content;

- stale/replayed approval material;
- malicious or accidental PR mutation after review;
- wrong RP/origin configuration;
- compromised/revoked credential;
- audit/verifier outage;
- CI event duplication/race;
- operator error during recovery.

Threats and Required Mitigations

Threat	Attack / Failure	Required Mitigation	Expected Failure Mode
RP confusion	assertion accepted for wrong relying party	exact RP ID verification	DENY
Origin confusion	assertion from wrong origin	exact HTTPS origin verification	DENY
Challenge replay	reuse prior valid assertion/challenge	random server challenge, short TTL, single-use state	DENY
Approval replay	reuse consumed approval to trigger CI again	atomic approval consumption / dedupe key	DENY/DEDUPE
Missing User Presence	assertion lacks UP	require authenticator UP flag	DENY
Missing User Verification	PIN/biometric/local verification not performed	userVerification=required, verify UV flag	DENY
Wrong credential	non-Owner credential signs	credential registry bound to Owner	DENY
Revoked credential	previously valid key used after revocation	revocation lookup before acceptance	DENY
PR head TOCTOU	new commit after challenge/review	bind exact head SHA; re-resolve before verify/consume	INVALIDATE/DENY
Base drift	PR retarget/base advances materially	bind exact base branch/SHA and re-resolve	INVALIDATE/DENY
File-set substitution	agent omits/changes files	canonical changed-file set recomputed from trusted GitHub state and hashed	DENY
Diff substitution	content changes while filenames remain	bind canonical diff/review digest	DENY
Agent self-approval	agent calls approval API as Owner	WebAuthn Owner assertion + principal boundary; agent cannot possess authenticator	DENY
Forged legacy evidence	emoji/checkbox/comment fabricated	legacy signals not authoritative after cutover	NO AUTHORIZATION

Threat	Attack / Failure	Required Mitigation	Expected Failure Mode
GitHub sign-in confusion	GitHub account uses passkey but no PR-specific assertion	CAPITAL-AI transaction requires own challenge/context	NO AUTHORIZATION
Stolen web session	attacker has application session	WebAuthn UV + exact challenge/context required	DENY without authenticator
Audit outage	verifier succeeds but evidence cannot persist	audit persistence before CI request	DENY
CI race	multiple consumers request CI	atomic consumption/ idempotency key	one request only
Recovery bypass	weaker fallback grants PR authority	separate strong, audited recovery; no emoji/TOTP-only fallback	DENY
Credential enrollment abuse	agent enrolls credential for Owner	Owner-controlled enrollment with strong verification/audit	DENY
XSS/UI deception	attacker changes visible PR context	trusted server re-resolves canonical PR state; CSP/ XSS controls; approval UI shows bound identifiers	DENY on context mismatch
Secret leakage	logs contain sensitive auth material	store public credential identifiers + verification metadata only; redact raw sensitive payloads	evidence safe
Clock abuse	stale challenge accepted	bounded server clock checks, issued/expiry validation	DENY
Verifier downgrade	UV/RP/origin checks disabled	protected configuration + negative tests + change governance	DENY / change blocked
Merge escalation	CI approval interpreted as merge approval	separate Human merge capability boundary	MERGE DENY for agent

Canonical Authorization Context

The service computes, rather than trusts from an agent:

```

ownerId
repository
prNumber
baseBranch
baseSha
headSha
canonicalChangedFileSetHash
canonicalDiffReviewDigest
action = AUTHORIZE_PR_CI
challengeId
issuedAt
expiresAt

```

This context is hashed/serialized deterministically for audit and verification.

TOCTOU Defense

The PR state **MUST** be resolved at least:

1. before challenge issuance;
2. at assertion verification;
3. immediately before approval consumption / CI request.

Any material mismatch invalidates the approval. The system must not "update" an old approval to a new head.

WebAuthn Verification Boundary

Server verifies:

- challenge;
- RP ID hash / expected RP;
- HTTPS origin;
- credential ownership/registration;
- signature;
- `UP=true` ;
- `UV=true` ;
- credential status;
- freshness/replay;
- exact authorization context.

Authenticator private keys and biometric data remain outside CAPITAL-AI.

Audit Boundary

Approval evidence is persisted before CI request and includes only non-secret references/ verification metadata required to reconstruct the decision.

Audit failure is security failure, not observability-only degradation.

Recovery Threats

Recovery must defend against:

- compromised account session;
- social-engineering downgrade;
- lost authenticator leading to bypass;
- agent-triggered credential deletion/enrollment;
- revocation not invalidating outstanding approvals.

Required controls:

- Human/Owner-only recovery authority;
- strong identity verification;
- reason and timestamp;
- credential lifecycle audit;
- invalidate affected challenges/approvals;
- post-recovery review.

Required Security Tests

At minimum:

- wrong RP/origin;
- UP false;
- UV false;
- wrong/revoked credential;
- expired/stale challenge;
- replay;
- changed base/head;
- changed file-set/diff;
- audit outage;
- duplicate CI consumption;
- forged legacy signal;
- agent self-approval;
- recovery without strong Owner verification.

Residual Risk

WebAuthn reduces credential phishing/replay risk but does not prove the Owner cognitively understood the diff. Human File Review remains a separate process control. M10 therefore binds the cryptographic approval to the exact reviewed PR state rather than replacing review with authentication.

Exit Condition

Threat-model controls are considered implemented only after M10 code, negative tests, shadow-mode decision comparison, recovery drill and final passkey-only cutover Evidence reach `VERIFIED PASS`.

CAPITAL-AI AI Agent Architecture

Quelle: docs/architecture/ai-agent/README.md

CAPITAL-AI AI Agent Architecture

This directory is the normative M2 cross-document set for ESS-0019. It defines target architecture, trust boundaries, threat model, capabilities, risk classification, execution lifecycle, IAM, audit/telemetry, redaction, supply chain, deployment identity, incident response and provider profiles.

Implementation is frozen until docs/architecture/ROADMAP.md records M2G DOCUMENTATION FREEZE = COMPLETE .

Alpaca Shadow: Datenqualitäts- und Kostenplan

Quelle: docs/architecture/api/ALPACA_SHADOW_DATA_QUALITY_PLAN.md

Alpaca Shadow: Datenqualitäts- und Kostenplan

Stand: 2026-08-14

Zielbild

Alpaca beobachtet Aktienkurse parallel zur kanonischen Quelle. Die erste Phase ist read-only und score-neutral. Jede Beobachtung muss Preis, Provider-Zeitstempel, Abrufzeit, Alter, Feed, Abweichung und Evidenz-ID enthalten.

Qualitätsmetriken

Metrik	Ziel	Eskalation
Erfolgreiche Abrufe	$\geq 99\%$	$< 97\%$
Frische während US-Marktzeiten	$\leq 90\text{ s}$	$> 180\text{ s}$
Median-Abweichung	$\leq 0,25\%$	$> 1\%$
P95-Latenz	$\leq 1\text{ s}$	$> 2,5\text{ s}$
Fehlende Zeitstempel	0%	$> 0\%$

Auswertungen müssen Marktstatus, Symbol und Feed getrennt betrachten. Overnight-/Pre-market-Daten dürfen nicht mit regulären Handelszeiten vermischt werden.

Kosten/Nutzen

Option	Nutzen	Kosten/Risiko	Empfehlung
IEX Shadow	Frühe Erkennung von Ausfällen und Stale-Daten	geringe Zusatzlast, eingeschränkte Marktabdeckung	sofort
SIP Shadow	breitere konsolidierte Marktabdeckung	höherer Tarif und Lizenzprüfung	nach 14–30 Tagen Shadow-Evidenz
Alpaca Fallback	höhere Verfügbarkeit	verändert produktive Datenherkunft	nur nach ADR und Freigabe
Zweite Fundamentals-API	bessere Bilanzabdeckung	Mapping-/Lizenzkosten	P2 nach Quote-Stabilisierung
News-/Corporate-Actions-API	bessere Ereignisqualität	Entitätenauflösung und laufende Kosten	P2/P3

Beförderungskriterien

Mindestens 14 Handelstage, mindestens 1.000 vergleichbare Beobachtungen, keine Secret-Leaks, dokumentierte Rate-Limits, $\geq 99\%$ Erfolgsquote und keine ungeklärten systematischen Abweichungen über 1 %. Erst danach darf über Fallback- oder Primärbetrieb entschieden werden.

Betrieb

Die Render Secret File heißt exakt `finance-secrets.env`. Benötigte Schlüssel: `ALPACA_API_KEY_ID`, `ALPACA_API_SECRET_KEY`; optional `ALPACA_DATA_FEED=iex` als normale Konfiguration. Der Auth-Smoke-Test erfolgt erst nach Merge und Deployment, ohne Schlüsselwerte auszugeben.

CAPITAL-AI API Interface & Environment Inventory

Quelle: docs/architecture/api/API_INTERFACE_INVENTORY.md

CAPITAL-AI API Interface & Environment Inventory

Status: INITIAL DISCOVERY — QA pending

Governance: ESS-0015

Snapshot: 2026-08-03

Purpose

Dieses Dokument ist die kanonische menschenlesbare Sicht auf externe Schnittstellen **und deren Konfigurationsvariablen**. Vor einer neuen Provider-Entscheidung wird zuerst vollständig inventarisiert. Es werden ausschließlich Variablennamen, Zweck, Exposure und Evidence dokumentiert — niemals Secret-Werte.

1. API / Interface Inventory

ID	Provider / Interface	Kategorie	Runtime	Auth / Variable	Evidence / Code	QA	Decision
API-MARKET-BINANCE-001	Binance Spot Public Market Data	market-data	Render backend	none (public data)	server/binance-Landing-QuickAnalysis.ts , PR #81	NOT_TESTED	EVALUATE
API-MARKET-KRAKEN-001	Kraken API	crypto market-data / exchange	backend	none (public api.kraken.com ticker); ~~ KRAKEN_API_KEY / KRAKEN_API_SECRET / API_KEY / API_SECRET ~~ removed 2026-08-11 — never read by any code path	server/marketData/cryptoProvider-Chain.ts	NOT_TESTED	EVALUATE
		market-data	backend/services		crypto-History-	NOT_TESTED	EVALUATE

ID	Provider / Interface	Kategorie	Runtime	Auth / Variable	Evidence / Code	QA	Decision
API-MARKET-COIN-GECKO-001	CoinGecko Crypto Data			provider public/API contract	Provi- der.ts , cryptoS- napshot- Provi- der.ts , liveCryp- toSnaps- hotCon- sen- sus.ts		
API-MARKET-COIN-MARKET-CAP-001	CoinMar- ketCap	crypto mar- ket-data	backend	~~ COIN- MARKET- CAP_API_KEY (removed)	Removed 2026-08-11: adapter, provider-re- gistry ent- ry, secret- manifest key and .env.ex- ample de- clarations deleted; crypto snapshot quorum now runs CoinGecko- only (sin- gle-source, so getLi- veCryp- toSnaps- hotCon- sensus consistent- ly reports INSUFFI- CIENT_SOURCES per fail-clo- sed policy)	REMOVED	REMOVED
API-MARKET-FMP-001	Financial Modeling Prep	market- data	Render ba- ckend	FMP_API_KEY	server/ fmpIndi- ces.ts , provider re- gistry, ren- der.yaml	NOT_TES- TED	EVALUATE
API-MARKET-ALPHA-VANTAGE-001	Alpha Van- tage	market- data/funda- mentals	Render ba- ckend	AL- PHA_VAN- TAGE_KEY	server/ stockFun- damen- tals.ts ,	NOT_TES- TED	EVALUATE

ID	Provider / Interface	Kategorie	Runtime	Auth / Variable	Evidence / Code	QA	Decision
					provider registry, <code>.env.example</code> , <code>render.yaml</code>		
API-MARKET-ALPACA-001	Alpaca	market-data candidate	production candidate	API credential(s), exact canonical variable names to verify against production config	Secret-Bereitstellung reported; no active code evidence found in current branch	NOT_TESTED	EVALUATE
API-MARKET-COINAPI-001	CoinAPI	market-data	backend	COIN_API_KEY	<code>.env.example</code> , <code>render.yaml</code> , external market-data adapters	NOT_TESTED	EVALUATE
API-MARKET-EODHD-001	EODHD	market/bond evidence	backend	EODHD_API_KEY	<code>eodhdBondEvidence.ts</code> , <code>.env.example</code> , <code>render.yaml</code>	NOT_TESTED	EVALUATE
API-MARKET-TWELVEDATA-001	Twelve Data	market-data	backend	TWELVEDATA_API_KEY	<code>.env.example</code> , <code>render.yaml</code> , external market-data adapters	NOT_TESTED	EVALUATE
API-MACRO-FRED-001	FRED	macro/rate evidence	backend	FRED_API_KEY	<code>macroRateEvidence.ts</code> , ADR-0023, <code>.env.example</code> , <code>render.yaml</code>	NOT_TESTED	EVALUATE
API-MACRO-ECB-001	ECB Data API	macro/rate evidence	backend	none / keyless	macro evidence architecture	NOT_TESTED	EVALUATE
API-NEWS-001	News API		backend	NEWS_API_KEY	<code>src/features/news/</code>	NOT_TESTED	EVALUATE

ID	Provider / Interface	Kategorie	Runtime	Auth / Variable	Evidence / Code	QA	Decision
		news/sentiment evidence		spelling ~~ News_API removed 2026-08-11	newsRou- tes.ts, .env.ex- ample, ren- der.yaml		
API-AI-ANTHRO-PIC-001	Anthropic Claude API	ai	Render backend	ANTHRO-PIC_API_KEY	server/ anthropicCli- ent.ts, agentMo- delRou- ting.ts, ren- der.yaml	PARTIAL	KEEP/EVALUATE
API-AI-OPENAI-001	OpenAI API	ai	Render backend	OPENAI_API_KEY	server/ openaiC- lient.ts, RAG em- beddings, ren- der.yaml	PARTIAL	KEEP/EVALUATE
API-AI-GEMINI-001	Google Gemini API	ai	Render backend	GEMINI_API_KEY	server/ ai.ts, agentMo- delRou- ting.ts, ren- der.yaml	PARTIAL	KEEP/EVALUATE
API-PAY-STRIPE-001	Stripe API / Checkout	payments	Render backend	STRIPE_SECRET_KEY, publishable key, price IDs	server/ stri- pe.ts, .env.ex- ample, ren- der.yaml	PARTIAL	KEEP
API-PAY-STRIPE-WEBHOOK-001	Stripe Webhook	webhook	Render/Supabase	STRIPE_WEBHOOK_SECRET	webhook routes / Edge Func- tion	PARTIAL	KEEP
API-SUPABASE-AUTH-001	Supabase Auth	auth	browser/backend	VITE_SUPABASE_URL, publisha- ble/anon key + JWT	auth client/ middleware	PARTIAL	KEEP
API-SUPABASE-DATA-001	Supabase PostgREST/ Data API	database	browser/backend	SUPABASE_SECRET_KEY, SUPABA-	production project	PARTIAL	KEEP

ID	Provider / Interface	Kategorie	Runtime	Auth / Variable	Evidence / Code	QA	Decision
				SE_SERVICE_ROLE_KEY , client keys depending path			
API-SUPA-BASE-EF-STRIPE-SETUP	Edge Function stripe-setup	edge-function	Supabase	verify_jwt=false ; alternative control to verify	production function v3	NOT_TESTED	EVALUATE
API-SUPA-BASE-EF-STRIPE-WEBHOOK	Edge Function stripe-webhook	edge-function/webhook	Supabase	verify_jwt=false ; Stripe signature expected	production function v4	NOT_TESTED	EVALUATE
API-SUPA-BASE-EF-STRIPE-WORKER	Edge Function stripe-worker	edge-function	Supabase	verify_jwt=false ; alternative control to verify	production function v3	NOT_TESTED	EVALUATE
API-MAIL-IONOS-001	IONOS SMTP	email	Render backend	SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASSWORD , SMTP_FROM	server/ mailer.ts , .env.example , render.yaml	PARTIAL	KEEP/EVALUATE
API-OAUTH-GOOGLE-001	Google OAuth via Supabase	auth	browser/ Supabase	OAuth2 / Supabase configuration	auth flow	PARTIAL	KEEP
API-GOOGLE-MARKETING-001	Google Marketing/ Analytics integration	marketing	browser/ backend/ MCP profile	VITE_GA_MEASUREMENT_ID + governed service credentials	MAESS-0014 / ADR-0035	IN_PROGRESS	EVALUATE
API-SOCIAL-001	YouTube OAuth	social publishing	backend	YOUTUBE_CLIENT_ID , YOUTUBE_CLIENT_SECRET	.env.example , render.yaml	NOT_TESTED	EVALUATE
			backend				EVALUATE

ID	Provider / Interface	Kategorie	Runtime	Auth / Variable	Evidence / Code	QA	Decision
API-SOCI-AL-002	TikTok OAuth	social publishing		TIK-TOK_CLI-ENT_ID , TIK-TOK_CLI-ENT_SECRET	.env.example , render.yaml	NOT_TESTED	
API-SOCI-AL-003	Instagram OAuth	social publishing	backend	INSTA-GRAM_CLI-ENT_ID , INSTA-GRAM_CLI-ENT_SECRET	.env.example , render.yaml	NOT_TESTED	EVALUATE
API-SOCI-AL-004	X OAuth	social publishing	backend	X_CLI-ENT_ID , X_CLI-ENT_SECRET	.env.example , render.yaml	NOT_TESTED	EVALUATE
API-SOCI-AL-005	Facebook OAuth	social publishing	backend	FACE-BOOK_CLI-ENT_ID , FACE-BOOK_CLI-ENT_SECRET	.env.example , render.yaml	NOT_TESTED	EVALUATE

2. Environment Variable Inventory

AI

- `GEMINI_API_KEY` — SECRET — server-only.
- `ANTHROPIC_API_KEY` — SECRET — server-only.
- `OPENAI_API_KEY` — SECRET — server-only; also used for RAG embeddings.

Market Data / FinTech / Macro / News

- `ALPHA_VANTAGE_KEY` — SECRET — Alpha Vantage.
- `FMP_API_KEY` — SECRET — Financial Modeling Prep; declared in Render manifest but absent from root `.env.example` at this snapshot: documentation drift finding.
- `COINMARKETCAP_API_KEY` — REMOVED 2026-08-11 — CoinMarketCap integration decommissioned; no longer read anywhere in the codebase, no longer part of the secret file manifest or `.env.example`.
- `COIN_API_KEY` — SECRET — CoinAPI.
- `EODHD_API_KEY` — SECRET — EODHD.
- `TWELVEDATA_API_KEY` — SECRET — Twelve Data.

- `FRED_API_KEY` — SECRET — FRED macro evidence.
- `NEWS_API_KEY` — SECRET — canonical News API variable.
- `News_API_KEY` — REMOVED 2026-08-11 — casing-duplicate fallback deleted from `news-Routes.ts` and both `.env.example` templates; `NEWS_API_KEY` is now the only variable read.
- `KRAKEN_API_KEY` — REMOVED 2026-08-11 — never read by any code path (Kraken adapter only calls the public, unauthenticated `api.kraken.com` ticker); deleted from both `.env.example` templates.
- `KRAKEN_API_SECRET` — REMOVED 2026-08-11 — same finding as `KRAKEN_API_KEY`.
- `API_KEY` — REMOVED 2026-08-11 — generic, unreferenced legacy alias declared alongside Kraken credentials; deleted.
- `API_SECRET` — REMOVED 2026-08-11 — same finding as `API_KEY`.
- Alpaca credential variable(s) — PRODUCTION CONFIG TO VERIFY — reported as present, but exact canonical names are not evidenced by current repository manifest and must be read from the deployment configuration before documentation is finalized.

Supabase

- `VITE_SUPABASE_URL` — PUBLIC CONFIG — browser-visible endpoint.
- `VITE_SUPABASE_PUBLISHABLE_KEY` — PUBLIC/PUBLISHABLE credential.
- `VITE_SUPABASE_ANON_KEY` — PUBLIC/PUBLISHABLE legacy/client credential; usage and coexistence with publishable key to review.
- `SUPABASE_SECRET_KEY` — SECRET — server-only.
- `SUPABASE_SERVICE_ROLE_KEY` — HIGH-PRIVILEGE SECRET — server-only, never browser-exposed.

Stripe

- `STRIPE_SECRET_KEY` — SECRET.
- `VITE_STRIPE_PUBLISHABLE_KEY` / deployment `STRIPE_PUBLISHABLE_KEY` — PUBLIC/PUBLISHABLE; naming drift to resolve.
- `STRIPE_WEBHOOK_SECRET` — SECRET.
- `STRIPE_PRICE_ID_STARTER` — CONFIG ID, non-secret.
- `STRIPE_PRICE_ID_PRO` — CONFIG ID, non-secret.
- `STRIPE_PRICE_ID_ENTERPRISE` — CONFIG ID, non-secret.
- `STRIPE_PRICE_ID_STARTER_YEARLY` — CONFIG ID declared in secondary env example; deployment presence to verify.
- `STRIPE_PRICE_ID_PRO_YEARLY` — CONFIG ID declared in secondary env example; deployment presence to verify.

SMTP

- `SMTP_HOST` — CONFIG.
- `SMTP_PORT` — CONFIG.

- `SMTP_USER` — SENSITIVE CONFIG.
- `SMTP_PASSWORD` — SECRET.
- `SMTP_FROM` — CONFIG.

Security / Runtime

- `TOTP_ENCRYPTION_KEY` — HIGH-PRIVILEGE SECRET.
- `METRICS_TOKEN` — SECRET; declared in Render manifest, purpose/consumer to trace.
- `APP_URL` — CONFIG.
- `AI_STUDIO_ORIGIN` — RETIRED; explicitly removed by the ADR-0009 addendum (2026-08-11, Q6) and must not be restored.
- `VITE_GA_MEASUREMENT_ID` — PUBLIC CONFIG, not a secret.
- `ORCHESTRATOR_ADMIN_TOKEN` — RETIRED; explicitly removed by ADR-0003.5 and must not be restored.

Social OAuth

- `YOUTUBE_CLIENT_ID`, `YOUTUBE_CLIENT_SECRET`
- `TIKTOK_CLIENT_ID`, `TIKTOK_CLIENT_SECRET`
- `INSTAGRAM_CLIENT_ID`, `INSTAGRAM_CLIENT_SECRET`
- `X_CLIENT_ID`, `X_CLIENT_SECRET`
- `FACEBOOK_CLIENT_ID`, `FACEBOOK_CLIENT_SECRET`

Client IDs are identifiers; client secrets are SECRET and server-only.

3. Configuration Drift Findings

1. **Kraken was missing from the first ESS-0015 inventory** although repository evidence exists. Corrected in this snapshot.
2. **News API casing duplicate resolved (2026-08-11):** `News_API_Key` was a noncanonical fallback in `newsRoutes.ts`; usage scan confirmed `NEWS_API_KEY` alone is sufficient, and the duplicate has been removed from code and both `.env.example` templates.
3. **Generic `API_KEY` / `API_SECRET` resolved (2026-08-11):** call-site scan found zero references anywhere in the codebase (Kraken adapter only hits the public, unauthenticated ticker endpoint). Removed along with the equally unreferenced `KRAKEN_API_KEY` / `KRAKEN_API_SECRET` from both `.env.example` templates.
4. **FMP drift:** `FMP_API_KEY` is declared by `render.yaml`, but not by the root `.env.example` snapshot.
5. **Stripe publishable-key drift:** `.env.example` uses `VITE_STRIPE_PUBLISHABLE_KEY`; `render.yaml` declares `STRIPE_PUBLISHABLE_KEY`. Call-site/build-time usage must determine the canonical name.
6. **Two env example files exist** (`.env.example` and `server/.env.example`) with divergent variable sets. They must be reconciled after inventory, not used as independent sources of truth.

7. **Alpaca:** credential presence is reported in production, but exact variable names and runtime consumer still require deployment/config evidence.

4. Inventory Completion Rule

Before any FinTech-provider architecture ADR is accepted, ESS-0015 QA must reconcile four evidence planes:

1. repository variable declarations and call sites (`process.env` , `getCleanEnv` , `import.meta.env`),
2. deployment environment declarations (Render),
3. Supabase secrets / Edge Function configuration where applicable,
4. external-provider/API inventory and runtime request evidence.

A variable is classified as `ACTIVE` only when both configuration and runtime/code consumer evidence exist. Variables with only configuration evidence are `CONFIGURED_UNVERIFIED` ; code-only variables are `REQUIRED_NOT_DEPLOYMENT_VERIFIED` ; obsolete aliases are `LEGACY_CANDIDATE` .

5. Immediate QA priorities

1. Complete repository-wide environment-variable extraction and map every variable to every call site.
2. Reconcile Render environment names against `.env.example` without exposing values.
3. Verify exact Alpaca production variable names and consumer.
4. Test Kraken, News API and every market-data provider for auth, entitlement, latency, freshness, provenance, rate limit and failure semantics.
5. Remove no key/alias until negative usage evidence is recorded.
6. Only after the inventory is complete, score providers and make the new FinTech API architecture decision.

Decision constraint

No provider listed as `EVALUATE` becomes a new canonical FinTech data path until the ESS-0015 inventory, QA report and subsequent provider architecture ADR are complete.

CAPITAL-AI API QA Report

Quelle: docs/architecture/api/API_QA_REPORT.md

CAPITAL-AI API QA Report

Status: INITIAL AUDIT SEED

Governance: ESS-0015

Snapshot: 2026-08-03

1. Scope

Initialer QA-Seed aus aktuellem GitHub-Repository und produktiver Supabase-Instanz. Dieser Bericht ist noch keine vollständige Provider-Freigabe; er definiert priorisierte Findings und die Tests, die vor der neuen FinTech-API-Architekturentscheidung abgeschlossen werden müssen.

2. Supabase Live Findings

QA-SUPA-001 — Edge Functions ohne JWT-Verifikation

Produktiv aktiv:

- `stripe-setup v3` — `verify_jwt=false`
- `stripe-webhook v4` — `verify_jwt=false`
- `stripe-worker v3` — `verify_jwt=false`

Bewertung: REVIEW REQUIRED. `verify_jwt=false` ist für Webhooks nicht per se unsicher, wenn die Funktion eine starke alternative Authentizitätsprüfung (z. B. Stripe-Signatur) implementiert. Für `stripe-setup` und `stripe-worker` muss der konkrete alternative Schutz ebenfalls codebasiert nachgewiesen werden.

Required tests: unsigned request, invalid signature/token, replay, malformed payload, privilege boundary, idempotency.

QA-SUPA-002 — `screening_slo_evidence` RLS ohne Policy

Security Advisor meldet `rls_enabled_no_policy` für `public.screening_slo_evidence`.

Bewertung: INFO / intentionality must be proven. Wenn die Tabelle bewusst service-role-only und fail-closed ist, muss dies im Inventory als Access Contract dokumentiert und per negativer anon/authenticated-Abfrage getestet werden.

Reference: https://supabase.com/docs/guides/database/database-linter?lint=0008_rls_enabled_no_policy

QA-SUPA-003 — Leaked Password Protection

Security Advisor meldet `auth_leaked_password_protection` als WARN.

Bewertung: bestehende plan-/risk-acceptance-relevante Abweichung; nicht Teil der Fin-Tech-Datenproviderentscheidung, aber als Plattform-API-Risiko im Inventory zu führen.

Reference: <https://supabase.com/docs/guides/auth/password-security#password-strength-and-leaked-password-protection>

QA-SUPA-004 — RLS Performance / InitPlan

Performance Advisor meldet mehrere Policies, die `auth.<function>()` bzw. `current_setting()` zeilenweise neu evaluieren, u. a. bei `subscriptions`, `profiles`, IAM-/Security-/Quota-/Compliance-Tabellen.

Bewertung: PERFORMANCE DEBT. Vor hoher API-Last sollen Policies auf `(select auth.<function>())`-Muster geprüft werden, ohne Semantik oder Authorization zu schwächen.

Reference: https://supabase.com/docs/guides/database/database-linter?lint=0003_auth_rls_initplan

QA-SUPA-005 — Unindexed Foreign Keys

Advisor meldet fehlende Covering Indexes u. a. für `audit_logs_iam.actor_user_id`, `audit_logs_iam.target_user_id`, `security_events.actor_user_id`.

Bewertung: INFO / scalability review.

Reference: https://supabase.com/docs/guides/database/database-linter?lint=0001_unindexed_foreign_keys

3. FinTech Market Data QA Priorities

Binance

- Contract: 24h ticker + klines schema
- symbol allowlist / normalization
- timeout / 429 / 5xx
- stale timestamp detection
- no fabricated fallback values
- compare selected symbols against second provider snapshots
- latency p50/p95

CoinGecko

- snapshot/history consistency
- freshness labels
- rate-limit behavior
- consensus interaction with alternate sources

FMP

- index symbol mapping accuracy
- quote/history freshness
- entitlement/plan failure behavior
- missing-symbol handling

Alpha Vantage

- fundamentals freshness
- provider rate-limit and throttling
- schema drift / null fields
- overlap with FMP/Alpaca candidates

Alpaca candidate

No active code integration was located in the current repository snapshot. Therefore Alpaca remains `EVALUATE`, irrespective of whether a production secret has already been provisioned. Before activation, QA must establish exact product (Market Data API vs Trading API), scopes, feed entitlement, asset coverage, rate limits, data timestamps, cost and fallback semantics.

4. AI Provider QA

For Anthropic/OpenAI/Gemini routing:

- prove actual fallback order and provider attribution
- schema-equivalence tests for structured outputs
- no silent semantic downgrade
- token/cost ledger
- timeout/rate-limit behavior
- prompt/version attribution
- AI output must not overwrite market provenance

5. Payment/API QA

Stripe Render backend and Supabase Stripe Edge Functions must be tested as one end-to-end topology:

- identify canonical webhook ingestion path
- prove event idempotency across all active paths
- invalid-signature rejection
- duplicate event replay
- checkout metadata/user-id integrity
- subscription state consistency between Stripe and Supabase

- no billing mutation from unauthenticated public endpoints

6. Required automated test suites

Target structure:

- tests/api/contracts/
- tests/api/auth/
- tests/api/failure/
- tests/api/market-data/
- tests/api/webhooks/
- tests/api/security/
- tests/api/performance/

Every production interface must have at least one contract test and one negative/failure-path test before `qa_status=PASS`.

7. Decision readiness

Current status: **NOT DECISION READY**.

The next provider architecture ADR must not be accepted until the ESS-0015 inventory reaches full production coverage and all FinTech market-data interfaces have measured QA results.

CAPITAL-AI Autonomous Content Engine Architecture

Quelle: docs/architecture/AUTONOMOUS_CONTENT_ENGINE_ARCHITECTURE.md

CAPITAL-AI Autonomous Content Engine Architecture

Document ID: ARCH-CONTENT-0001 Status: DRAFT — IMPLEMENTATION NOT AUTHORIZED
Date: 2026-08-12 Baseline: `main@3a733f2b6efeffb3013fbb2c558b5ef4c185125e` Owner: Platform Director / Repository Owner Related: ESS-0014, ESS-0019, ESS-0022, ADR-0026, ADR-0068

1. Executive decision

CAPITAL-AI extends the existing SocialMediaEngine with an Autonomous Content Engine instead of replacing the existing OAuth/publishing stack with a second social-media platform.

Existing distribution remains authoritative for:

- OAuth and account state;
- token storage;
- user/Founder/Owner access checks;
- publish API calls;
- platform-specific publishing;
- publish history/evidence.

The new architecture adds the missing upstream layers:

Source/Evidence -> Content Planning -> Generation -> Validation -> Provenance/Compliance -> Human Approval -> Media Rendering -> Asset Registry -> Existing Social Publishing -> Analytics Feedback.

2. Verified repository boundary

Current repository contracts already contain:

- `SocialMediaQuestionnaire` ;
- `ScriptScene` ;
- `PodcastDialogueEntry` ;
- `GeneratedMediaItem` ;
- `SocialMediaSeriesPackage` ;
- `marketingPack` ;
- `audioSsml` ;
- `thumbnailPrompt` ;
- optional `mediaUrl` .

The current client service explicitly documents that `generateSeries`, fallback package generation and `/api/social-media/generate` are not implemented.

The existing platform publishers already perform real external calls. YouTube/TikTok/Instagram require a real reachable media asset; X/Facebook text publishing can work without rendered video.

Therefore the architecture evolves existing contracts instead of creating a duplicate social stack.

3. Target planes

3.1 Evidence Plane

Sources:

- verified market/scoring data;
- approved news/research sources;
- SEO/Search Console/GA4 evidence;
- approved product/documentation sources;
- Human input.

Output: immutable/referential `ContentSourceReference` records with evidence digest and freshness metadata.

3.2 Content Intelligence Plane

Components:

- `ContentPlanner` ;
- `SourceEvidenceResolver` ;
- `PromptTemplateRegistry` ;
- `ContentGenerationOrchestrator` ;
- `PlatformVariantPlanner` ;
- provider-neutral text-generation interface.

The content plane cannot publish.

3.3 Trust & Compliance Plane

Components:

- `FinancialClaimValidator` ;
- `ContentPolicyGate` ;
- `AIContentProvenance` ;
- `DisclosurePolicy` ;
- `ContentApprovalService` .

This plane decides whether content is `ALLOW`, `REVIEW_REQUIRED` or `DENY` for the requested use.

3.4 Media Plane

Provider-neutral interfaces:

- `ImageGenerationProvider`;
- `ImageEditProvider`;
- `TextToSpeechProvider`;
- `VideoRenderingProvider`;
- `AssetValidator`;
- `AssetRegistryService`.

Candidate workers may include FLUX-compatible image generation, an image-edit model, TTS providers, a MoneyPrinterTurbo-compatible short-video sidecar and deterministic Remotion templates.

Media workers have no publishing or infrastructure authority.

3.5 Distribution Plane

Existing CAPITAL-AI components remain authoritative:

- `SocialMediaGeneratorService` publishing methods;
- `socialMediaRoutes.ts`;
- `platformPublishers.ts`;
- `tokenStore.ts`;
- `publishLog.ts`;
- access-control/OAuth components.

A new `ContentPublishingBridge` maps an approved content package/asset into the existing `PublishRequestPayload` rather than bypassing the publisher.

3.6 Feedback Plane

Inputs:

- Search Console;
- GA4;
- approved platform metrics;
- publish history;
- content metadata/provenance.

Output:

- content-performance score;
- template/provider recommendations;

- next-plan recommendation.

Feedback cannot self-grant publish authority.

4. Proposed repository structure

```
src/platform/SocialMediaEngine/
├── Contracts/
│   ├── ContentPackage.ts
│   ├── ContentSource.ts
│   ├── ContentProvenance.ts
│   ├── GeneratedAsset.ts
│   ├── ContentApproval.ts
│   └── ProviderContracts.ts
├── Planning/
│   ├── ContentPlanner.ts
│   ├── PlatformVariantPlanner.ts
│   └── PromptTemplateRegistry.ts
├── Generation/
│   ├── ContentGenerationOrchestrator.ts
│   ├── TextGenerationProvider.ts
│   └── SourceEvidenceResolver.ts
├── Compliance/
│   ├── ContentPolicyGate.ts
│   ├── FinancialClaimValidator.ts
│   ├── ProvenanceValidator.ts
│   └── DisclosurePolicy.ts
├── Rendering/
│   ├── MediaRenderer.ts
│   ├── AssetValidator.ts
│   └── ProviderRegistry.ts
├── Providers/
│   ├── FluxImageAdapter.ts
│   ├── ImageEditAdapter.ts
│   ├── TtsProviderAdapter.ts
│   ├── MoneyPrinterTurboAdapter.ts
│   └── RemotionAdapter.ts
├── Services/
│   ├── ContentPackageService.ts
│   ├── ContentApprovalService.ts
│   ├── AssetRegistryService.ts
│   └── ContentPublishingBridge.ts
server/socialMedia/
├── generationRoutes.ts
├── contentStore.ts
├── contentApproval.ts
├── assetStore.ts
├── contentAudit.ts
├── oauthExchange.ts           # existing
├── platformPublishers.ts     # existing
├── publishLog.ts             # existing
├── tokenStore.ts             # existing
├── accessControl.ts          # existing
```

Exact paths remain subject to implementation review; this document does not authorize file creation outside a future work package.

5. Core ContentPackage contract

Target conceptual contract:

```
export type ContentAuthorship = 'human' | 'ai_assisted' | 'ai_generated';

export type ContentLifecycleState =
  | 'DRAFT'
  | 'GENERATED'
```

```

| 'VALIDATING'
| 'REVIEW_REQUIRED'
| 'APPROVED'
| 'RENDERING'
| 'READY'
| 'PUBLISHING'
| 'PUBLISHED'
| 'PARTIAL'
| 'FAILED'
| 'STALE';

export interface ContentSourceReference {
  sourceId: string;
  sourceType: 'market_data' | 'scoring' | 'news' | 'seo' | 'document' | 'manual';
  sourceUri?: string;
  sourceTimestamp?: string;
  evidenceHash: string;
  dataBasis?: 'market-data' | 'verified-document' | 'manual';
}

export interface ContentProvenance {
  authorship: ContentAuthorship;
  provider?: string;
  model?: string;
  promptTemplateId?: string;
  promptTemplateVersion?: string;
  generatedAt?: string;
  sourceReferences: ContentSourceReference[];
  generationRunId?: string;
  humanReviewed: boolean;
  reviewedBy?: string;
  reviewedAt?: string;
  contentHash: string;
}

```

The existing `SocialMediaSeriesPackage` should be migrated/evolved through explicit compatibility rules rather than silently removed.

6. GeneratedAsset contract

Conceptual target:

```

export interface GeneratedAsset {
  id: string;
  contentPackageId: string;
  type: 'image' | 'thumbnail' | 'audio' | 'short_video' | 'long_video';
  provider: string;
  model?: string;
  mimeType: string;
  storageKey: string;
  sha256: string;
  width?: number;
  height?: number;
  durationMs?: number;
  status: 'GENERATING' | 'VALIDATING' | 'READY' | 'FAILED' | 'QUARANTINED';
  delivery?: {
    serverFetchUrl?: string;
    providerPullUrl?: string;
    expiresAt?: string;
  };
}

```

`mediaUrl` remains a publishing compatibility field, but internally it must resolve from a validated asset rather than an arbitrary untrusted URL.

7. Provider isolation

Domain code only depends on interfaces.

```
export interface ImageGenerationProvider {  
  generate(request: ImageGenerationRequest): Promise<GeneratedAsset>;  
}  
  
export interface TextToSpeechProvider {  
  synthesize(request: SpeechRequest): Promise<GeneratedAsset>;  
}  
  
export interface VideoRenderingProvider {  
  render(request: VideoRenderRequest): Promise<GeneratedAsset>;  
}
```

Provider adapters receive minimal task input and cannot access:

- Social OAuth tokens;
- GitHub credentials;
- Supabase privileged credentials;
- Stripe credentials;
- Owner session/TOTP/passkey material;
- Render/IONOS unrestricted credentials.

8. Media-generation strategy

Images

Use a provider adapter for generative background/illustration assets.

Financial values, scores, disclaimers and brand-critical text should be rendered deterministically using HTML/SVG/Canvas/Remotion rather than trusted to generative image text rendering.

Audio/TTS

Use a provider registry with explicit language/quality benchmark. German finance terms, numbers, percentages, abbreviations, company names, crypto assets and commodity names must be tested before production selection.

Short video

A MoneyPrinterTurbo-compatible worker may be used as an isolated renderer. Its own social publishing features are not part of CAPITAL-AI authority and should not be used.

Deterministic premium/learning video

A Remotion-compatible renderer can produce branded videos from structured data, charts, disclosure text, approved script and voiceover. License review is a required provider gate.

9. Financial content policy

Conceptual decision:

```
export interface ContentComplianceDecision {  
  decision: 'ALLOW' | 'REVIEW_REQUIRED' | 'DENY';  
  financialClaimRisk: 'NONE' | 'LOW' | 'MEDIUM' | 'HIGH';  
  investmentRecommendationRisk: boolean;  
  disclosureRequired: boolean;  
  provenanceComplete: boolean;  
  unsupportedClaims: string[];  
  reasons: string[];  
}
```

DENY examples:

- invented market data;
- no evidence for a concrete quantitative claim;
- synthetic/heuristic value presented as live fact;
- manipulated score;
- unsupported performance promise;
- incomplete provenance;
- approval hash mismatch;
- prompt/retrieval injection attempting policy/tool control.

REVIEW_REQUIRED examples:

- forecast;
- ranking;
- asset recommendation-like statement;
- scoring conclusion used for persuasion;
- regulatory/compliance-sensitive comparison.

10. AI content transparency

Each content package records at least:

- content id/version;
- authorship classification;
- provider/model;
- generation timestamp;
- source references/digest;
- prompt template id/version or safe hash;
- human review state;
- content hash;
- final asset hashes;
- target platforms;

- approval reference.

Suggested visible disclosure policy:

- human: no AI disclosure by default;
- ai_assisted: KI-unterstützt erstellt, redaktionell geprüft;
- ai_generated: KI-generierter Inhalt, vor Veröffentlichung geprüft.

Exact legal/platform disclosure language requires separate compliance review before production.

11. Approval contract

Conceptual target:

```
export interface ContentApproval {
  approvalId: string;
  contentPackageId: string;
  approvedContentHash: string;
  approvedAssetHashes: string[];
  approvedPlatforms: string[];
  approverUserId: string;
  decision: 'APPROVED' | 'REJECTED';
  issuedAt: string;
  expiresAt?: string;
  usedAt?: string;
}
```

Material content/asset/platform changes invalidate the approval.

12. Storage model

Proposed additive logical data domains:

- social_content_packages;
- social_content_assets;
- social_content_approvals.

Existing domains remain authoritative:

- social_media_accounts;
- social_media_oauth_states;
- social_media_publish_log.

No production database mutation is authorized by this architecture draft. Migrations, storage buckets and secrets require the normal DevelopmentChain mutation handoff and Owner-controlled production gate.

13. Asset delivery / SSRF protection

Before any server-side fetch or provider-pull URL is accepted:

- require HTTP(S);

- validate host and resolved IP;
- deny localhost/loopback/private/link-local/cloud-metadata destinations;
- validate redirects again;
- enforce Content-Length/timeout limits;
- validate MIME and magic bytes;
- hash the asset;
- register it before publishing.

`file:`, `data:`, `ftp:` and equivalent unsupported schemes are denied.

This is particularly important because the existing YouTube path fetches `mediaUrl` server-side.

14. API evolution

Proposed new endpoints:

- `POST /api/social-media/generate;`
- `GET /api/social-media/content/:id;`
- `POST /api/social-media/content/:id/validate;`
- `POST /api/social-media/content/:id/approve;`
- `POST /api/social-media/content/:id/reject;`
- `POST /api/social-media/content/:id/render;`
- `GET /api/social-media/content/:id/assets.`

Existing publishing remains:

- `POST /api/social-media/publish.`

`ContentPublishingBridge` performs the controlled translation to the existing publish contract.

15. Event and audit model

Before adding any event, check the canonical Event Mesh catalog for semantic duplicates.

Candidate domain events:

- `ContentGenerationRequestedEvent;`
- `ContentGeneratedEvent;`
- `ContentValidationFailedEvent;`
- `ContentReviewRequiredEvent;`
- `ContentApprovedEvent;`
- `ContentRejectedEvent;`
- `AssetRenderingRequestedEvent;`

- `AssetRenderedEvent` ;
- `AssetValidationFailedEvent` ;
- `ContentPublishRequestedEvent` ;
- `ContentPublishedEvent` ;
- `ContentPublishFailedEvent` .

Keep three evidence domains separate:

1. content business state;
2. publishing evidence (`social_media_publish_log`);
3. agent/authorization audit (`agent_audit_events`).

16. Observability

Correlate:

- request/trace/generation run;
- content package id;
- source count/digest;
- provider/model/template version;
- generation/validation/render latency;
- provider token/compute/cost metadata where safe;
- content hash and asset hashes;
- approval id;
- publish log ids.

Do not persist full prompts/diffs/raw sensitive requests as generic telemetry. Prefer template ids/versions and safe hashes consistent with existing redaction policy.

17. Threat model

Required threats/tests include:

- prompt/retrieval injection;
- hallucinated financial claims;
- approval replay/stale approval;
- malicious media worker;
- cross-tenant asset access;
- SSRF through media URLs;
- poisoned analytics feedback;
- provider/supply-chain compromise;
- unauthorized platform publishing;
- audit outage causing side effects;

- content/asset mutation after review.

18. Feature flags

Recommended default rollout flags:

```
CONTENT_GENERATION_ENABLED=false  
CONTENT_IMAGE_GENERATION_ENABLED=false  
CONTENT_TTS_ENABLED=false  
CONTENT_VIDEO_RENDERING_ENABLED=false  
CONTENT_PROVIDER_FLUX_ENABLED=false  
CONTENT_PROVIDER_MPT_ENABLED=false  
CONTENT_PROVIDER_REMOTION_ENABLED=false  
CONTENT_AUTO_PUBLISH_ENABLED=false
```

Each capability is enabled independently after its roadmap gate.

`CONTENT_AUTO_PUBLISH_ENABLED=false` remains mandatory through the initial implementation/pilot phases.

19. Acceptance criteria

The architecture is not production-ready until at least:

1. existing OAuth/account/publishing behavior has no regression;
2. X/Facebook text publishing remains functional;
3. no model/renderer receives prohibited credentials;
4. quantitative claims without evidence are blocked;
5. incomplete provenance prevents publish;
6. content/asset mutation invalidates approval;
7. cross-user access is denied;
8. media URL path is SSRF-hardened;
9. provider outage produces honest failure, no fake success;
10. YouTube/TikTok/Instagram remain fail-closed without valid assets;
11. provider replacement does not change domain contracts;
12. content and asset hashes are reproducible;
13. audit/event/publish evidence can be correlated;
14. production configuration/database/provider mutations remain separately Owner-gated;
15. auto-publish remains disabled until separately approved.

20. Recommended implementation order

Contracts -> text generation -> provenance/compliance/approval -> asset infrastructure -> image/TTS -> short-video rendering -> publishing bridge -> UI -> analytics feedback.

This order gives useful X/Facebook content generation before expensive media infrastructure and keeps public publishing behind the trust plane.

CAPITAL-AI Documentary D0 — Baseline, Manifest & Version Authority

Quelle: docs/architecture/DOCUMENTARY_D0_BASELINE_VERSION_AUTHORITY.md

CAPITAL-AI Documentary D0 — Baseline, Manifest & Version Authority

Status: IMPLEMENTED IN DRAFT

Date: 2026-08-10

Roadmap: DOCUMENTARY_EVENT_VALUE_CHAIN_ROADMAP.md

Primary Authority: ESS-0010 Documentary Engine

Related: ESS-0004 Version Manager, ESS-0011 Traceability, ESS-0012 Documentation Governance

Ziel

D0 schafft eine belastbare Ausgangsbasis für die weitere Documentary-Engine-Implementierung. Der Schritt implementiert noch keine Core Engine und keine neuen Events. Er beseitigt Versionsdrift, macht den realen Implementierungsgrad maschinenlesbar und trennt Component-, Document-Schema- und Platform-Version voneinander.

Verifizierter Ausgangsstand

D0 basiert auf dem nach Roadmap-PR #160 verifizierten `main`-Commit `0657e6a224d-c8a70ab73500297cd000c6cd064ac`. Main-CI #675 war einschließlich TypeScript, Unit-Tests, Production Build, CSP, Deployment Readiness, Docker Build, Runtime-Metadaten und Render-Production-Gate erfolgreich.

Version Authorities

Documentary Component Version

Authority: `src/platform/Documentary/manifest.json#version`

Aktueller Wert: `1.1.0`.

Die README darf die Version anzeigen, ist aber keine zweite Authority. Ein Regressionstest erzwingt die Übereinstimmung.

Documentary Document Schema Version

Authority: `src/platform/Documentary/Versioning/DocumentaryVersion.ts#DOCUMENTARY_DOCUMENT_SCHEMA_VERSION`

Initialer Wert: `1.0.0`.

Diese Version beschreibt ausschließlich das Schema zukünftiger Documentary Document Models. Sie darf unabhängig von Component- und Platform-Version evolvieren.

CAPITAL-AI Platform Version

Authority Chain: `src/platform/VersionManager/platformVersionAuthority.ts#getPlatformVersion`
→ `package.json#version`.

Documentary erzeugt keine eigene Plattformversion. Der neue side-effect-freie Version-Manager-Adapter liest ausschließlich die bereits durch GOV-VER-001/GOV-VER-002 geschützte Repository-Version.

Implementierungsbaseline

`src/platform/Documentary/Architecture/documentary-baseline.json` trennt real implementierte von lediglich vorbereiteten Bereichen.

Implementiert: - Contracts; - Documentation; - Versioning.

Noch nicht als Documentary Runtime implementiert: - Discovery; - Engine; - Events; - Generators; - Governance; - Interfaces; - Knowledge; - Mermaid; - Migration; - Models; - Plugins.

`Architecture` enthält derzeit Baseline-/Architekturmetadaten und ist selbst kein Beleg für eine laufende Engine.

Manifest Integrity

Das Documentary-Manifest wird in D0 auf Component-Version `1.1.0`, Partial-Implementation-Metadaten, explizite Version Authorities, reale Contracts/Tests/Dokumentation sowie die semantische Version-Manager-Abhängigkeit synchronisiert.

Das Vorhandensein eines Placeholder-Verzeichnisses darf nicht mehr als Implementierungsbeleg interpretiert werden. Der D0-Test verlangt für jedes als `implementedArea` deklarierte Gebiet mindestens eine reale TypeScript-Runtime-Datei.

Schutzgrenzen

- keine Documentary Core Engine in D0;
- keine neuen Events;
- kein zweiter Version Manager;
- keine zweite Platform-Version-Authority;
- keine automatische Dokumentmutation;
- keine Änderung an Stripe, Supabase, Render, APIs, DB-Schemas oder ENV Keys;
- kein zusätzlicher GitHub-Actions-Workflow;
- bestehende CI-Kostenrichtlinie bleibt bindend.

Exit-Kriterien

- [x] README-/Manifest-Versionsdrift beseitigt;
- [x] Component-/Schema-/Platform-Version semantisch getrennt;
- [x] Platform-Version über Version Manager Authority angebunden;
- [x] realer Documentary-Implementierungsgrad maschinenlesbar;
- [x] D0-Regressionstests implementiert;
- [] Draft-PR-CI und Governance erfolgreich;
- [] Merge und main-CI-/Docker-/Render-Gate erfolgreich.

Nächster Schritt

Nach produktiv verifiziertem D0 folgt D1 — Code Integration & Discovery. D1 muss read-only Code Evidence mit Commit-SHA, Pfad, Symbol, Component-ID sowie Import-/Export-/Contract-/Route-/Event-Beziehungen erzeugen.

Documentary D1 — Code Integration & Discovery

Quelle: docs/architecture/DOCUMENTARY_D1_CODE_INTEGRATION_DISCOVERY.md

Documentary D1 — Code Integration & Discovery

Status: IMPLEMENTED IN DRAFT

Date: 2026-08-10

Authority: ESS-0010, ESS-0011, ESS-0012

Foundation: D0 / PR #161 / main `0d3ed9c5fde536435c1a21d6f4e0ca4c92385b7b`

Deutsch

D1 führt eine read-only Code-Evidence-Schicht für Documentary ein. Ziel ist nicht, Source Code zu verändern, sondern den aktuellen Repository-Zustand reproduzierbar an einen Git-Commit zu binden.

Evidence-Modell

Jedes Evidence-Element enthält: - `evidenceId` als stabilen Hash aus Kind, Commit, Pfad und Symbol; - `kind`: module, export, contract, route, event, manifest oder dependency; - `componentId`; - `sourceCommit`; - Repository-Pfad; - optional Symbol und Detail.

Discovery Scope

Der Scanner erfasst: 1. Source-Module unter den unterstützten JS/TS-Erweiterungen; 2. exportierte Klassen, Funktionen, Konstanten, Interfaces, Types und Enums; 3. Type-/Interface-Contracts; 4. Express-/Router-Routen mit statischem Pfad; 5. Event-Symbole nach dem kanonischen `*Event`-Muster; 6. Platform-`manifest.json`-Dateien; 7. deklarierte Manifest-Abhängigkeiten.

Determinismus und Provenance

Ein gültiger Git-Commit-SHA ist verpflichtend. Ohne Commit schlägt die Discovery fail-closed fehl. Der Output enthält keinen Laufzeit-Timestamp und wird stabil sortiert; derselbe Repository-Zustand und derselbe Commit erzeugen damit dieselbe Evidence Map.

Grenzen

- keine AST-basierte Mutation;
- keine Dateiänderung durch Discovery;
- keine autonome Dokumentgenerierung;
- keine Event-Publikation;
- keine Änderung von APIs, DB-Schemas, ENV Keys, Stripe, Supabase oder Render;
- keine zweite Traceability- oder Event-Authority.

Regex Discovery ist bewusst die erste Evidence-Schicht. Eine spätere TypeScript-Compiler-/AST-Analyse darf die Präzision erhöhen, muss jedoch denselben read-only Evidence Contract bedienen.

Exit-Kriterien

- [x] Code-Evidence-Contract implementiert;
- [x] Module/Exports/Contracts/Routes/Events/Manifeste/Dependencies erkennbar;
- [x] Commit-Provenance verpflichtend;
- [x] deterministische Evidence IDs und Sortierung;
- [x] Regressionstests implementiert;
- [] Draft-CI und Governance grün;
- [] Merge und main-CI-/Docker-/Render-Gate grün.

English

D1 introduces a read-only repository code-evidence layer for Documentary. Evidence is bound to a required source commit and covers modules, exports, contracts, static routes, event symbols, platform manifests and declared dependencies. The scanner is deterministic for the same repository state and commit, performs no code mutation, and creates no new event or traceability authority.

D2 — Documentary Core Engine

Quelle: docs/architecture/DOCUMENTARY_D2_CORE_ENGINE.md

D2 — Documentary Core Engine

Status: IMPLEMENTED IN DRAFT

Authority: ESS-0010 / ESS-0011 / ESS-0012

Basis: D1 Code Evidence + D3 Document Models/Provenance + E0/E3 Event Version Evidence

Ziel

D2 führt `DocumentaryEngine` als deterministische Orchestrierungsschicht ein. Die Engine erzeugt ausschließlich strukturierte `DocumentaryDocument`-Models und keine freien Markdown-Texte.

Input Contract

Pflichtfelder sind `correlationId`, Document Identity/Type, vollständiger Source Commit, D1 `CodeEvidenceMap`, Vocabulary Concept IDs, Traceability IDs, D3 Version Context sowie Inhalt und Titel. Optionale zusätzliche Authority-Provenance kann ESS-, ADR-, Vocabulary-, Event- oder manuelle Evidence tragen.

Fail-closed Regeln

- Source Commit muss ein vollständiger 40-stelliger Git SHA sein.
- Evidence Map und alle Evidence-Elemente müssen zum selben Commit gehören.
- Mindestens eine Code-Evidence, Concept-ID und Traceability-ID ist erforderlich.
- Version Context muss vollständig sein.
- Dieselbe `correlationId` ist nur für denselben semantischen Request idempotent wiederwendbar.

Lifecycle Boundary

Die Engine erzeugt ausschließlich `reviewStatus: generated`. Review/Approval, Event-Publishing, Rendering, Knowledge Projection und geschützte Dokumentfreigaben bleiben getrennte Workstreams.

Versionierung

Documentary Component Version: `1.4.0`. Document Schema und Platform Version bleiben über die bestehenden D0 Authorities getrennt.

Documentary D3 — Document Models, Schemas & Provenance

Quelle: docs/architecture/DOCUMENTARY_D3_DOCUMENT_MODELS_PROVENANCE.md

Documentary D3 — Document Models, Schemas & Provenance

Status: IMPLEMENTED IN DRAFT

Authority: ESS-0010, ESS-0011, ESS-0012

D3 definiert den kanonischen strukturierten Documentary-Output. Dokumente tragen Document-ID/-Typ, Schema-/Component-/Platform-Versionen, Source Commit, Lifecycle-Status, Concept-/Traceability-IDs, Provenance-Referenzen und einen deterministischen SHA-256-Fingerprint.

Unterstützte Provenance-Arten sind Code, ESS, ADR, Vocabulary, Event und manuelle Evidence. Code-Provenance erfordert Git-Commit und Repository-Pfad. Dokumente ohne Provenance werden fail-closed abgewiesen.

Der Fingerprint umfasst stabile Inhalte, Versionen, Source Commit, semantische IDs und Provenance. Lifecycle-Metadaten wie `generatedAt` und `reviewStatus` verändern den Fingerprint nicht.

D3 implementiert noch keine Core Engine, Renderer, Event-Publikation, automatische Approval-Transitions oder Source-Code-Mutation.

Documentary D4 — Review & Lifecycle Governance

Quelle: docs/architecture/DOCUMENTARY_D4_REVIEW_LIFECYCLE_GOVERNANCE.md

Documentary D4 — Review & Lifecycle Governance

Status: IMPLEMENTED IN DRAFT Date: 2026-08-10 Basis: D5/E1/E4 / PR #169 / main

020c89abf4fbc9b94128b33e77e780e48b1e1c fb

Deutsch

D4 führt einen expliziten, evidence-basierten Lifecycle für Documentary-Dokumente ein. Der zulässige Hauptpfad lautet `generated -> reviewed -> approved`. Nach Freigabe sind ausschließlich kontrollierte Übergänge nach `superseded` oder `archived` zulässig; `superseded -> archived` ist ebenfalls erlaubt.

Jeder Übergang verlangt Actor-ID, Aktion, Zeitpunkt und mindestens eine Evidence-Referenz. Übersprungene Status, falsche Aktionen und fehlende Evidence werden fail-closed blockiert. Der Dokument-Fingerprint bleibt bei reinen Lifecycle-Transitions unverändert, weil Review-Status kein Content-Feld ist.

D4 führt keine autonome Freigabe ein. Die Governance-Schicht validiert lediglich eine explizit angeforderte Transition. IAM-/Owner-Step-Up und persistente Audit-Speicherung bleiben getrennte Integrationsaufgaben; D4 umgeht diese Grenzen nicht.

English

D4 establishes an explicit evidence-backed Documentary lifecycle. The governed path is `generated -> reviewed -> approved`, followed only by controlled supersede/archive transitions. Every transition requires an actor, action, timestamp and evidence reference. Invalid skips, mismatched actions and evidence-free transitions fail closed. Lifecycle-only changes preserve the content fingerprint.

Documentary D5 + E1 + E4 — Traceability & Event Integration

Quelle: docs/architecture/DOCUMENTARY_D5_E1_E4_TRACEABILITY_EVENTS.md

Documentary D5 + E1 + E4 — Traceability & Event Integration

Status: IMPLEMENTED IN DRAFT Date: 2026-08-10 Basis: D2 / PR #168 / main

c805914e544d63c6cdce1ed8a759b8bc48664fce

Deutsch

D5 verknuepft Documentary-Dokumente mit correlationId, causationId, Source Commit, Document Fingerprint, Concept IDs, Traceability IDs und Provenance-Referenzen. E1 bindet Documentary als kontrollierten Consumer und Producer an den bestehenden EventMesh an. E4 schliesst die End-to-End Evidence Chain zwischen vorgelagertem Event und Documentary Output.

Consumer: RepositoryScannedEvent und PlatformDecisionEvent. Die Event-ID des ausloesenden Events wird als causationId uebernommen; die bestehende correlationId wird unveraendert fortgefuehrt.

Producer: DocumentationGeneratedEvent und DocumentationValidatedEvent. Beide tragen Source Commit, Document Fingerprint, causationId und den vollstaendigen DocumentaryTraceabilityRecord im Payload.

Schutzgrenzen: kein zweiter EventBus, keine zweite Registry, keine automatische Approval-Transition, keine Source-Code-Mutation und keine Aenderungen an Stripe, Supabase, Render, DB-Schemas oder ENV Keys.

English

D5/E1/E4 establishes an evidence-preserving bridge between canonical upstream events, Documentary documents and Traceability. Correlation IDs are preserved end-to-end, the triggering event ID becomes the causation ID, and generated/validated document events carry commit, fingerprint and traceability evidence through the existing EventMesh only.

Documentary D6 — Generators & Renderers

Quelle: docs/architecture/DOCUMENTARY_D6_GENERATORS_RENDERERS.md

Documentary D6 — Generators & Renderers

Status: IMPLEMENTED IN DRAFT Date: 2026-08-10 Basis: D4 / PR #171 / verified main

114356fbfb2a7a3743cfb85f73b04ab673847546

Deutsch

D6 führt eine deterministische Output-Schicht für bereits governierte

`DocumentaryDocument`-Modelle ein. Die Core Engine bleibt die Erzeugungs-Authority; der Renderer transformiert ausschließlich ein vorhandenes Dokument in ein transportierbares Markdown-Artefakt.

Unterstützte Dokumenttypen sind `architecture`, `component`, `api`, `runbook`, `release-evidence` und `handoff`. Jeder Typ besitzt ein eigenes Abschnittsprofil. Für jedes Dokument können deutsche und englische Markdown-Artefakte erzeugt werden. Metadatenüberschriften werden lokalisiert; Inhalt, Fingerprint, Source Commit, Provenance, Concept-IDs und Traceability-IDs bleiben unverändert beziehungsweise deterministisch sortiert.

D6 führt keine automatische Übersetzung fachlicher Inhalte durch. Eine echte bilinguale Content-Generierung bleibt an die vorhandenen Vocabulary-/Documentary-Contracts gebunden und darf nicht durch implizite Modellübersetzung ersetzt werden.

Schutzgrenzen: kein Approval, keine Lifecycle-Transition, keine Persistenz, keine Source-Code-Mutation, kein neuer EventBus, keine neue Version Authority.

English

D6 adds a deterministic output layer for existing governed `DocumentaryDocument` models. The Core Engine remains the creation authority; the renderer only transforms an existing document into a portable Markdown artifact.

Supported document types are `architecture`, `component`, `api`, `runbook`, `release-evidence`, and `handoff`. Each type has a dedicated section profile. German and English Markdown artifacts can be generated with localized metadata labels while preserving content, fingerprint, source commit, provenance, concept IDs, and traceability IDs.

D6 does not autonomously translate domain content and does not perform approval, lifecycle transitions, persistence, source-code mutation, event-bus creation, or version-authority changes.

Documentary D7 — Knowledge Integration

Quelle: docs/architecture/DOCUMENTARY_D7_KNOWLEDGE_INTEGRATION.md

Documentary D7 — Knowledge Integration

Status: IMPLEMENTED IN DRAFT Date: 2026-08-10 Basis: D6 / PR #173 / verified main

890552455370b81c0285fdbc42bb724bb412afde

Deutsch

D7 implementiert die deterministische Übergabegrenze zwischen Documentary und der in ESS-0009 spezifizierten zentralen Knowledge Engine. Aus einem `DocumentaryDocument` wird eine `DocumentaryKnowledgeProjection` mit origin-basiertem Dokumentknoten, gerichteten Beziehungen, Source Commit, Fingerprint, Concept-/Traceability-IDs, Provenance und SHA-256-Prüfsumme abgeleitet.

Die Projektion ist reproduzierbar: identische Dokumente erzeugen identische Nodes, Relationships und Checksums. Laufzeit-Timestamps werden nicht neu erzeugt und beeinflussen die Projection nicht.

D7 implementiert ausdrücklich nicht die zentrale Knowledge Engine.

`src/platform/Knowledge` bleibt gemäß ESS-0009 eine eigenständige Authority und aktuell specification-only. D7 schreibt nicht nach `.ai/knowledge/`, startet keinen Knowledge Build, führt keine zweite Registry ein und verändert keine ESS/ADR/Contracts/Quellcode-Metadaten.

English

D7 implements the deterministic handoff boundary between Documentary and the central Knowledge Engine specified by ESS-0009. A governed `DocumentaryDocument` is projected into origin-backed documentation nodes, directed relationships, source commit, fingerprint, concept/traceability identifiers, provenance references and a SHA-256 checksum.

The projection is deterministic and side-effect free. It does not persist knowledge, start a Knowledge build, introduce a parallel graph or mutate source architecture artifacts.

CAPITAL-AI Documentary Engine & Event-Driven Value Chain Roadmap

Quelle: docs/architecture/DOCUMENTARY_EVENT_VALUE_CHAIN_ROADMAP.md

CAPITAL-AI Documentary Engine & Event-Driven Value Chain Roadmap

Status: ACTIVE

Date: 2026-08-10

Primary Authority: ESS-0010 Documentary Engine

Related Authorities: ESS-0011 Traceability, ESS-0012 Documentation Governance, ESS-0017 Vocabulary Governance

Foundation: Vocabulary Governance Roadmap COMPLETE / OPERATIONAL

1. Zielbild

Die nächste Entwicklungsstufe macht Documentary zu einer ausführbaren Plattformkomponente statt eines überwiegend vorbereiteten Zielbaums. Dokumentation soll aus Code-, Architektur-, Event-, Versionierungs- und Release-Evidence deterministisch abgeleitet, versioniert, geprüft und über die bestehende Event-Driven Value Chain aktualisiert werden. Dokumentationshygiene wird dabei kontinuierlich statt punktuell betrieben.

Schutzgrenzen: - keine zweite EventMesh-Implementierung; - keine zweite Vocabulary- oder Traceability-Authority; - keine autonome Architektur- oder Release-Freigabe; - keine Dokumentgenerierung ohne Provenance/Evidence; - keine automatische Mutation geschützter Contracts, APIs, DB-Schemas oder ENV Keys; - bestehende GitHub-Actions-Budgetrichtlinie bleibt bindend.

2. Aktueller Documentary-Stand

Documentary ist derzeit nur teilweise implementiert. Produktiv vorhanden ist der bilinguale Vocabulary-basierte Contract-/Projection-Layer. Mehrere vorgesehene Bereiche (Engine , Events , Generators , Discovery , Knowledge , Mermaid , Migration , Models , Plugins , Interfaces) sind noch überwiegend Zielstruktur. README und Manifest weisen die Komponente entsprechend als Partial Implementation / development aus.

Zusätzlich besteht Versionsdrift: README.md nennt Documentary Version 1.1.0, während manifest.json Version 1.0.0 führt. Diese Inkonsistenz wird als früher Roadmap-Blocker behandelt.

3. Workstream D — Documentary Engine

D0 — Baseline, Manifest und Version Authority

Priorität: P0

- Documentary-README, Manifest, ESS-0010 und realen Codebestand gegeneinander validieren.
- eine eindeutige Documentary-Komponentenversion definieren;
- Version Manager als Quelle für Release-/Plattform-Versionen anbinden;
- Component-Version, Document-Schema-Version und Plattformversion semantisch trennen;
- Versionierungsregeln für generierte Dokumente definieren;
- Manifest-Integrität um Implementierungsgrad und reale Dateien erweitern;
- leere Placeholder-Strukturen entweder mit geplantem Status kennzeichnen oder erst bei Implementierung materialisieren.

Exit: keine Versionsdrift; Manifest entspricht realem Codebestand; Versionierungscontract ist getestet.

D1 — Code Integration & Discovery

Priorität: P0

- read-only Code Discovery für Module, Exports, Contracts, Routes, Events und Manifeste;
- Code-Evidence mit Commit-SHA, Pfad, Symbol und Component-ID versehen;
- dokumentierbare Komponenten automatisch inventarisieren;
- Abhängigkeiten und Import-/Export-Beziehungen als Evidence bereitstellen;
- keine AST-basierte Mutation; Discovery bleibt read-only.

Exit: Documentary kann aus dem aktuellen Repository reproduzierbar eine Code-Evidence-Map erzeugen.

D2 — Documentary Core Engine

Priorität: P0

- `DocumentaryEngine` als Orchestrierungsservice implementieren;
- Input: Evidence + Concept IDs + Traceability + Version Context;
- Output: deterministische Document Models, nicht direkt freie Markdown-Texte;
- idempotente Verarbeitung über Correlation-/Evidence-IDs;
- fail-closed bei fehlender Authority, Provenance oder Version;
- Human-Approval für geschützte Dokumentklassen.

Exit: ein dokumentierter Komponenten-Use-Case läuft vollständig über den Engine-Contract.

D3 — Document Models, Schemas & Provenance

Priorität: P0

- typisierte Document Model Contracts;
- `documentId`, `documentType`, `schemaVersion`, `componentVersion`, `platformVersion`, `sourceCommit`, `generatedAt`, `reviewStatus`;
- Provenance für Code, ESS, ADR, Vocabulary, Event und manuelle Evidence;
- Statusmodell: draft / generated / reviewed / approved / superseded / archived;
- Hash/Fingerprint zur Drift- und Reproduzierbarkeitsprüfung.

Exit: jedes generierte Dokument ist versioniert, provenance-fähig und reproduzierbar.

D4 — Generators & Bilingual Rendering

Priorität: P1

- Markdown Generator;
- DE/EN Rendering aus gemeinsamer semantischer Identität;
- Mermaid/Diagramm-Generator auf strukturierten Models;
- Templates für Architecture, Component, API, Runbook, Release Evidence und Handoff;
- Vocabulary- und Naming-Gates vor Ausgabe;
- keine voneinander unabhängigen DE-/EN-Quellen.

Exit: beide Sprachen werden aus demselben Document Model erzeugt.

D5 — Traceability Integration

Priorität: P0

- ESS ↔ ADR ↔ Code ↔ Tests ↔ Events ↔ Documents ↔ Release verbinden;
- Traceability IDs in Document Models verpflichtend machen;
- orphaned Documents und orphaned Code Components erkennen;
- Impact-Analyse bei Code-/Contract-Änderungen bereitstellen.

Exit: Documentary-Ausgaben sind bidirektional zur Enterprise Traceability verknüpft.

D6 — Knowledge Integration

Priorität: P1

- freigegebene Documentary-Ausgaben in Knowledge-Projektionen überführen;
- superseded/archived Inhalte aus aktuellen Knowledge Views ausschließen;
- Concept-ID und Provenance erhalten;
- keine separate Knowledge-Authority erzeugen.

Exit: Knowledge konsumiert ausschließlich freigegebene, versionierte Documentary-Artefakte.

D7 — Governance, Review & Quality

Priorität: P0

- Documentation Governance Validator direkt anbinden;
- Pflichtfelder, Links, Versionen, Vocabulary, Traceability und Provenance prüfen;
- Review-/Approval-Grenzen je Dokumentklasse definieren;
- stale, duplicate, conflicting und orphaned Dokumente erkennen;
- Audit Evidence für automatische Generierung speichern.

Exit: kein geschütztes Dokument kann ohne Governance-Gate als approved gelten.

D8 — Migration & Legacy Compatibility

Priorität: P1

- bestehende manuelle Dokumente klassifizieren: canonical / generated / evidence / legacy / archive;
- aktuelle Root-Kompatibilitätsausnahmen wie `DATENSCHUTZ_PROTOKOLL.md` kontrolliert migrieren;
- Runtime-Referenzen vor Moves aktualisieren;
- Redirect-/Alias-Strategie für dokumentierte Pfade definieren;
- keine historischen Evidence-Inhalte umschreiben.

Exit: aktive Dokumentation folgt der kanonischen Hierarchie; historische Evidence bleibt unverändert nachvollziehbar.

D9 — Documentary Observability

Priorität: P2

- Metriken: freshness, coverage, orphan rate, stale rate, generation failures, approval latency;
- strukturierte Logs mit correlationId/documentId;
- Health Report für Documentary-Komponente;
- keine personenbezogenen oder geheimen Inhalte in Telemetrie.

Exit: Documentary-Qualität ist messbar und auditierbar.

4. Weitere verbesserungsbedürftige Documentary-Punkte

1. Vollständige Engine fehlt; aktuell existiert primär der Bilingual-Layer.
2. README-/Manifest-Versionen sind inkonsistent.
3. Code Discovery ist nicht als produktiver Documentary-Input implementiert.
4. Dokument-Schema-/Component-/Platform-Versionen sind nicht sauber getrennt.
5. Generators sind noch nicht als ausführbare Pipeline implementiert.
6. Documentary-spezifische Event Producer/Consumer sind nicht vollständig verdrahtet.

7. Traceability ist noch nicht verpflichtender Bestandteil jedes Document Models.
8. Knowledge-Projektion ist noch nicht produktiv an approved Documents gekoppelt.
9. Provenance/Fingerprints für generierte Inhalte fehlen.
10. Review-/Approval-Lifecycle ist nicht vollständig implementiert.
11. Stale-/Duplicate-/Conflict-/Orphan-Erkennung muss automatisiert werden.
12. Mermaid-/Architecture-Diagramme sind noch nicht modellgetrieben generiert.
13. Legacy-/Migration-Pfade benötigen kontrollierte Runtime-Kompatibilität.
14. Dokumentations-Freshness und Coverage besitzen noch keine operativen SLOs/KPIs.
15. Component-Manifeste und Documentary müssen stärker automatisch synchronisiert werden.
16. Release-/Deployment-Evidence muss Documentary deterministisch aktualisieren können.
17. Fehler-/Retry-/Idempotency-Verhalten der Engine ist noch zu definieren und zu testen.
18. Security-/Compliance-Klassifizierung für sensible Dokumenttypen muss in den Generator-Contract einfließen.

5. Workstream E — Event-Driven Value Chain 2.0

E0 — Event Contract Inventory

Priorität: P0

- Producer/Consumer-Matrix aller kanonischen Events erzeugen;
- Events ohne Producer, Consumer oder Authority erkennen;
- Event-Versionen und Payload-Schemas inventarisieren;
- keine neuen Events anlegen, solange ein bestehender kanonischer Event geeignet ist.

E1 — Documentary Event Integration

Priorität: P0

Zielkette: Code/Architecture Change -> Supervisor Evidence -> Platform Decision -> EventMesh -> Documentary Impact -> Traceability -> Quality/Security/Compliance -> Version Manager -> Release Evidence .

- Documentary konsumiert nur freigegebene relevante Events;
- `correlationId` über die gesamte Kette erhalten;
- Documentary-Ausgabe erzeugt Folge-Evidence, keine autonome Entscheidung.

E2 — Idempotency, Ordering & Replay

Priorität: P0

- deduplication/idempotency keys;
- definierte Ordering-Grenzen;

- Replay-Verhalten für Documentary/Knowledge-Projektionen;
- Dead-letter-/Failure-Evidence statt stiller Verluste;
- Tests für doppelte und verspätete Events.

E3 — Version-Aware Events

Priorität: P0

- Event schemaVersion;
- producerComponentVersion;
- sourceCommit/platformVersion;
- Compatibility Policy für Payload-Änderungen;
- Breaking Changes nur über ADR/Contract-Versionierung.

E4 — End-to-End Traceability

Priorität: P1

- correlationId + causationId + evidencId;
- Event → Decision → Document → Version → Release nachvollziehbar;
- Audit Query/Report für eine komplette Wertschöpfungskette.

E5 — Reliability & Observability

Priorität: P1

- Event throughput/failure/retry/latency metrics;
- Consumer Health;
- Poison-event-Erkennung;
- strukturierte Audit Logs;
- keine zusätzliche Vollpipeline für reine Observability-Checks.

E6 — Governance Boundaries

Priorität: P0

- Human Approval bleibt vor geschützten Platform Decisions;
- Quality/Security/Compliance können blockieren;
- Version Manager darf nur verifizierte Evidence übernehmen;
- Render-Deploy bleibt ausschließlich nach verifiziertem `main`-CI möglich.

6. Workstream H — Documentation Hygiene 2.0

H0 — Canonical Folder Policy

- erlaubte Root-Dokumente explizit definieren;
- neue lose Dokumente im CI blockieren;

- Fachordner anhand Document Type/Authority festlegen.

H1 — Continuous Orphan/Stale Detection

- unreferenzierte Dokumente erkennen;
- Links und Cross References validieren;
- Dokumente gegen reale Code-/Manifest-Komponenten prüfen;
- Stale-Schwellen nach Dokumenttyp definieren.

H2 — Archive Lifecycle

- Archivierung statt Löschung für historische Evidence;
- `supersededBy` / `archivedAt` / `sourceVersion` Metadaten;
- Archive vom aktuellen Governance-/Vocabulary-Status sauber trennen.

H3 — Runtime Path Hygiene

- `DATENSCHUTZ_PROTOKOLL.md` nach `docs/compliance/` migrieren;
- `MarkdownOrchestrator` und andere Runtime-Pfade gleichzeitig aktualisieren;
- Hardcoded Document Paths durch Registry/Document IDs ersetzen.

H4 — Registry & Index

- maschinenlesbarer Document Registry/Index;
- Document ID statt Dateipfad als stabile Identität;
- Type, owner, authority, version, language, lifecycle status und path registrieren.

H5 — Hygiene CI Gate

- Root Policy, orphan detection, broken links, duplicate IDs und version drift prüfen;
- vorhandenen CI-Hauptlauf nutzen;
- keine zweite vollständige Actions-Pipeline.

7. Empfohlene Ausführungsreihenfolge

1. D0 — Documentary Baseline & Version Authority.
2. D1 + D3 — Code Discovery sowie Document Models/Provenance.
3. E0 + E3 — Event Inventory und versionierte Event Contracts.
4. D2 — Documentary Core Engine.
5. D5 + E1 + E4 — Traceability und End-to-End Event Integration.
6. D7 + E2 + E6 — Governance, Idempotency und Protected Boundaries.
7. D4 + D6 — Bilingual Generators und Knowledge Projection.
8. H0 + H4 + H5 — Document Registry und kontinuierliche Hygiene-Gates.
9. D8 + H1 + H2 + H3 — Legacy-/Archive-/Runtime-Pfadmigration.
10. D9 + E5 — Observability und operative Qualitätsmetriken.

Jeder Schritt wird als separater Draft-PR auf einem zuvor verifizierten `main` ausgeführt. Aktive Code-/Contract-Migrationen werden vorab auf Dependency-, Regex-, Manifest-, Event- und Deployment-Auswirkungen geprüft.

8. Programm-Exit-Kriterien

Das Programm gilt als abgeschlossen, wenn:

- Documentary eine reale, getestete Engine statt nur Zielstruktur besitzt;
- Code-Evidence und Versionierung deterministisch in Document Models eingehen;
- DE/EN aus derselben semantischen Quelle erzeugt werden;
- Traceability vom Code bis Release bidirektional nachvollziehbar ist;
- Event-Verarbeitung versioniert, idempotent, replay-fähig und beobachtbar ist;
- Dokumentationshygiene kontinuierlich durch Registry und CI geschützt wird;
- keine aktuelle Authority als loses/verwaistes Dokument außerhalb der kanonischen Struktur existiert;
- alle geschützten Entscheidungen weiterhin Human-/Governance-Gates respektieren;
- CI-/Render-Deployment-Grenzen und Budget Policy unverändert eingehalten werden.

Documentary Status-Event-Drift Contract

Quelle: docs/architecture/DOCUMENTARY_STATUS_EVENT_DRIFT_CONTRACT.md

Documentary Status-Event-Drift Contract

Status: ACCEPTED
Date: 2026-08-15 (Phase B/C: 2026-08-16)
Authority: ESS-0010 Documentary Engine, ESS-0004 Enterprise Version Manager
Related: ADR-0014 (Documentation Governance Validator, read-only), ADR-0020 (Repository Convention Validator, read-only), ESS-0011 Traceability, ESS-0012 Documentation Governance
Roadmap context: DOCUMENTARY_EVENT_VALUE_CHAIN_ROADMAP (D1/D3/D7/H1 line)
Phase: A complete (PR #346). B complete (PR #358). C implemented (controlled updater + first hygiene Draft-PR).

1. Problem statement (observed drift)

Recurring pattern in the repository:

- Runbook headers remain on values such as `Status: PRE-MUTATION / OWNER-FREIGABE ERFOR- DERLICH` (or equivalent `OWNER ACTION REQUIRED`, `HUMAN APPROVAL REQUIRED`).
- The same document already contains an **Apply-Evidenz** (or equivalent evidence) section documenting successful apply, timestamps, and verification results.
- Work claims under `.ai/work-claims/*.json` are set to `status: "applied"` with populated `externalMutations[]`.
- Status/Handoff docs and roadmap tables are sometimes updated manually to `applied / VERIFIED`, while the runbook header lags.

Canonical example (Phase B detection; Phase C hygiene applied on Draft-PR):

Artifact	Observed (pre Phase C)
<code>docs/runbooks/SEO_WP_S1_LED-GER_RECONCILIATION.md</code>	Header was <code>PRE-MUTATION / OWNER-FREIGABE ER- FORDERLICH</code>
Same file, section <code>## Apply-Evidenz</code>	Executed 2026-08-15; steps marked success; ledger verification complete
<code>.ai/work-claims/SEO-WP-S1-FOLLOWUP-HARDE- NING-2026-08-15.json</code>	<code>"status": "applied"</code> , <code>externalMutations</code> lists mutations

After Phase C controlled update (Draft-PR only): header → `VERIFIED PASS`. Apply-Evidenz body unchanged.

2. Scope (inventory paths)

In-scope paths for discovery, detection, and **header-only** updates:

Area	Paths
Runbooks	<code>docs/runbooks/**</code>
Work claims	<code>.ai/work-claims/*.json</code> (read for detection; not rewritten by Phase C updater)
SEO / status / handoff	<code>docs/seo/**</code>
Evidence (status headers only)	<code>docs/evidence/**</code>
Architecture status docs	<code>docs/architecture/**</code> (header pattern only)

Out of scope unchanged: Supabase/Render/Stripe mutations, ADR auto-numbering, second CI gate, rewriting historical Apply-Evidenz bodies, global auth/billing/deploy config in the same change as status headers.

3-5. Taxonomy, sources of truth, drift rule

Unchanged from Phase A/B. Detector remains the sole machine recommendation source for Phase C proposals.

6. Detector (Phase B, read-only) — complete

Item	Path
Types	<code>src/platform/Documentary/Discovery/StatusEventEvidence.ts</code>
Detector	<code>src/platform/Documentary/Discovery/StatusEventDriftDetector.ts</code>
Tests	<code>tests/unit/statusEventDriftDetector.test.ts</code>
Detector version	<code>0.1.0-phase-b</code>

Successful merge remains an optional **trigger annotation** only.

7. Controlled updater (Phase C) — implemented

Item	Path
Updater	<code>src/platform/Documentary/Discovery/StatusEventDriftUpdater.ts</code>
Tests	<code>tests/unit/statusEventDriftUpdater.test.ts</code>
Updater version	<code>0.1.0-phase-c</code>

Rules

- Proposals only from Phase B findings with `drift: true`, `conflict: false`, and non-null `recommendedHeaderStatus`.
- Path allowlist: `docs/runbooks/`, `docs/seo/`, `docs/evidence/`, `docs/architecture/`.
- **dryRun defaults to true** (no filesystem write).

- Write mode (`dryRun: false`) is for preparing a **Draft-PR** working tree only — never silent write to `main` , never auto-merge.
- Header line only; Apply-Evidenz body is never rewritten.
- Fail-closed on missing file, missing recommendation, or path outside allowlist.

First controlled hygiene case

- Document: `docs/runbooks/SEO_WP_S1_LEDGER_RECONCILIATION.md`
- Change: `PRE-MUTATION / OWNER-FREIGABE ERFORDERLICH` → `VERIFIED PASS`
- Provenance: claim `SEO-WP-S1-FOLLOWUP-HARDENING-2026-08-15` + Apply-Evidenz 2026-08-15

8. Authority and non-goals

Hard boundaries unchanged: no second EventMesh/Traceability/Version-Manager implementation; no automatic mutation of protected contracts/APIs/DB/ENV/Stripe/Supabase/Render; existing CI budget and PR-contract rules remain binding.

9. Phase status

Phase	Status
A — Contract	Done (PR #346)
B — Read-only detector + tests	Done (PR #358)
C — Controlled updater + first header hygiene	Implemented on <code>feat/status-event-drift-updater-phase-c</code>

10. Handoff note

Automated	Remains Owner/manual
Detect drift + recommend class	Merge of Draft-PR
Propose/apply header in working tree (<code>dryRun: false</code>)	Opening further Draft-PRs for additional drift docs
Path allowlist + fail-closed	Business decision on open migrations

Document class: Architecture / Contract
Owner: SvenKulessa

Documentary Status-Event-Drift — Handoff

Quelle: docs/architecture/DOCUMENTARY_STATUS_EVENT_DRIFT_HANDOFF.md

Documentary Status-Event-Drift — Handoff

Date: 2026-08-16
Authority: ESS-0010 / ESS-0004
Contract: docs/architecture/DOCUMENTARY_STATUS_EVENT_DRIFT_CONTRACT.md

Delivered

Phase	Artifact	PR
A	Contract	#346 (merged)
B	Read-only detector + tests	#358 (merged)
C	Controlled updater + SEO_WP_S1 header hygiene + scan CLI	#364 (Draft)

Code

- src/platform/Documentary/Discovery/StatusEventEvidence.ts
- src/platform/Documentary/Discovery/StatusEventDriftDetector.ts
- src/platform/Documentary/Discovery/StatusEventDriftUpdater.ts
- tests/unit/statusEventDriftDetector.test.ts
- tests/unit/statusEventDriftUpdater.test.ts
- scripts/automation/scanStatusEventDrift.ts (dry-run by default)

Operational scan

```
npx tsx scripts/automation/scanStatusEventDrift.ts
npx tsx scripts/automation/scanStatusEventDrift.ts --path docs/runbooks/F00.md
# write headers only in local Draft-PR working tree:
npx tsx scripts/automation/scanStatusEventDrift.ts --apply
npm run status:scan-drift
```

Exit code 2 when drift findings exist in dry-run (advisory signal only; not a CI gate).

What is detected automatically

- Header class vs work-claim applied + externalMutations[]
- Header class vs completed Apply-Evidenz / Apply Evidence section
- Recommended header (APPLIED / VERIFIED PASS) when sources agree
- Merge-success as optional **trigger annotation** (not a write)

What stays manual / Owner

- Opening and merging Draft-PRs
- Exact final wording beyond the recommended class
- Resolving conflicting evidence
- Any Supabase / Render / Stripe mutation
- Promotion of remaining open PRE_MUTATION runbooks that have **not** been applied

Inventory (2026-08-16) — remaining PRE_MUTATION (no false upgrade)

These headers are **correctly** still PRE_MUTATION / owner-required: no applied claim with mutations and no completed Apply-Evidenz success.

Document	Header	Detector expectation
docs/runbooks/GITGUARDIAN_SNYK_APP_INTEGRATION.md	PRE-MUTATION / OWNER ACTION REQUIRED	no drift
docs/runbooks/M5_SUPABASE_AGENT_AUDIT_MUTATION.md	PRE-MUTATION / HUMAN APPROVAL REQUIRED	no drift
docs/runbooks/M5A_SUPABASE_TOTP_AAL2_HARDENING.md	PLANNED — NO PRODUCTION MUTATION AUTHORIZED	OTHER / no upgrade

Hygiene already proposed (Phase C Draft-PR #364)

Document	Change
docs/runbooks/SEO_WP_S1_LEDGER_RECONCILIATION.md	PRE-MUTATION → VERIFIED PASS (Apply-Evidenz body unchanged)

Only one work-claim with `status: "applied"` was found under `.ai/work-claims/` at inventory time: `SEO-WP-S1-FOLLOWUP-HARDENING-2026-08-15`.

Owner next steps

1. Review and merge Draft-PR **#364** (updater + SEO_WP_S1 header + scan CLI + this handoff).
2. After merge, run `npm run status:scan-drift` on `main` — expect **0 drift** for the `SEO_WP_S1` path.
3. Further header hygiene: only when new Apply-Evidenz + applied claims appear; always via new Draft-PR.

Non-goals (still out of scope)

- Second CI pipeline solely for status hygiene
- Silent write to `main`
- Auto-merge
- Rewriting historical evidence sections

CAPITAL-AI ESS Consolidation Report

Quelle: docs/architecture/ESS_CONSOLIDATION_REPORT.md

CAPITAL-AI ESS Consolidation Report

Enterprise Report

Document ID

ARCH-CONSOL-0001

Version

1.0.0

Status

Enterprise Analysis — **Phase 2: Konsolidierung ausgeführt**

Phase 1 (Analyse) und Phase 2 (additive Umsetzung) sind abgeschlossen.

Es wurde kein Dokument gelöscht, verschoben oder inhaltlich überschrieben. Sämtliche Eingriffe an bestehenden Dokumenten sind Ergänzungen im Frontmatter oder im Dokumentkopf.

Die Nummernentscheidung aus Teil H wurde durch den Platform Director getroffen: **Option A.**

Umsetzung dokumentiert in ADR-0013.

Datum

2026-07-31

Basis

ESS-0001, ESS-0001-CONTRACTS, ESS-0002, ESS-0003

`.ai/registry/ess-registry.json`

ADR-0010, ADR-0011

Enterprise Purpose

Dieser Report analysiert sämtliche Enterprise-Dokumente des CAPITAL-AI Core auf

- doppelte Verantwortlichkeiten
- inhaltliche Überschneidungen
- konkurrierende Contracts

- unklare Zuständigkeiten
- fehlende Cross-References

Er dokumentiert ausschließlich den Ist-Zustand und leitet daraus vorgeschlagene, rein additive Maßnahmen ab.

Keine Maßnahme wurde ausgeführt.

Kein Dokument wurde gelöscht, verändert oder überschrieben.

Teil A — Bestandsaufnahme

A.1 Tatsächlich vorhandene ESS-Dokumente

Dokument	Pfad	Größe	Frontmatter	references:
ESS-0001	.ai/skills/ ESS-0001-Docu- mentary-Archi- tect.md	61.697 B	✓ skill:	x
ESS-0001-CON- TRACTS	.ai/skills/ ESS-0001-Con- tracts.md	162.647 B	x keine	x
ESS-0002	.ai/skills/ ESS-0002- Supervisor-Ar- chitect.md	15.679 B	✓ skill:	✓
ESS-0003	.ai/skills/ ESS-0003-Plat- form-Direc- tor.md	14.600 B	✓ skill:	✓

Vollständigkeit der Suche

`find . -name "ESS-*)"` über das gesamte Repository liefert exakt diese vier Dateien.

A.2 Abweichung zwischen Aufgabenstellung und Repository-Zustand

Die Aufgabenstellung setzt einen Dokumentenbestand voraus, der im Repository nicht existiert. Diese Abweichung ist vor jeder Konsolidierung zu klären, da sie die Nummernvergabe sämtlicher Folgedokumente bestimmt.

In der Aufgabe genannt	Zustand im Repository
docs/ess/	existiert nicht — kein ESS-Dokument liegt unter docs/
ESS-0001-CONTRACTS	✓ vorhanden

In der Aufgabe genannt	Zustand im Repository
ESS-0001-Documentary-Architect	✓ vorhanden
ESS-0004-Documentary-Engine	existiert nicht — ESS-0004 ist anderweitig reserviert, siehe K-02
ESS-0011-CONTRACTS	existiert nicht
ESS-0011-Enterprise-Traceability	existiert nicht

Die Aufgabe beschreibt damit überwiegend einen **Zielzustand**, nicht den Ist-Zustand. Die Konsolidierung ist folglich nicht nur eine Präzisierung bestehender Verantwortlichkeiten, sondern setzt die Neuanlage von drei Dokumenten voraus.

Teil B — Befunde

K-01 — `docs/ess/` **existiert nicht; ESS-Dokumente liegen unter** `.ai/skills/`

Stufe Medium

Kategorie Struktur

Nachweis

```
ls docs/ess/ → No such file or directory.
```

Sämtliche vier ESS-Dokumente liegen unter `.ai/skills/`.

Bewertung

Die Ablage unter `.ai/skills/` ist vertragskonform und beabsichtigt:

- ESS-0001-CONTRACTS Chapter 2 weist `.ai/` ausdrücklich `Skills`, `Prompts`, `Contracts`, `Templates`, `Schemas`, `Registries` und `Knowledge Seeds` zu.
- ESS-0001, ESS-0002 und ESS-0003 tragen im Frontmatter den Schlüssel `skill:` mit `id:` — sie sind formal als Skills modelliert.
- Chapter 5 (Documentation Naming) nennt `ESS-0001.md` als Beispiel für Enterprise-Dokumente, ohne einen Ablageort vorzuschreiben.
- Chapter 5 (AI Artifact Naming) verortet KI-Artefakte ausdrücklich unter `.ai/`.

Konsequenz

Die Einführung von `docs/ess/` wäre eine Strukturänderung nach Chapter 2 und Chapter 16 und erforderte einen eigenen ADR. Sie würde zudem die vier bestehenden Dokumente verschieben — eine Migration nach Chapter 14 mit Referenzanpassung in mindestens elf Dokumenten.

Empfehlung

docs/ess/ **nicht** anlegen. Die bestehende Ablage unter .ai/skills/ beibehalten und in der Responsibility Matrix als verbindlichen Ort festschreiben. Kein Nutzen steht dem Migrations- und Governance-Aufwand gegenüber.

K-02 — ESS-0004 ist verbindlich als Enterprise Version Manager reserviert

Stufe Critical

Kategorie Governance / Nummernraum

Nachweis

ESS-0001, Abschnitt Related Enterprise Specifications:

ESS-0004 – Enterprise Version Manager

.ai/registry/ess-registry.json (gemergt über PR #1):

```
{ "id": "ESS-0004", "title": "Enterprise Version Manager", "status": "reserved",  
  "reservedBy": "ESS-0001" }
```

ESS-0001-CONTRACTS Chapter 16, ESS Registry Contract:

Jede ESS-Nummer wird genau einmal vergeben. Vergebene Nummern werden niemals umbenannt. Reservierte Nummern werden niemals abweichend belegt.

Bewertung

Die Aufgabenstellung weist ESS-0004 der Documentary Engine zu. Das kollidiert unmittelbar mit drei verbindlichen Festlegungen: der Reservierung in ESS-0001 selbst, dem gemergten Registry-Eintrag und der Regel aus Chapter 16.

Es kollidiert zusätzlich mit der Rollenvorgabe dieser Aufgabe:

„ohne bestehende Enterprise-Regeln zu verändern“

Eine Belegung von ESS-0004 mit Documentary Engine wäre genau eine solche Änderung.

Auflösungsoptionen

Option	Bewertung
A — Nächste freie Nummer verwenden	ESS-0010 ist frei (in der Registry als not_required für Repository Governance geführt, Nummer ausdrücklich freigegeben). Documentary Engine → ESS-0010 . Verletzt keine Regel, erfordert keinen ADR. Die in der Aufgabe gewünschte ESS-0011-Belegung für Traceability bleibt unverändert möglich.

Option	Bewertung
B — ESS-0004 umwidmen	Erfordert einen ADR, der die Reservierung in ESS-0001 aufhebt, sowie eine Registry-Änderung. Widerspricht der Rollenvorgabe dieser Aufgabe. Der Version Manager verlöre seine reservierte Nummer und benötigte eine neue.

Empfehlung Option A.

K-03 — ESS-0011 existiert nicht; freier Nummernraum beginnt bei ESS-0010

Stufe High

Kategorie Governance / Nummernraum

Nachweis

`.ai/registry/ess-registry.json` : `"freeNumberSpaceStartsAt": "ESS-0010"`.

ESS-0010 ist als `not_required` geführt mit dem Vermerk „Die Nummer bleibt frei“, da Repository Governance vollständig durch ESS-0001-CONTRACTS Chapter 16 abgedeckt ist.

Bewertung

Die Vergabe von ESS-0011 bei gleichzeitig freiem ESS-0010 erzeugte eine Lücke im Nummernraum. Chapter 16 fordert:

Neue ESS-Dokumente erhalten die nächste freie Nummer. Zwischennummern sind nicht zulässig.

Empfehlung

Vergabe lückenlos ab ESS-0010. In Verbindung mit K-02 ergibt sich zwanglos:

ESS-0010	Documentary Engine (technische Spezifikation)
ESS-0011	Enterprise Traceability Matrix
ESS-0011-CONTRACTS	ETM Contracts

Damit bleibt die in der Aufgabe vorgesehene ESS-0011-Belegung exakt erhalten; lediglich ESS-0004 wandert auf die nächste freie Nummer.

K-04 — ESS-0001-CONTRACTS besitzt kein Frontmatter

Stufe Medium

Kategorie Metadaten / Cross-Reference

Nachweis

Dokument	Frontmatter
ESS-0001	✓
ESS-0001-CONTRACTS	✗
ESS-0002	✓
ESS-0003	✓

ESS-0001-CONTRACTS beginnt unmittelbar mit `# ESS-0001-CONTRACTS`.

Bewertung

Der geforderte Kopfabschnitt (Depends On, Related ESS, Related ADR, Related Components, Related Skills) kann in ESS-0001-CONTRACTS nicht in derselben YAML-Form umgesetzt werden wie in den übrigen Dokumenten, ohne die Dateistruktur zu ändern.

Empfehlung

Cross-Reference-Block als **Markdown-Abschnitt** unmittelbar nach dem bestehenden Status-Abschnitt ergänzen, nicht als YAML. Rein additiv, keine Zeile wird überschrieben. Die Konvention ist in der Responsibility Matrix festzuhalten: YAML-Frontmatter für skill-basierte ESS-Dokumente, Markdown-Abschnitt für Contract-Dokumente.

K-05 — ESS-0001 enthält umfangreiche technische Details, soll aber Vision-Dokument werden

Stufe High

Kategorie Verantwortungsabgrenzung

Nachweis

Die Aufgabe weist ESS-0001-Documentary-Architect ausschließlich Motivation, Zielbild, Architekturidee, Vision und Designprinzipien zu — „Keine technischen Implementierungsdetails.“

Der Ist-Inhalt widerspricht dem erheblich:

Kapitel	Inhalt	Charakter
1	Mission, Vision, Philosophy, Prinzipien	Vision ✓
2	Repository Intelligence, Discovery Lifecycle, Klassifizierungsliste, Metadatenfelder	technisch
3	Code Intelligence, AST-Analyse, Parser, Dependency-Graphen, <code>components.json</code> , <code>technical-debt.json</code>	technisch
4		technisch

Kapitel	Inhalt	Charakter
	Knowledge Graph, Knoten-/Beziehungstypen, Dateiliste knowledge/*.json	
5	Documentation Engine, Generator-Namen, Template-Liste, Metadatenfelder	technisch
6	Architecture Intelligence, Digital Twin, Metriken	technisch
7	Impact Analysis, Migrationsklassen, Risikoarten	technisch
8	Event-Kategorien, Event-Modell, Routing, Replay	technisch
9	Version Manager, Versionskategorien, Entscheidungsmatrix	technisch

Rund acht der neun Kapitel enthalten technische Spezifikationsinhalte.

Kernkonflikt

Die Aufgabe verlangt gleichzeitig:

1. ESS-0001 soll Vision-Dokument ohne technische Details sein.
2. „Vorhandene Inhalte dürfen nicht gelöscht oder überschrieben werden.“

Beides ist zusammen nur erfüllbar, wenn die Umwidmung **ausschließlich deklaratorisch** erfolgt: Das Dokument wird als Foundational Architecture Document gekennzeichnet und verweist für die technische Autorität auf das neue Documentary-Engine-Dokument — der technische Inhalt bleibt jedoch physisch erhalten.

Konsequenz, die explizit festzuhalten ist

Nach der Umwidmung existieren technische Aussagen an zwei Orten. Ohne eine Vorrangregel entstünde genau die konkurrierende Zuständigkeit, die diese Konsolidierung beseitigen soll.

Empfehlung

Verbindliche Vorrangregel im Kopf von ESS-0001 ergänzen, analog zur bereits etablierten Auflösung in ESS-0001-CONTRACTS Chapter 20 (Normative Clarifications):

Die technischen Kapitel 2 bis 9 dieses Dokumentes beschreiben den ursprünglichen Architekturentwurf. Für die verbindliche technische Spezifikation gilt ausschließlich ESS-00XX (Documentary Engine). Bei Abweichungen besitzt ESS-00XX Vorrang.

Diese Formulierung löscht nichts, ändert keine bestehende Regel und beseitigt die Doppelzuständigkeit vollständig.

K-06 — Bereits normativ aufgelöste Überschneidungen

Stufe Information

Folgende Überschneidungen zwischen ESS-0001 und ESS-0001-CONTRACTS wurden mit ADR-0010 bereits verbindlich aufgelöst und sind **kein** offener Befund:

Thema	Auflösung
Event-Namenskonvention (mit/ohne Suffix <code>Event</code>)	Chapter 20, Normative Clarifications — Contracts haben Vorrang
<code>Core</code> als Pflichtmodul	Chapter 16 + Chapter 20 — technische Basisschicht
Layer-Zuordnung der Querschnittsmodule	Chapter 16, Cross Cutting Modules
Lifecycle vs. Version Category	Chapter 14, Vocabulary Mapping

Diese Auflösungen sind bei der Konsolidierung zu referenzieren, nicht zu wiederholen.

K-07 — Komponentenspezifische Contracts in ESS-0002 und ESS-0003

Stufe Low

Kategorie Verantwortungsabgrenzung

Nachweis

Dokument	Contract-Abschnitte
ESS-0002	Finding Contract, Escalation Contract, Interface Contract, Determinism Contract, Persistence Contract, Read Only Contract
ESS-0003	Decision Contract, Interface Contract, Traceability Contract, Documentation Contract

Bewertung

Beide Dokumente definieren Contracts, obwohl ESS-0001-CONTRACTS laut Aufgabenstellung „die einzige globale Contract-Referenz“ ist.

Die Prüfung ergibt jedoch: Es handelt sich durchgängig um **komponentenspezifische Spezialisierungen**, nicht um globale Regeln. Beispiel — ESS-0002 Interface Contract listet ausschließlich die Methoden `observe`, `evaluate`, `report`, `block`, `escalate`, `describe`. Globale Interface-Regeln (Signaturen, Generics, Exports) verbleiben in Chapter 4.

Es liegt **keine** konkurrierende Contract-Definition vor.

Empfehlung

Kein Eingriff. Zur Absicherung genügt ein Satz in der Responsibility Matrix: Komponenten-ESS dürfen ausschließlich komponentenspezifische Contracts definieren; bei Konflikt gilt ESS-0001-CONTRACTS.

K-08 — Cross-References sind unvollständig und uneinheitlich

Stufe High

Kategorie Cross-Reference

Ist-Zustand

Dokument	Depends On	Related ESS	Related ADR	Related Components	Related Skills
ESS-0001	X	X (nur Fließtext im Anhang)	X	X	X
ESS-0001-CONTRACTS	X	X (nur Fließtext in Chapter 20)	X	X	X
ESS-0002	X	🔍 references:	🔍 references:	X	X
ESS-0003	X	🔍 references:	🔍 references:	X	X

ESS-0002 und ESS-0003 führen einen flachen `references:`-Block ohne die geforderte Untergliederung. ESS-0001 und ESS-0001-CONTRACTS führen Beziehungen ausschließlich als Fließtext am Dokumentende — maschinell nicht auswertbar.

Empfehlung

Einheitlichen Cross-Reference-Block in allen vier Dokumenten ergänzen (additiv). Für ESS-0002 und ESS-0003 wird der bestehende `references:`-Block **erweitert**, nicht ersetzt.

K-09 — ADR-Referenzen unvollständig

Stufe Medium

Kategorie Cross-Reference

Nachweis

Dokument	ADR-Bezug
ESS-0001	keiner
ESS-0001-CONTRACTS	keiner im Frontmatter; ADR-0010 nur im Fließtext
ESS-0002	ADR-0006
ESS-0003	ADR-0006, ADR-0007

Vorhandene ADRs: 0004, 0005, 0006, 0007, 0010, 0011 sowie 0003.5, 0008, 0009 (resolved). ADR-0001 bis ADR-0003 existieren weiterhin ausschließlich in `adr_history.json` (GAP-026).

Empfehlung

ADR-0010 und ADR-0011 in ESS-0001-CONTRACTS referenzieren, da beide dieses Dokument unmittelbar betreffen. Die fehlenden ADR-Dokumente 0001-0003 bleiben als GAP-026 offen und sind nicht Gegenstand dieser Konsolidierung.

K-10 — Die Documentary Engine besitzt keine eigene technische Spezifikation

Stufe High

Kategorie Verantwortungslücke

Nachweis

Nennungen von Documentary Engine je Dokument:

Dokument	Nennungen
ESS-0001-CONTRACTS	42
ESS-0001	26
ESS-0003	5
ESS-0002	3

Die Documentary Engine ist damit über vier Dokumente verteilt beschrieben, besitzt jedoch kein eigenes Spezifikationsdokument. Ihre Regeln liegen in Contracts Chapter 15, 17, 18 und 19, ihre Vision in ESS-0001.

Bewertung

Dies ist die eigentliche, sachlich berechnete Lücke, die die Aufgabenstellung schließen will. Ein dediziertes technisches Dokument ist begründet — die Documentary Engine ist die einzige Enterprise-Komponente mit 19 Unterverzeichnissen und zugleich ohne eigene Spezifikation, während Supervisor (ESS-0002) und Platform Director (ESS-0003) je eine besitzen.

Empfehlung

Anlage befürwortet — unter der in K-02 geklärten Nummer.

Teil C — Responsibility Matrix

Vorgeschlagener Zielzustand. Noch nicht umgesetzt.

Dokument	Verantwortung	Darf enthalten	Darf NICHT enthalten
ESS-0001 Documentary & Code Intelligence Architect	Foundational Architecture Document — Gründungs- und Visionsdokument	Motivation, Zielbild, Architekturidee, ursprüngliche Vision, Designprinzipien,	verbindliche technische Spezifikationen, neue

Dokument	Verantwortung	Darf enthalten	Darf NICHT enthalten
		historischer Architektur-entwurf	Contracts, Implementierungsvorgaben
ESS-0001-CONTRACTS Enterprise Technical Contracts	Master Enterprise Standard — einzige globale Contract-Referenz	globale Contracts, Repository Standards, Architekturregeln, Naming, Layer, Versionierung, Governance, AI Standards, normative Klarstellungen	komponentenspezifische Spezifikationen, Vision, historische Inhalte, ETM-Detailprozesse
ESS-0002 Supervisor Architect	Komponentenspezifikation Supervisor	Beobachtungsmodell, Health/Lifecycle-Überwachung, Findings, Eskalation, supervisorspezifische Contracts	globale Contracts, Entscheidungsbefugnisse, Dokumentationserzeugung
ESS-0003 Platform Director	Komponentenspezifikation Platform Director	Entscheidungsbefugnis, Governance-Modell, Ausnahmen, Eigentümer-schaft, Release-Autorität	globale Contracts, Überwachungslogik, Implementierungsdetails anderer Komponenten
ESS-00XX Documentary Engine (neu)	Technische Spezifikation der Documentary Engine	Komponenten, Services, APIs, Events, Workflows, Trigger, Discovery, Registry, Digital Twin, Integration	Vision, Motivation, historische Inhalte, globale Contracts
ESS-00YY Enterprise Traceability (neu)	ETM-Spezifikation	ETM-Architektur, ETM-Komponenten, ETM-Prozesse, ETM-Reports, ETM-Integration, ETM-Workflows	allgemeine Enterprise-Regeln, globale Contracts
ESS-00YY-CONTRACTS ETM Contracts (neu)	Traceability-Matrix-Contracts	ausschließlich ETM-bezogene Contracts	globale Repository-, Naming-, Layer- und Governance-Contracts

Platzhalter **ESS-00XX** und **ESS-00YY** bleiben bis zur Nummernentscheidung (K-02) offen.

Teil D — Cross Reference Matrix

D.1 Ist-Zustand

Von → Nach	ESS-0001	ESS-0001-C	ESS-0002	ESS-0003
ESS-0001	—	Fließtext	Fließtext	Fließtext
ESS-0001-CONTRACTS	Fließtext	—	Fließtext	Fließtext
ESS-0002	✓ references:	✓ references:	—	✓ Fließtext
ESS-0003	✓ references:	✓ references:	✓ references:	—

Maschinell auswertbare Referenzen existieren ausschließlich in ESS-0002 und ESS-0003.

D.2 Zielzustand

Jedes ESS-Dokument erhält einen Kopfabschnitt mit fünf Feldern:

Depends On	hierarchisch übergeordnete Dokumente
Related ESS	fachlich verbundene ESS-Dokumente
Related ADR	zugehörige Architekturentscheidungen
Related Components	Komponenten unter <code>src/platform/</code>
Related Skills	zugehörige KI-Artefakte unter <code>.ai/skills/</code>

Dokument	Depends On	Related ADR	Related Components
ESS-0001	— (Wurzel)	ADR-0010	Documentary
ESS-0001-CONTRACTS	ESS-0001	ADR-0010, ADR-0011	alle 22
ESS-0002	ESS-0001, ESS-0001-CONTRACTS	ADR-0006	Supervisor
ESS-0003	ESS-0001, ESS-0001-CONTRACTS, ESS-0002	ADR-0006, ADR-0007	PlatformDirector
Documentary Engine	ESS-0001, ESS-0001-CONTRACTS	ADR-0010	Documentary, Knowledge, Discovery, Registry
Enterprise Traceability	ESS-0001-CONTRACTS	ADR-0010	Registry, Knowledge, Architecture
ETM-CONTRACTS	ESS-0001-CONTRACTS, Enterprise Traceability	ADR-0010	Registry

Teil E — Vorgeschlagene Maßnahmen

Sämtliche Maßnahmen sind **additiv**. Keine löscht oder überschreibt bestehende Inhalte.

Nr.	Maßnahme	Art	Betroffenes Dokument
M-01	Cross-Reference-Block ergänzen	Ergänzung	ESS-0001
M-02	Cross-Reference-Abschnitt in Markdown ergänzen (kein Frontmatter vorhanden, siehe K-04)	Ergänzung	ESS-0001-CONTRACTS
M-03	Bestehenden <code>references</code> : -Block um <code>dependsOn</code> , <code>relatedComponents</code> , <code>relatedSkills</code> erweitern	Ergänzung	ESS-0002, ESS-0003
M-04	Kennzeichnung als Foundational Architecture Document inkl. Vorrangregel für Kapitel 2-9	Ergänzung	ESS-0001

Nr.	Maßnahme	Art	Betroffenes Dokument
M-05	Documentary-Engine-Spezifikation anlegen	Neuanlage	neu
M-06	Enterprise-Traceability-Spezifikation anlegen	Neuanlage	neu
M-07	ETM-Contracts anlegen	Neuanlage	neu
M-08	Registry um drei Einträge ergänzen	Ergänzung	<code>.ai/registry/ess-registry.json</code>
M-09	Responsibility Matrix als eigenständiges Governance-Dokument ablegen	Neuanlage	neu
M-10	ADR für die Konsolidierung	Neuanlage	ADR-0013

M-10 ist erforderlich, weil die Anlage neuer ESS-Dokumente nach Chapter 16 eine Nummernvergabe durch den Platform Director voraussetzt und Chapter 20 Änderungen am Standard ausschließlich über ADR zulässt.

Teil F — Documentary Integration

Die Aufgabenstellung sieht vor, dass die Documentary Engine künftig automatisch prüft:

- doppelte Verantwortlichkeiten
- widersprüchliche Regeln
- konkurrierende Standards
- fehlende Referenzen

Bewertung

Diese Prüfungen sind bereits vertraglich verankert und benötigen keine neue Regel:

Prüfung	Bestehender Contract
doppelte Verantwortlichkeiten	Chapter 3 — eine Verantwortung, genau ein Verzeichnis
widersprüchliche Regeln	Chapter 12 — <code>ContractViolationEvent</code> , Chapter 20 — Rangfolge
konkurrierende Standards	Chapter 16 — ESS Registry Contract
fehlende Referenzen	Chapter 7 — Validation (ESS-/ADR-Referenzen verpflichtend)
Wissenskonflikte	Chapter 15 — <code>KnowledgeConflictDetectedEvent</code>

Konsequenz

Es ist **kein** neuer Contract zu schreiben. Erforderlich ist ausschließlich ein Validator (`DocumentResponsibilityValidator`) als 17. Pflichtvalidator in der Umsetzung von Chapter 12. Dieser gehört in Umsetzungsstufe 4 aus `REPOSITORY_STRUCTURE_ANALYSIS.md` und ist nicht Gegenstand dieser Konsolidierung.

Teil G — Validierung des Ist-Zustands

Prüfkriterium	Ergebnis	Bemerkung
keine Dokumentenduplikate	✓	vier Dokumente, keine inhaltliche Dublette
keine doppelten Contracts	✓	K-07: ausschließlich komponentenspezifische Spezialisierungen
keine konkurrierenden Verantwortlichkeiten	✗	K-05: technische Autorität zwischen ESS-0001 und Contracts nicht abgegrenzt
vollständige Cross-References	✗	K-08: nur zwei von vier Dokumenten maschinell auswertbar
ESS-Nummerierung konsistent	✗	K-02, K-03: Aufgabenstellung kollidiert mit bindender Reservierung
ADR-Referenzen vollständig	✗	K-09: ESS-0001 und ESS-0001-CONTRACTS ohne ADR-Bezug
Documentary kompatibel	✓	keine Regel steht der automatischen Auswertung entgegen
Version Manager kompatibel	✓	sämtliche Dokumente führen Versionen
Platform Director kompatibel	✓	Nummernvergabe und Freigabe vertraglich verankert

Gesamturteil

Vier von neun Kriterien sind erfüllt. Kein Befund erfordert die Änderung einer bestehenden Regel — sämtliche offenen Punkte sind durch Ergänzungen auflösbar.

Teil H — Blockierende Entscheidung

Die Umsetzung von M-05 bis M-08 kann erst nach Klärung von K-02 beginnen, da die Nummernvergabe sämtliche Dateinamen, Registry-Einträge und Cross-References bestimmt.

Vorgelegt zur Entscheidung durch den Platform Director:

Option	Documentary Engine	Traceability	Regeländerung nötig
A (empfohlen)	ESS-0010		nein

Option	Documentary Engine	Traceability	Regeländerung nötig
		ESS-0011 / ESS-0011-CONTRACTS	
B	ESS-0004	ESS-0011 / ESS-0011-CONTRACTS	ja — ADR zur Aufhebung der Reservierung, Version Manager benötigt neue Nummer

Entscheidung des Platform Director am 2026-07-31: Option A.

ESS-0004 bleibt als Enterprise Version Manager reserviert. Der freie Nummernraum beginnt nach dieser Vergabe bei ESS-0012.

Teil I — Umsetzungsstand Phase 2

I.1 Ausgeführte Maßnahmen

Nr.	Maßnahme	Status	Art des Eingriffs
M-01	Cross-Reference-Block in ESS-0001	✓	Frontmatter ergänzt
M-02	Cross-Reference-Abschnitt in ESS-0001-CONTRACTS	✓	Markdown ergänzt
M-03	crossReference: in ESS-0002 und ESS-0003	✓	Frontmatter ergänzt, references: unverändert erhalten
M-04	ESS-0001 als Foundational Architecture Document	✓	Frontmatter und Kopfabschnitt ergänzt
M-05	ESS-0010 Documentary Engine	✓	Neuanlage
M-06	ESS-0011 Enterprise Traceability	✓	Neuanlage
M-07	ESS-0011-CONTRACTS	✓	Neuanlage
M-08	Registry um drei Einträge ergänzt	✓	Ergänzung
M-09	Responsibility Matrix als Governance-Dokument	✓	Neuanlage ARCH-RESP-0001
M-10	ADR-0013	✓	Neuanlage

Zehn von zehn Maßnahmen ausgeführt.

I.2 Befundstatus nach Umsetzung

Befund	Stufe	Status
K-01 — docs/ess/ existiert nicht	Medium	aufgelöst — Ablage unter .ai/skills/ in ARCH-RESP-0001 verbindlich festgeschrieben, kein neues Verzeichnis
K-02 — ESS-0004 Nummernkonflikt	Critical	aufgelöst — Option A, Reservierung unangetastet
K-03 — ESS-0011 ohne ESS-0010	High	aufgelöst — lückenlose Vergabe ab ESS-0010
K-04 — kein Frontmatter in Contracts	Medium	aufgelöst — Markdown-Konvention festgeschrieben
K-05 — Vision/Technik vermischt	High	entschärft — Vorrangregel gesetzt; technischer Inhalt bleibt physisch erhalten (Löschverbot)
K-06 — bereits aufgelöste Überschneidungen	Info	unverändert, referenziert
K-07 — komponentenspezifische Contracts	Low	aufgelöst — Abgrenzung in ARCH-RESP-0001 und in den Klassifizierungen
K-08 — unvollständige Cross-References	High	aufgelöst — sieben von sieben Dokumenten vollständig
K-09 — unvollständige ADR-Referenzen	Medium	aufgelöst für ESS-Dokumente; GAP-026 bleibt offen
K-10 — Documentary Engine ohne Spezifikation	High	aufgelöst — ESS-0010

Neun von zehn Befunden vollständig aufgelöst, einer entschärft.

I.3 Validierung nach Umsetzung

Prüfkriterium	vorher	nachher
keine Dokumentenduplikate	✓	✓
keine doppelten Contracts	✓	✓
keine konkurrierenden Verantwortlichkeiten	✗	✓ — Vorrangregeln in ARCH-RESP-0001
vollständige Cross-References	✗	✓ — 7 von 7
ESS-Nummerierung konsistent	✗	✓ — lückenlos, frei ab ESS-0012
ADR-Referenzen vollständig	✗	✓ für ESS-Dokumente
Documentary kompatibel	✓	✓
Version Manager kompatibel	✓	✓

Prüfkriterium	vorher	nachher
Platform Director kompatibel	✓	✓

Neun von neun Kriterien erfüllt (zuvor vier von neun).

I.4 Verbleibende Einschränkung

Die Durchsetzung ist dokumentarisch, nicht maschinell.

`DocumentResponsibilityValidator` und `TraceabilityValidator` setzen Umsetzungsstufe 4 aus `REPOSITORY_STRUCTURE_ANALYSIS.md` voraus und existieren nicht. Bis dahin sind die Verantwortungszuordnungen eine verbindliche Zusage, deren Einhaltung manuell zu prüfen ist.

Related Documents

ESS-0001, ESS-0001-CONTRACTS, ESS-0002, ESS-0003

`.ai/registry/ess-registry.json`

ADR-0010 — Enterprise Standard Extension

ADR-0011 — Bestandsschutz Root-Abweichungen

`docs/architecture/ARCHITECTURE_GAP_REPORT.md` — GAP-026, GAP-029

Version History

Version	Status	Beschreibung
1.0.0	Phase 1	Dokumentenanalyse, Responsibility Matrix, Cross Reference Matrix, Maßnahmenvorschlag — keine Änderung ausgeführt

End of Document

ARCH-CONSOL-0001

CAPITAL-AI ESS Consolidation Report

Version 1.0.0

CAPITAL-AI ESS Responsibility Matrix

Quelle: docs/architecture/ESS_RESPONSIBILITY_MATRIX.md

CAPITAL-AI ESS Responsibility Matrix

Enterprise Governance Document

Document ID

ARCH-RESP-0001

Version

1.0.0

Status

Enterprise Governance — verbindlich

Datum

2026-07-31

ADR-Referenz

ADR-0013 — ESS Documentation Responsibility Consolidation

Enterprise Purpose

Dieses Dokument legt verbindlich fest, welches ESS-Dokument welche Verantwortung trägt.

Es beseitigt konkurrierende Zuständigkeiten und verhindert, dass künftige Erweiterungen Inhalte an mehreren Orten gleichzeitig regeln.

Es definiert keine neuen Enterprise-Regeln.

Es ordnet ausschließlich bestehende Verantwortlichkeiten zu.

Geltung

Diese Matrix gilt für sämtliche ESS-Dokumente des CAPITAL-AI Core.

Vor der Anlage eines neuen ESS-Dokumentes ist sie zu prüfen und zu erweitern.

Vor der Ergänzung eines bestehenden ESS-Dokumentes ist zu prüfen, ob der Inhalt der zugewiesenen Verantwortung entspricht.

Ablageort

Sämtliche ESS-Dokumente liegen verbindlich unter

`.ai/skills/`

Begründung

ESS-0001-CONTRACTS Chapter 2 weist `.ai/` ausdrücklich Skills, Prompts, Contracts, Templates, Schemas, Registries und Knowledge Seeds zu.

Chapter 5 (AI Artifact Naming) verortet KI-Artefakte ausschließlich unter `.ai/`.

ESS-0001, ESS-0002, ESS-0003, ESS-0010 und ESS-0011 tragen im Frontmatter den Schlüssel `skill:` mit `id:` — sie sind formal als Skills modelliert.

Ein Verzeichnis `docs/ess/` wird **nicht** eingeführt. Es wäre eine Strukturänderung nach Chapter 2 und Chapter 16 mit ADR-Pflicht und erforderte eine Migration sämtlicher Dokumente samt Referenzanpassung, ohne einen Nutzen zu erzeugen.

Dokumentklassen

Klasse	Bedeutung	Kennzeichnung im Frontmatter
Foundational Architecture Document	Gründungs- und Visionsdokument	<code>classification.type: Foundational Architecture Document</code>
Master Enterprise Standard	einzige globale Contract-Referenz	Markdown-Abschnitt Dokumentklassifizierung
Component Specification	Spezifikation genau einer Plattformkomponente	<code>classification.type: Component Specification</code>
Technical Specification	technische Spezifikation eines Subsystems	<code>classification.type: Technical Specification</code>
Scoped Contracts	Contracts mit begrenzter Geltung	Markdown-Abschnitt Dokumentklassifizierung

Responsibility Matrix

Dokument	Klasse	Verantwortung	Darf enthalten	Darf NICHT enthalten
ESS-0001 Documentary & Code Intelligence Architect	Foundational Architecture Document	Gründungs- und Visionsdokument der Documentary Engine	Motivation, Zielbild, Architekturidee, ursprüngliche Vision, Designprinzipien,	verbindliche technische Spezifikationen, neue Contracts, Implementierungsvorgaben

Dokument	Klasse	Verantwortung	Darf enthalten	Darf NICHT enthalten
			historischer Architekturentwurf	
ESS-0001-CONTRACTS Enterprise Technical Contracts	Master Enterprise Standard	einzige globale Contract-Referenz der Plattform	globale Contracts, Repository Standards, Architekturregeln, Naming, Layer, Versionierung, Governance, AI Standards, normative Klarstellungen	komponentenspezifische Spezifikationen, Vision, historische Inhalte, ETM-Detailprozesse
ESS-0002 Supervisor Architect	Component Specification	Komponentenspezifikation Supervisor	Beobachtungsmodell, Health- und Lifecycle-Überwachung, Findings, Eskalation, supervisor-spezifische Contracts	globale Contracts, Entscheidungsbefugnisse, Dokumentationserzeugung
ESS-0003 Platform Director	Component Specification	Komponentenspezifikation Platform Director	Entscheidungsbefugnis, Governance-Modell, Ausnahmen, Eigentümerschaft, Release-Autorität	globale Contracts, Überwachungslogik, Implementierungsdetails anderer Komponenten
ESS-0004 Enterprise Version Manager	Component Specification	Komponentenspezifikation Version Manager	Komponentenaufbau, Interfaces, Events, Rollback-Planung, Versions-synchronisation	Versionsstrategie, Versionskategorien, globale Release-Contracts
ESS-0005 Quality Center	Component Specification	Komponentenspezifikation Quality Center	Validator-Registry, Gate-Ausführung, Score-Berechnung, Technical Debt Register	Validator-Vertrag, Severity-Stufen, Quality-Gate-Definition, Schwellwerte
ESS-0006 Security & Compliance	Component Specification	Komponentenspezifikation Security Center und Compliance Center	Classification Registry, Audit Trail, Risk Register, Evidence Collector	sämtliche Sicherheits- und Compliance-Regeln, IAM-Regeln
ESS-0007 Enterprise Release Center	Component Specification	Komponentenspezifikation Release Center	Release-Vorbereitung, Deployment-Planung, Rollback-Ausführung, Snapshots	Versionsregeln, Freigabekriterien, Freigabeentscheidung
ESS-0008 AI Agent Framework	Technical Specification	Rahmenwerk fachlicher Domänen-Agenten	Agent Contract, Agent-Identität, Determinismus-Deklaration, Orchestrator Contract, Registrierung	Scoring-Algorithmen, Bewertungslogik, Orchestrierung der KI-Entwicklungssysteme

Dokument	Klasse	Verantwortung	Darf enthalten	Darf NICHT enthalten
ESS-0009 Enterprise Knowledge Platform	Component Specification	Komponentenspezifikation Knowledge Engine	Knowledge Builder, Validator, Query, Versionierung, Store	Wissensmodell, Knoten- und Beziehungsregeln, Vertrauenswerte
ESS-0010 Documentary Engine	Technical Specification	technische Spezifikation der Documentary Engine	Komponenten, Services, APIs, Events, Workflows, Trigger, Discovery, Registry, Digital Twin, Integration	Vision, Motivation, historische Inhalte, globale Contracts
ESS-0011 Enterprise Traceability	Technical Specification	Spezifikation der Enterprise Traceability Matrix	ETM-Architektur, ETM-Komponenten, ETM-Prozesse, ETM-Reports, ETM-Integration, ETM-Workflows	allgemeine Enterprise-Regeln, globale Contracts
ESS-0011-CONTRACTS ETM Contracts	Scoped Contracts	Traceability-Matrix-Contracts	ausschließlich ETM-bezogene Contracts	globale Repository-, Naming-, Layer- und Governance-Contracts
ESS-0012 Documentation Governance	Technical Specification	Spezifikation des Documentation Governance Validator	Prüfbereiche, Komponentenarchitektur des Validators, Trigger, Reports, Integration, Rollentrennung	Validator-Basisregeln, Severity-Definitionen, Coverage-Schwellwerte, Twin-Regeln, globale Contracts
ESS-0012-CONTRACTS Governance Contracts	Scoped Contracts	Governance-Regelwerk und Scoring	Regel-IDs, Regeltexte, Schweregradzuordnung, Nachweistypen, Scoring-Formeln, Governance-Events	globale Repository-, Naming-, Layer- und Governance-Contracts, Severity-Definitionen, Orphan-Klassen
SKILL-GOV-0001 Documentation-Governance-Validator	Operational Skill	Handlungsanweisung für KI-Systeme	Anwendungsreihenfolge, Trigger, Pflichten je KI-System, Abbruchbedingungen, Verweise	normative Regeln, Regeltexte, Schweregrade, Schwellwerte
ESS-0013 Enterprise Event Mesh	Technical Specification	Komponentenspezifikation der Enterprise Event Mesh	Core-Klassen, Contract-Typen, Registry-Mechanik, Validator-Kette, Report-Typen, Standard Event Catalog	Event-Namensregel, Event-Prinzip, globale Producer-/Consumer-Grundregeln, Routing-Prinzip (bleiben in ESS-0001-CONTRACTS Chapter 8)
ESS-0013-CONTRACTS Enterprise Event Mesh Contracts	Scoped Contracts	Event-Katalog-, Kompatibilitäts-, Registry-, Routing-, Discovery-, Policy-	ausschließlich EventMesh-Komponenten-Contracts	globale Event-Namensregeln, globale Repository-, Layer-

Dokument	Klasse	Verantwortung	Darf enthalten	Darf NICHT enthalten
		und Report-Contracts		und Governance-Contracts
SKILL-EVT-0001 Enterprise-Event-Mesh	Operational Skill	Handlungsanweisung für KI-Systeme zur Event-Nutzung	Event Governance, Producer-/Consumer-Pflichten, Trigger, ETM-/Documentary-Integration, Versionierung	normative Regeln, Event-Kategorien-Definition, globale Contracts

Hinweis zu SKILL-GOV-0001 und SKILL-EVT-0001: Beide Skills sind keine ESS-Dokumente und belegen keine ESS-Nummer. Sie besitzen keine normative Wirkung — bei Abweichung gelten die jeweils referenzierten ESS-Dokumente (ESS-0012/ESS-0012-CONTRACTS bzw. ESS-0013/ESS-0013-CONTRACTS). Sie sind hier aufgeführt, weil sie als Nicht-ESS-Artefakte unter `.ai/skills/` liegen und damit derselben Verantwortungsabgrenzung unterliegen.

Vorrangregeln

Bei inhaltlichen Abweichungen zwischen Dokumenten gilt verbindlich:

Konfliktfall	Vorrang
ESS-0001 gegen ESS-0010	ESS-0010 — technische Autorität
ESS-0001 gegen ESS-0001-CONTRACTS	ESS-0001-CONTRACTS — Chapter 1, Rangfolge
ESS-0011-CONTRACTS gegen ESS-0001-CONTRACTS	ESS-0001-CONTRACTS — globale Wirkung schlägt begrenzte
ESS-0012-CONTRACTS gegen ESS-0001-CONTRACTS	ESS-0001-CONTRACTS — globale Wirkung schlägt begrenzte
ESS-0013-CONTRACTS gegen ESS-0001-CONTRACTS Chapter 8	ESS-0001-CONTRACTS — globale Wirkung schlägt begrenzte
SKILL-EVT-0001 gegen ESS-0013	ESS-0013 — Skills besitzen keine normative Wirkung
ESS-0012 gegen ESS-0011-CONTRACTS (Coverage, Orphans)	ESS-0011-CONTRACTS — Traceability ist dort geregelt
SKILL-GOV-0001 gegen ESS-0012	ESS-0012 — Skills besitzen keine normative Wirkung
Komponenten-ESS gegen ESS-0001-CONTRACTS	ESS-0001-CONTRACTS
ADR gegen jedes ESS-Dokument	ADR — Chapter 1, Rangfolge

Die Gesamtrangfolge aus ESS-0001-CONTRACTS Chapter 1 bleibt unverändert gültig:

ADR → ESS Contracts → Enterprise Specifications → Projektdokumentation → Implementierung

Cross-Reference-Konvention

Jedes ESS-Dokument führt fünf Referenzfelder.

Depends On	hierarchisch übergeordnete Dokumente
Related ESS	fachlich verbundene ESS-Dokumente
Related ADR	zugehörige Architekturentscheidungen
Related Components	Komponenten unter src/platform/
Related Skills	zugehörige Artefakte unter .ai/skills/

Formvorgabe

Dokumenttyp	Form
Dokumente mit YAML-Frontmatter	Schlüssel <code>crossReference:</code> im Frontmatter
Dokumente ohne Frontmatter (*-CONTRACTS)	Markdown-Abschnitt Cross Reference nach dem Status-Abschnitt

Beide Formen sind gleichwertig. Die Unterscheidung ist historisch bedingt: ESS-0001-CONTRACTS besitzt kein Frontmatter, und dessen nachträgliche Einführung wäre eine Änderung an einem bestehenden Dokument statt einer Ergänzung.

In ESS-0002 und ESS-0003 bleibt der ursprüngliche flache `references:`-Block unverändert erhalten; `crossReference:` wurde additiv daneben ergänzt.

Aktueller Referenzstand

Dokument	Form	Depends On	Related ESS	Related ADR	Related Components	Related Skills
ESS-0001	<code>crossReference:</code>	✓	✓	✓	✓	✓
ESS-0001-CONTRACTS	Markdown	✓	✓	✓	✓	✓
ESS-0002	<code>crossReference:</code>	✓	✓	✓	✓	✓
ESS-0003	<code>crossReference:</code>	✓	✓	✓	✓	✓
ESS-0010	<code>crossReference:</code>	✓	✓	✓	✓	✓
ESS-0011	<code>crossReference:</code>	✓	✓	✓	✓	✓
ESS-0011-CONTRACTS	Markdown	✓	✓	✓	✓	✓
ESS-0012	<code>crossReference:</code>	✓	✓	✓	✓	✓

Dokument	Form	Depends On	Related ESS	Related ADR	Related Components	Related Skills
ESS-0012-CONTRACTS	Markdown	✓	✓	✓	✓	✓
SKILL-GOV-0001	crossReference:	✓	✓	✓	✓	✓

Vollständig — zehn von zehn Artefakten unter `.ai/skills/`.

Nummernvergabe

Die ESS Registry unter `.ai/registry/ess-registry.json` ist die alleinige Quelle der Nummernvergabe.

Verbindlich gilt ESS-0001-CONTRACTS Chapter 16, ESS Registry Contract:

- Jede ESS-Nummer wird genau einmal vergeben.
- Vergebene Nummern werden niemals umbenannt.
- Reservierte Nummern werden niemals abweichend belegt.
- Neue Dokumente erhalten die nächste freie Nummer.
- Zwischennummern sind nicht zulässig.
- Kein KI-System vergibt Nummern eigenständig.

Stand 2026-07-31

Bereich	Zustand
ESS-0001 bis ESS-0012	vergeben, sämtlich mit Dokument
freier Nummernraum	ab ESS-0013

Seit ADR-0016 existiert **kein reservierter Nummernplatz ohne Dokument** mehr. Die Registry führt 15 von 15 Einträgen mit hinterlegtem Dokument.

Abgrenzung Vertrag gegen Komponente

Die Dokumente ESS-0004 bis ESS-0009 sind **Komponentenspezifikationen**, keine Regelwerke. Die Unterscheidung ist verbindlich und verhindert Duplikate:

Frage	Zuständig
Wie wird versioniert?	ESS-0001-CONTRACTS Chapter 9
Woraus besteht der Version Manager?	ESS-0004
Welche Validierungsregeln gelten?	ESS-0001-CONTRACTS Chapter 12
Wie führt das Quality Center sie aus?	ESS-0005

Frage	Zuständig
Welche Sicherheitsregeln gelten?	ESS-0001-CONTRACTS Chapter 11
Woraus bestehen Security und Compliance Center?	ESS-0006
Wie ist das Wissensmodell definiert?	ESS-0001-CONTRACTS Chapter 15
Woraus besteht die Knowledge Engine?	ESS-0009

Eine Komponentenspezifikation wiederholt niemals eine Regel. Sie verweist auf sie.

Implementierende Komponenten

Zwei ESS-Dokumente besitzen seit ADR-0014 und ADR-0015 eine implementierende Komponente. Die Zuordnung ist verbindlich und verhindert, dass Spezifikationsinhalte in den Komponenten-READMEs wiederholt werden.

Spezifikation	Implementierende Komponente	Operative Doku
ESS-0011 / ESS-0011-CONTRACTS	src/platform/Traceability/	docs/traceability/
ESS-0012 / ESS-0012-CONTRACTS	src/platform/Documentary/Go- vernance/	—

Regel Komponenten-READMEs und operative Dokumentation wiederholen niemals Spezifikationsinhalte. Sie verweisen auf das zuständige ESS-Kapitel. Bei Abweichung gilt die Spezifikation.

Prüfung vor Neuanlage

Vor der Anlage eines neuen ESS-Dokumentes ist verbindlich zu prüfen:

- ✓ Ist der Regelbereich bereits durch ein bestehendes Dokument abgedeckt?
- ✓ Wäre das neue Dokument ein Duplikat im Sinne von Zero Duplication?
- ✓ Ist die nächste freie Nummer aus der Registry verwendet?
- ✓ Ist eine Dokumentklasse zugewiesen?
- ✓ Sind sämtliche fünf Referenzfelder befüllt?
- ✓ Ist die Verantwortung in dieser Matrix ergänzt?
- ✓ Existiert ein ADR für die Nummernvergabe?

Wird eine dieser Prüfungen nicht bestanden, ist das Dokument nicht anzulegen.

Automatisierte Durchsetzung

Die in dieser Matrix festgelegten Zuordnungen sind maschinell prüfbar.

Die Prüfung erfolgt künftig durch den `DocumentResponsibilityValidator` als Bestandteil der Validatoren aus ESS-0001-CONTRACTS Chapter 12.

Zu prüfen sind

- doppelte Verantwortlichkeiten
- widersprüchliche Regeln
- konkurrierende Standards
- fehlende Referenzen

Es ist **kein** neuer Contract erforderlich — die Prüfungen sind bereits durch Chapter 3, Chapter 7, Chapter 12, Chapter 15 und Chapter 16 gedeckt.

Der Validator gehört in Umsetzungsstufe 4 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md`.

Related Documents

`docs/architecture/ESS_CONSOLIDATION_REPORT.md` — ARCH-CONSOL-0001

`.ai/registry/ess-registry.json`

ADR-0013 — ESS Documentation Responsibility Consolidation

ESS-0001-CONTRACTS Chapter 1, Chapter 16, Chapter 20

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Verbindliche Verantwortungszuordnung für sieben ESS-Dokumente
1.1.0	Update	ESS-0013, ESS-0013-CONTRACTS und SKILL-EVT-0001 ergänzt (ADR-0018, Enterprise Event Mesh)

End of Document

ARCH-RESP-0001

CAPITAL-AI ESS Responsibility Matrix

Version 1.0.0

Event Value Chain E6 — Governance Boundaries

Quelle: docs/architecture/EVENT_VALUE_CHAIN_E6_GOVERNANCE_BOUNDARIES.md

Event Value Chain E6 — Governance Boundaries

Status: IMPLEMENTED IN DRAFT Date: 2026-08-10 Basis: E2/E5 / PR #177 / verified main 32d7ca0bcfef5f7eb5cb54be7772a6c69c8d3b4c / main CI #720 / Render production verified

Deutsch

E6 macht die geschützten Entscheidungsgrenzen der Event-Driven Value Chain explizit prüfbar. Ein `PlatformDecisionRecord` darf die Boundary nur passieren, wenn er `APPROVED` ist, vom `Platform Director` entschieden wurde, eine Correlation-ID und mindestens eine Decision-Basis-Evidence besitzt und keine geblockte Supervisor-Evidence enthält.

Für `Release Decision` gelten zusätzliche fail-closed Regeln: Release-Candidate-Evidence und Rollback-Plan müssen vorhanden sein; Quality-, Security-, Compliance- und Version-Gate müssen jeweils `PASS` sein. `FAIL` und `UNAVAILABLE` blockieren die Boundary.

E6 erzeugt keine Entscheidung, genehmigt nichts autonom und ersetzt weder Platform Director, Version Manager, Release Center noch EventMesh. Die Policy ist ein reiner Boundary-Validator vor einer bereits bestehenden Propagation.

English

E6 makes protected governance boundaries explicitly testable. Approved Platform Director decisions require correlation and decision-basis evidence and must not contain a blocked Supervisor assessment. Release decisions additionally require release-candidate evidence, a rollback plan and PASS results for quality, security, compliance and version gates.

The validator does not create or approve decisions and does not introduce a second governance, release, versioning or event authority.

EventMesh E0 + E3 — Event Contract Inventory & Versioning

Quelle: docs/architecture/EVENTMESH_E0_E3_EVENT_CONTRACT_VERSIONING.md

EventMesh E0 + E3 — Event Contract Inventory & Versioning

Status: IMPLEMENTED IN DRAFT

Date: 2026-08-10

Authority: ESS-0013 / ESS-0013-CONTRACTS

Foundation: D3 / PR #165 / main 65e9950b07cb8bad863ee387a811a52972697e11

E0 — Event Contract Inventory

`buildEventContractInventory()` erzeugt eine deterministische Producer-/Consumer-Matrix aus dem bestehenden `STANDARD_EVENT_CATALOG` und den deklarierten `events.produces` / `events.consumes` der Platform-Manifeste. Es wird keine zweite Event Registry eingeführt.

Erkannte Gap-Typen: - `NO_PRODUCER` - `NO_CONSUMER` - `NO_AUTHORITY` - `UNREGISTERED_EVENT`

Unregistrierte oder authority-lose Events sind im Automation Entry Point blockierend. Producer-/Consumer-Lücken bleiben sichtbare Evidence und werden nicht automatisch durch erfundene Komponentenbeziehungen geschlossen.

E3 — Version-aware Event Contracts

`EventRuntimeVersionContext` ergänzt den bestehenden Event-Contract additiv um: - `schema-Version` - `producerComponentVersion` - `platformVersion` - `sourceCommit`

Alle Versionen müssen strict SemVer sein; `sourceCommit` muss ein vollständiger 40-stelliger Git-SHA sein. Die bestehende `EventVersion`-Authority bleibt unverändert.

Compatibility Policy: - gleiches Major + monotone Minor/Patch-Entwicklung: kompatibel; - Major-Wechsel: breaking und nur über ADR/Contract-Migration zulässig; - Downgrade: nicht kompatibel; - keine automatische Contract-Migration durch EventMesh.

Schutzgrenzen

- kein zweiter EventBus / keine zweite Event Registry;
- keine neuen fachlichen Event-Typen;
- keine autonome Producer-/Consumer-Zuordnung;
- keine Änderung an Stripe, Supabase, Render, DB-Schemas oder ENV Keys;
- keine neue GitHub-Actions-Vollpipeline;
- bestehende Human-/Governance-Grenzen bleiben unverändert.

Exit

E0 ist erfüllt, wenn die kanonischen Events mit Producer-/Consumer-/Authority-Evidence inventarisierbar sind. E3 ist erfüllt, wenn Events eine validierbare Schema-/Producer-/Plattform-/Commit-Versionsevidence tragen können und Breaking Changes fail-closed erkennbar sind.

EventMesh E2 + E5 — Replay, Ordering & Reliability Evidence

Quelle: docs/architecture/EVENTMESH_E2_E5_REPLAY_RELIABILITY.md

EventMesh E2 + E5 — Replay, Ordering & Reliability Evidence

Status: IMPLEMENTED IN DRAFT Date: 2026-08-10 Basis: D7 / PR #175 / verified main 3d-b07f55c161453dfc285f318cd95780416b6bcf / main CI #717 / Render production verified

Deutsch

E2 führt eine explizite Replay-/Ordering-Grenze für Event-Verarbeitung ein. `eventId` ist der Idempotency Key. Deliberate Replays derselben Event-ID werden als `duplicate` erkannt. Ordering wird nur innerhalb derselben `correlationId` bewertet; verspätete Events werden als `stale` klassifiziert, ohne andere Correlations zu blockieren.

E5 leitet aus dem bestehenden EventMesh Delivery Log strukturierte Reliability Evidence ab: Delivery-/Failure-/No-Consumer-Zähler, Failure Rate, betroffene Event-Typen, fehlgeschlagene Consumer, Failure Evidence und Poison-Event-Kandidaten. Die Auswertung ist deterministisch und read-only.

Schutzgrenzen: keine zweite EventMesh, keine neue Registry, keine autonome Retry-Orchestrierung, keine fachliche Entscheidung, keine direkte Knowledge-/Documentary-Kopplung. Die aktuelle Replay-Guard ist in-memory; durable Multi-Instance-Deduplication bleibt ein separater Infrastruktur-Scope.

English

E2 adds an explicit replay and ordering boundary. `eventId` is the idempotency key; deliberate duplicate deliveries are classified as `duplicate`, while ordering is scoped to a single `correlationId` and older arrivals are classified as `stale`.

E5 derives deterministic reliability evidence from the existing EventMesh delivery log: delivery/failure/no-consumer counts, failure rate, failing event types and consumers, structured failure evidence, and poison-event candidates. No second event bus, registry, retry pipeline, or business decision path is introduced.

ADR-0014 Phase 3.4.5 — FMP Index Provider Stage

Quelle: docs/architecture/evidence/server-domain-phase-3-4-5-fmp-stage.md

ADR-0014 Phase 3.4.5 — FMP Index Provider Stage

Scope

This phase completes the provider-stage preparation for market-data refresh by adapting the existing FMP index cache/cooldown implementation to the canonical `MarketDataProviderStage` contract.

Invariants

- `server/fmpIndices.ts` remains the authoritative FMP API/cache implementation.
- Provider stage order is Crypto → Stooq → FMP.
- FMP refresh stays non-blocking and cooldown-aware.
- Only usable cached quotes are emitted as `dataSource: live`.
- Missing index quotes remain absent from the stage and are completed later by the coordinator fallback step.
- History refresh is best-effort and must not delay quote delivery.
- No scoring, Stripe, runtime-security, Render, secret-validation, or TOTP diagnostic behavior is changed.

Next step

Perform a fresh overlap check against `main`, then cut `server.application.ts` over to `refreshMarketData(createMarketDataProviderStages(...))` in a dedicated compatibility PR. The inline provider implementations are removed only in that cutover.

Generalisierter Systemadministrator-Ausführungshost — Architektur-Diskussionspapier

Quelle: docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md

Generalisierter Systemadministrator-Ausführungshost — Architektur-Diskussionspapier

Status: DISCUSSION PAPER — kein ADR, keine Implementierung, kein neues Mandat. Reine Architekturanalyse auf Owner-Anfrage ("Design für echten Ausführungs-Host skizzieren").

Nachtrag 2026-08-15: Modell A (Abschnitt 3) wurde inzwischen auf separate Owner-Anweisung tatsächlich umgesetzt und end-to-end real bewiesen (nicht nur strukturell getestet) — siehe docs/adr/ADR-0074-generalized-systemadmin-work-package-catalog.md, VERIFIED PASS seit 2026-08-15 (Issue #325, PR #326). Modell B (Abschnitt 3/4) bleibt unverändert unbegonnen; der in Abschnitt 4 identifizierte Blocker (Tool-Call-Vermittlungsschicht) wurde nicht angegangen. Datum: 2026-08-15 Authority-Referenzen: .ai/skills/ESS-0021-Systemadmin-Roadmap-Executor.md, docs/adr/ADR-0065-systemadmin-roadmap-execution-mandate.md, docs/adr/ADR-0067-systemadmin-github-actions-execution-host.md, docs/adr/ADR-0068-first-bounded-autonomous-work-package.md, docs/governance/ROADMAP_EXECUTION_MANDATE.schema.json

0. Anlass

Owner-Anweisung: "fahre mit der Ausführung des Systemadministrators fort, der die DevelopmentChain Roadmap codebasiert umsetzt". Vor diesem Dokument wurde geklärt, dass es aktuell **keinen** wiederverwendbaren, generalisierten Ausführungshost gibt — nur zwei verbrauchte Einzelnachweise (SA3B, SA4). Dieses Dokument beschreibt, was ein echter, sicherer, generalisierter Host tatsächlich bräuchte, ohne ihn zu bauen.

1. Bestandsaufnahme: was heute existiert

Host	Workflow	Fähigkeit	Status
SA3B	<code>.github/workflows/systemadmin-roadmap-executor.yml</code>	Erzeugt genau einen leeren Branch unter Audit (BRANCH -Capability only)	COMPLETE/VERIFIED PASS als Nachweis, reaktiviert für exakt dasselbe Probe-Mandat (REM-SA3B-PROBE-001, allowedPaths: ["docs/evidence/sa3b/**"])
SA4	<code>.github/workflows/systemadmin-sa4-pilot.yml</code>	Erzeugt Branch, committet eine hartkodierte, deterministische Datei (docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md) öffnet Draft-PR	Permanent deaktiviert (if: false && ...); Zielfeile bereits auf main gemergt → self-Reaktivierung

Beide Hosts teilen ein zentrales, bewusstes Sicherheitsmuster (ADR-0068): der Issue-Body (der einzige externe Eingabekanal) wird strikt validiert (`scripts/systemadmin/validateExecutionIssue.mjs` / `validateSa4PilotIssue.mjs`) und darf **niemals** Dateiinhalte, Shell-Befehle, beliebige Pfade oder Workflow-Definitionen liefern — nur eng begrenzte Metadaten (z. B. welche vordefinierte Branch-Namenskonvention). Der tatsächliche Dateiinhalt kommt aus vertrauenswürdigem, bereits auf `main` gemergtem Code. Diese Eigenschaft ist der Grund, warum SA4 überhaupt sicher betrieben werden konnte: es gibt buchstäblich keinen Codepfad, über den externer Text zu ausgeführtem Code oder beliebigem committetem Inhalt wird.

`server/systemadmin/systemadminExecutionBrokerRouter.ts` verschärft das zusätzlich: der Broker kennt **hartkodiert genau zwei** Workflow-Referenzen und bildet sie 1:1 auf genau zwei Mandate ab (`expectedMandateForWorkflow()`). Ein drittes Mandat für ein drittes, neues Arbeitspaket würde vom Broker heute mit `workflow-mandate-binding-mismatch` abgelehnt, auch wenn die Mandat-JSON-Datei selbst schemakonform und Owner-approved wäre. Eine Generalisierung erfordert also **Code-Änderungen am Broker selbst**, nicht nur eine neue Mandat-Datei.

2. Zieldefinition: was "codebasiert die Roadmap umsetzt" tatsächlich bedeutet

ESS-0021 Abschnitt 5 beschreibt die eigentlich gewünschte autonome Schleife:

```
main/Roadmap/Evidence lesen → nächstes Arbeitspaket wählen → Preflight →  
frischer Branch → Code/Doku/Tests IMPLEMENTIEREN → gezielte Checks →  
Security-Selbstprüfung → committen → iterieren → PR öffnen/aktualisieren →  
CI-Mängel im Scope beheben → STOPP bei Human/Owner-Review
```

Der Schritt "Code/Doku/Tests implementieren" ist bei SA3B/SA4 nicht vorhanden — beide Hosts kennen ihren gesamten Output bereits zur Build-Zeit (leerer Branch bzw. eine fixe Datei). Ein Host, der **beliebige** Roadmap-Arbeitspakete umsetzt, muss zur Laufzeit entscheiden, welchen Code er schreibt. Das ist der eigentliche, bislang ungelöste Kern dieser Anfrage.

3. Zwei grundsätzlich verschiedene Modelle

Modell A — Katalog vorab genehmigter Skripte (Erweiterung des SA4-Musters)

Für jedes einzelne, vom Owner freigegebene Arbeitspaket wird ein eigenes, kleines, menschlich/interaktiv geschriebenes und reviewtes Skript auf `main` gemergt (analog `runSa4Pilot.mjs`, aber pro Paket). Der Issue-Body wählt nur **welches** bereits gemergte Skript läuft, plus eng begrenzte Parameter — nie Freitext, nie Code.

- **Vorteil:** verändert das bestehende Sicherheitsmuster nicht. Jede einzelne Änderung wurde bereits vor dem Merge auf `main` von einem Menschen (oder dieser interaktiven Sitzung, mit anschließender Owner-Freigabe) inhaltlich geprüft — der GitHub-Actions-Host führt nur eine bereits genehmigte Änderung aus, er entscheidet nichts.

- **Nachteil:** das ist keine "autonome Umsetzung neuer Arbeitspakete" im eigentlich gemeinten Sinn — die kreative/analytische Arbeit (welchen Code schreiben) bleibt vollständig vorgelagert. Es automatisiert **Ausführung**, nicht **Implementierung**. Skaliert linear mit der Anzahl vorab genehmigter Pakete; jedes neue Paket braucht wieder ein neues Skript + Review
- Broker-Erweiterung (siehe Abschnitt 1).

Modell B — LLM-gestützter Host mit echter Laufzeit-Implementierung

Der Workflow ruft innerhalb des GitHub-Actions-Runners ein LLM (z. B. über die Anthropic-API) mit eingeschränktem Tool-Zugriff auf (Datei lesen/schreiben, Tests ausführen, `git / gh`), das anhand der im Issue beschriebenen Aufgabe tatsächlich Code generiert und committet.

Das entspricht dem, was in dieser interaktiven Sitzung ohnehin bereits geschieht — nur ohne einen Menschen, der bei jedem Schritt zusieht.

Das ist der Fall, für den heute keine tragfähige Sicherheitsarchitektur existiert.

Grund:

- Die heutige Autorisierung (`authorizeSystemadminAuditedExecution` → Broker → Permit) prüft **eine grobkörnige Absicht** einmal vorab ("ich möchte jetzt Capability X auf Ziel Y ausführen") und erlaubt danach genau **eine** konkrete Git-Operation (ein Branch, ein Commit, ein PR). Ein LLM mit echter Werkzeugschleife führt aber potenziell **Dutzende bis Hunderte** einzelne Tool-Aufrufe pro Lauf aus (Datei lesen, Datei schreiben, `grep`, Testlauf, erneutes Schreiben ...). Es gibt aktuell **keinen Mechanismus**, der `mandate.allowedPaths / allowedCapabilities` auf **jeden einzelnen** dieser Tool-Aufrufe anwendet — nur auf die drei groben Git-Endoperationen.
- Das ist exakt die in `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` Abschnitt 1.2 dokumentierte, als dauerhafte Scope-Grenze offengelegte Lücke dieser Sitzung selbst: die Tool-Aufrufe eines laufenden LLM-Agenten sind strukturell nicht durch `agentIam.ts` vermittelt, weil die Berechtigung von der äußeren Ausführungsumgebung kommt, nicht vom Repository-Code. Ein LLM-Host **innerhalb** von GitHub Actions hat exakt dasselbe Problem — nur ohne den einzigen Kompensationsfaktor, den die interaktive Sitzung heute hat: einen Menschen, der in Echtzeit zusieht und jederzeit unterbrechen kann.
- Der Issue-Body wird zur Aufgabenbeschreibung des LLM — klassische Prompt-Injection-Fläche. Der Trigger ist zwar auf `github.event.issue.user.login == 'SvenKulesa'` beschränkt, das reduziert externe Angreifer, aber nicht das Risiko, dass ein Issue-Text (z. B. aus einer externen Quelle kopiert) unbeabsichtigt Anweisungen enthält, die das LLM als Instruktion statt als zu bearbeitenden Inhalt interpretiert.

4. Anforderungen, die ein echter Modell-B-Host erfüllen müsste

Diese Liste ist der eigentliche Kern eines künftigen ADR, falls Modell B je verfolgt wird — hier nur als Anforderungskatalog, nicht spezifiziert:

1. **Tool-Call-Vermittlungsschicht ("Interceptor").** Jeder einzelne Datei-/Shell-Zugriff des LLM-Agenten muss vor Ausführung gegen `mandate.allowedPaths / allowedCapabilities` geprüft werden — eine Erweiterung von `evaluateAgentAuthorization() / checkProviderProfileScope()` von "einmal pro Anfrage" auf "einmal pro Tool-Aufruf, mit demselben REM im Kontext". Ohne das ist jede weitere Anforderung in dieser Liste wirkungslos, weil `allowedPaths` nicht durchsetzbar wäre.
2. **Broker-Generalisierung.** `systemadminExecutionBrokerRouter.ts` müsste von der hartkodierten Zwei-Workflow-Zuordnung (Abschnitt 1) auf eine generische, mandatgetriebene Zuordnung umgestellt werden, ohne die bestehende OIDC-Workflow-Identitätsbindung zu schwächen.
3. **Getrennte Vertrauenszonen für Anweisung vs. Inhalt.** Der Issue-Body darf als Aufgabenbeschreibung dienen, muss aber im Prompt-Aufbau eindeutig von zitiertem/gelesenem Repository-Inhalt getrennt sein — dasselbe Prinzip, das diese Sitzung bereits auf externe PR-/Review-Inhalte anwendet (`untrusted-keys` , `trust="relay"` in den Event-Umschlägen).
4. **Feingranulares Audit.** Nicht nur Branch/Commit/PR-Ereignisse, sondern jeder einzelne Tool-Aufruf müsste mit `mandateId + roadmapItem + Tool + Pfad + Entscheidung` korreliert und in der M5-Audit-Kette persistiert werden (ESS-0021 Abschnitt 9 verlangt das bereits konzeptionell, aber ohne die technische Granularität, die ein LLM-Agent-Loop erzeugt).
5. **Sofort wirksamer Kill-Switch.** `killSwitch.enabled` müsste einen laufenden mehrere-Minuten -Agentenlauf unterbrechen können, nicht nur künftige Anfragen blockieren — heute prüft die Kette den Kill-Switch nur bei der einmaligen Vorab-Autorisierung.
6. **CI-/Kosten-Deckel.** Ein LLM-Agent-Loop mit potenziell vielen API-Aufrufen braucht ein hartes Zeit-/Kosten-Limit pro Lauf (`ciBudgetPolicyRef` existiert bereits als Konzept, aber nicht für LLM-API-Kosten).
7. **Unveränderter Human-Gate.** Merge bleibt in jedem Modell ausschließlich Owner-Aktion — das ist bereits heute vollständig durch die Abwesenheit einer `MERGE` -Capability im gesamten System strukturell erzwungen (`agentIam.ts`) und müsste unverändert bleiben.

Punkt 1 ist der Blocker: ohne ihn ist jeder weitere Punkt Kosmetik, weil `allowedPaths` nicht technisch durchsetzbar wäre — ein LLM-Agent könnte trotz engem Mandat jede Datei im Repository lesen und schreiben, sobald er einmal grob autorisiert ist.

5. Offene Fragen, falls Modell B verfolgt werden soll

- Welcher LLM-Provider/welches Modell läuft innerhalb des GitHub-Actions-Runners, und wie werden dessen API-Zugangsdaten als GitHub-Actions-Secret verwaltet (eigene Ro-

tations-/Exposure-Fragen, die über den bestehenden GitHub-OIDC-Mechanismus hinausgehen, der für den Broker-Zugriff selbst bereits gelöst ist)?

- Ist ein **unbeaufsichtigter** Host mit Punkt 1-6 tatsächlich sicherer oder wertvoller als der heutige Zustand — eine interaktive Sitzung, die dieselbe Arbeit macht, aber mit einem Menschen, der jeden PR vor Merge sieht und jederzeit "stop" sagen kann? Diese gesamte Sitzung (M7-Abschluss, M8-Elemente, P2-1-Bestandsaufnahme, F1/F4-Fixes) wurde bereits **codebasiert** über die DEVELOPMENT-Chain-Roadmap umgesetzt — nur eben interaktiv, nicht über einen unbeaufsichtigten GitHub-Actions-Host. Der Mehrwert eines Modell-B-Hosts wäre in erster Linie Durchsatz (mehrere Läufe parallel/über Nacht), nicht neue Fähigkeit.
- Falls gewünscht: welches wäre das erste, bewusst kleine Zielpaket für einen Modell-B-Piloten — analog zu SA4s "eine Datei, ein PR" -Ansatz, aber mit echter (wenn auch trivialer) Laufzeit-Entscheidung statt hartkodiertem Inhalt?

6. Empfehlung

Modell A ist mit dem bestehenden Sicherheitsmuster sofort erweiterbar, liefert aber keine echte autonome Implementierung — nur wiederholbare Ausführung bereits genehmigter Änderungen. Modell B ist das, was mit "codebasiert die Roadmap umsetzen" wahrscheinlich eigentlich gemeint ist, hat aber einen ungelösten Kernblocker (Punkt 1, Tool-Call-Vermittlung) und würde eine eigene, mehrstufige Roadmap-Phase mit eigenem ADR und eigenem Sicherheits-Review rechtfertigen — nicht als Erweiterung von M8 oder P2-1, sondern als eigenständiges Vorhaben. Diese Sitzung empfiehlt, Modell B nicht ungeprüft zu beginnen, bevor Punkt 1 konzeptionell gelöst ist, und in der Zwischenzeit die DEVELOPMENT-Chain-Roadmap wie bisher interaktiv fortzusetzen.

7. Nicht Bestandteil dieses Dokuments

- Keine Implementierung, kein neuer Workflow, kein neuer Broker-Code.
- Kein neues REM-Mandat (weder Entwurf noch Freigabe).
- Kein neues ADR — dieses Dokument ist bewusst ein Diskussionspapier mit niedrigerer Verbindlichkeit.
- Keine Entscheidung, ob Modell B überhaupt verfolgt wird — das ist eine Owner-Entscheidung, die dieses Dokument vorbereitet, aber nicht trifft.

Verwandte Dokumente

- `.ai/skills/ESS-0021-Systemadmin-Roadmap-Executor.md`
- `docs/adr/ADR-0067-systemadmin-github-actions-execution-host.md`, `docs/adr/ADR-0068-first-bounded-autonomous-work-package.md`
- `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` Abschnitt 1.2 (Interaktive Sitzung- Bypass-Befund, dieselbe Grenzfrage)
- `server/systemadmin/systemadminExecutionBrokerRouter.ts`, `.github/workflows/systemadmin-roadmap-executor.yml`, `.github/workflows/systemadmin-sa4-pilot.yml`

CAPITAL-AI Legacy Documentation Consolidation

Quelle: docs/architecture/LEGACY_DOCUMENTATION_CONSOLIDATION_2026-08-09.md

CAPITAL-AI Legacy Documentation Consolidation

Date: 2026-08-10
Status: CONSOLIDATION IMPLEMENTED IN DRAFT
Authority: ESS-0010, ESS-0011, ESS-0012
Scope: legacy and loose documentation under docs/

Deutsch

Kanonische Hierarchie

- 1. ESS/Contracts und Registries unter .ai/ .
- 2. ADRs unter docs/adr/ .
- 3. Aktuelle Architektur unter docs/architecture/ .
- 4. Fachliche Implementierungsdokumentation unter docs/<domain>/ .
- 5. Datiertes Evidence-/Audit-Material.
- 6. docs/archive/ ausschließlich als historische, nicht-kanonische Quelle.

Abgeschlossene Konsolidierung

Früherer Root-Pfad	Neuer Pfad	Status
docs/API.md	docs/archive/raw-materials/API.md	historischer API-Snapshot; aktuelle API Authority: docs/architec-ture/api/
docs/Classification-Agent.md	docs/archive/raw-materials/Classification-Agent.md	historisch
docs/Classification-Model.md	docs/archive/raw-materials/Classification-Model.md	historisch
docs/Fundamentals-Agent.md	docs/archive/raw-materials/Fundamentals-Agent.md	historisch
docs/Orchestrator.md	docs/archive/raw-materials/Orchestrator.md	historisch
docs/Risk-Agent.md	docs/archive/raw-materials/Risk-Agent.md	historisch
docs/Scoring-Model.md	docs/archive/raw-materials/Scoring-Model.md	historisch
docs/Valuation-Agent.md	docs/archive/raw-materials/Valuation-Agent.md	historisch

Früherer Root-Pfad	Neuer Pfad	Status
docs/ARCHITECTURE_REVIEW.md	docs/archive/legacy/ARCHITECTURE_REVIEW.md	nicht-kanonischer historischer Review
docs/Documentary.md	docs/archive/legacy/Documentary.md	superseded Blueprint
docs/PRODUCTION_DEPLOYMENT_GUIDE.md	docs/archive/legacy/PRODUCTION_DEPLOYMENT_GUIDE.md	superseded Operations Guide
docs/integration-plan.md	docs/archive/legacy/integration-plan.md	superseded Integrationsplan
docs/COMPLIANCE_REPORT.md	docs/archive/evidence/COMPLIANCE_REPORT.md	historischer Evidence-Snapshot
docs/SECURITY_AUDIT.md	docs/archive/evidence/SECURITY_AUDIT.md	historischer Evidence-Snapshot
docs/changelog-dev.md	docs/release/changelog-dev.md	aktives Release-Artefakt
docs/changelog-prod.md	docs/release/changelog-prod.md	aktives Release-Artefakt

Bewusste Root-Ausnahme

`docs/DATENSCHUTZ_PROTOKOLL.md` bleibt temporär am Root, weil `MarkdownOrchestrator` diesen exakten Pfad als initialen Runtime-Lese Pfad verwendet. Das Dokument ist fachlich Compliance-/Privacy-Evidence; die spätere Verschiebung nach `docs/compliance/` benötigt eine gekoppelte UI-Pfadmigration.

Regeln

- Archivdateien sind Evidence, keine aktuelle Authority.
- Neue fachliche Dokumente gehören grundsätzlich in einen bestehenden Fachordner.
- Root-Dateien benötigen einen dokumentierten Entry-Point-, Kompatibilitäts- oder Governance-Grund.
- Keine historische Datei wird gelöscht, solange ihr Evidence-Wert nicht nachweislich entfallen ist.

English

Legacy root documentation has been consolidated into domain-specific archive locations while preserving Git content and evidence. Active changelogs move to `docs/release/`. Archived documents are explicitly non-canonical; ESS, ADRs and current architecture/API evidence remain authoritative.

`docs/DATENSCHUTZ_PROTOKOLL.md` is a documented temporary root exception because the current `MarkdownOrchestrator` reads that exact path at runtime. Its later move to `docs/compliance/` must be coupled with a UI path migration.

M4 Agent IAM — Implementation Plan

Quelle: docs/architecture/M4_AGENT_IAM_IMPLEMENTATION_PLAN.md

M4 Agent IAM — Implementation Plan

Status: IN PROGRESS Date: 2026-08-11 Authority: ADR-0058, ADR-0050, ADR-0051, ESS-0019 Roadmap phase: M4 Agent IAM

Objective

Implement the provider-neutral Agent IAM control boundary required by the CAPITAL-AI DevelopmentChain. Authorization is based on attributable principals, explicit capabilities and risk classes, never on provider/model identity or prompt instructions.

Canonical capabilities

- READ
- ANALYZE
- PLAN
- BRANCH
- COMMIT
- PR
- CI_REQUEST
- DEPLOY_REQUEST
- PRODUCTION_MUTATION

`MERGE` is intentionally not an agent capability. Merge remains a separately authorized Human/Owner-controlled action under `docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md`.

Risk classes

Class	Default handling
LOW	permitted only when explicitly granted
MEDIUM	permitted only when explicitly granted and attributable
HIGH	explicit approval / step-up required
CRITICAL	explicit Human/Owner authorization / step-up required; agent self-approval forbidden

All unspecified capabilities and principals are denied by default.

Identity binding

Every authorization decision must be attributable to:

1. human actor;
2. app/client;
3. agent/session;
4. tool credential holder;
5. requested capability;
6. risk class;
7. target resource/action.

Provider names and model names are metadata only and MUST NOT become authorization principals.

Implementation slices

M4.1 Capability contract

Create a typed canonical capability/risk contract and a deny-by-default authorization result. No implicit inheritance between capabilities is allowed.

Acceptance: - unknown capability => DENY; - missing principal => DENY; - missing grant => DENY; - capability grants are explicit and enumerable.

M4.2 Principal context

Introduce an attributable AgentPrincipalContext that binds human actor, client, agent/session and credential holder. Retrieved or model-generated content cannot mutate this context.

Acceptance: - principal context is immutable for an authorization decision; - provider/model metadata cannot elevate privilege; - missing attribution fails closed.

M4.3 Risk authorization

Map requested capabilities/actions to LOW/MEDIUM/HIGH/CRITICAL handling. HIGH and CRITICAL actions require explicit approval/step-up evidence according to policy.

Acceptance: - no self-approval; - expired/missing approval => DENY; - approval is scoped to actor, action and target; - approval cannot be reused for a different capability.

M4.4 Git capability separation

Agent execution may receive BRANCH, COMMIT, PR and CI_REQUEST independently. MERGE remains outside the agent capability vocabulary and is enforced through the Human/Owner PR gate.

Acceptance: - agent can prepare a PR without merge authority; - successful CI does not create merge authority; - ☐ / okay Owner review remains required for the current PR head; - a new push invalidates prior current-commit approval evidence.

M4.5 Production boundary

DEPLOY_REQUEST and PRODUCTION_MUTATION remain separate capabilities. Development agents must not directly mutate Stripe, Supabase or Render production configuration. Such work is represented as production handoff/instructions until separately authorized in the production path.

Acceptance: - DEPLOY_REQUEST does not imply PRODUCTION_MUTATION; - production mutation without explicit production authorization => DENY; - no development credential can silently become a production credential.

M4.6 Negative tests and evidence

Add tests proving privilege non-inheritance and denial of unauthorized HIGH/CRITICAL actions.

Minimum negative cases: - READ does not imply ANALYZE/PLAN/write; - COMMIT does not imply PR; - PR does not imply CI_REQUEST; - CI_REQUEST does not imply DEPLOY_REQUEST; - DEPLOY_REQUEST does not imply PRODUCTION_MUTATION; - no capability implies MERGE; - provider/model identity does not grant capabilities; - retrieved content cannot alter authorization; - agent cannot approve itself; - missing/expired step-up fails closed.

M4 closure gate

M4 can move to COMPLETE only when:

- capability and principal contracts are implemented;
- deny-by-default enforcement is code-backed;
- negative tests are green;
- Human/Owner merge separation is preserved;
- ROADMAP and M0-M9 traceability are updated with implementation evidence;
- no production Stripe/Supabase/Render mutation was introduced from the development branch;
- the PR passes technical/governance checks and the Human/Owner gate.

After M4 closure, M5 Observability/Telemetry/Audit becomes the next authorized phase.

M8 (I1) — Was ein "realer Aufrufer" für `claude-code-cli` bedeuten müsste

Quelle: docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md

M8 (I1) — Was ein "realer Aufrufer" für `claude-code-cli` bedeuten müsste

Status: **DESIGN-DISKUSSION** — keine Implementierung, keine Mutation, kein neuer realer Aufrufer Datum: 2026-08-15 Roadmap phase: I1 (docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md , Option B: „Claude-Code-Caller / Design-Evidence“) Authority: docs/runbooks/M8_AGENT_CUTOVER.md , src/platform/Security/providerProfile.ts , docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md (Modell A/B), docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md §1.2

0. Zweck

docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md §3.2 stellt fest, dass `claude-code-cli` `BLOCKED` bleibt, weil kein realer Aufrufer existiert — 4 von 6 `ProviderCutoverEvidence`-Feldern fehlen strukturell. Dieses Dokument formalisiert **was genau** „realer Aufrufer“ für dieses eine Profil bedeuten müsste, und warum das keine triviale Nachbau-Aufgabe der SA3B/SA4-Vorlage ist. Es entscheidet nichts und baut nichts — das bleibt eine separate, vom Owner zu autorisierende Folgearbeit.

1. Warum die interaktive Sitzung selbst niemals zählt

Bereits abschließend dokumentiert (M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md §1.2): jede interaktive Claude-Code-Sitzung — einschließlich dieser — erhält ihren MCP-Tool-Zugriff von der äußeren CCR-/Sitzungs-Laufzeitumgebung, nicht durch `agentIam.ts` oder `providerProfile.ts` vermittelt. Es gibt kein In-Repo-Credential, das diese Sitzung selbst als „Aufrufer“ registrieren, gaten oder deaktivieren könnte. Ein „realer Aufrufer“ im Sinne von `realCallerVerified` muss daher zwingend etwas **anderes** sein als „ein Mensch chattet mit Claude Code“ — er muss ein code-adressierbarer, vom Repository selbst gateder Ausführungspfad sein, analog zu SA3B/SA4.

2. Zwei strukturell verschiedene Wege, das zu bauen

2.1 Weg 1 — Claude als reiner Content-Generator innerhalb eines Model-A-Work-Package (buildbar heute)

Der bestehende ADR-0074-Katalog (scripts/systemadmin/workPackages/) erlaubt bereits, dass ein Work-Package-Modul einen beliebigen deterministischen `generate()`-Aufruf macht — Node-Code, der Markdown/Code-Text produziert. **Nichts hindert ein solches Modul daran, innerhalb von `generate()` einen einzelnen, nicht-agentischen Anthropic-API-Aufruf zu machen** (fixer System-Prompt, fixe Eingabe, reine Textantwort, **kein** Tool-

Zugriff für das Modell selbst), dessen Ausgabe anschließend — wie jede andere `generate()`-Ausgabe auch — deterministisch validiert und vom umgebenden Script committet wird. Das Modell bekäme dabei **keinerlei** eigene Datei-, Shell- oder Git-Capability; die gesamte BRANCH→COMMIT→PR-Kette bliebe exakt wie heute vollständig skriptgesteuert und IAM-gegated.

Sicherheitseigenschaft: identisch zu Modell A — kein neuer Autorisierungsmechanismus nötig, weil das Modell nie einen eigenen Tool-Aufruf tätigt, der vermittelt werden müsste. Das ist **kein** Modell-B-Host.

Aber: Das wäre ehrlicherweise kein „`claude-code-cli`“-Aufrufer im Sinne der `ProviderProfile`-appld — es wäre ein bloßer Anthropic-API-Textgenerierungsaufruf innerhalb eines Model-A-Work-Package, näher verwandt mit einer neuen `appId` wie `anthropic-api-content-generation` als mit „Claude Code, das mit eigenen Werkzeugen agiert“. Die `claude-code-cli`-Profildefinition selbst (`src/platform/Security/providerProfile.ts:101`) modelliert explizit einen Agenten mit Tool-Zugriff (`allowedCapabilities` umfasst `BRANCH / COMMIT` etc.), nicht eine reine Textgenerierungs-Funktion. Weg 1 würde daher entweder (a) ein neues, enger gefasstes Profil für reine Content-Generierung ohne mutierende Capability rechtfertigen (analog zu `google-ai-studio`s Development-Plane-Einstufung — vermutlich `NOT_APPLICABLE` für privilegierten Cutover, weil keine mutierende Capability benötigt wird), oder (b) das bestehende `claude-code-cli`-Profil bliebe weiterhin `BLOCKED`, weil Weg 1 es nicht tatsächlich bedient.

2.2 Weg 2 — Echter agentischer Claude-Code-Host mit eigenem Tool-Zugriff (heute nicht buildbar)

Das wäre die wörtliche Bedienung des `claude-code-cli`-Profils: ein GitHub-Actions-Workflow, der Claude Code (CLI oder Agent SDK) headless mit echtem Datei-/Shell-/Git-Zugriff aufruft, das anhand einer Aufgabenbeschreibung tatsächlich autonom Code liest, schreibt, testet und committet — exakt das in `GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md` als „Modell B“ beschriebene und als **nicht sicherheitsarchitektonisch tragfähig** bewertete Szenario.

Der dortige Blocker gilt hier unverändert und uneingeschränkt: die heutige Autorisierungskette prüft eine grobkörnige Absicht **einmal** vorab und erlaubt danach genau **eine** Git-Endoperation; ein echter Agentenlauf tätigt aber Dutzende bis Hunderte individuelle Tool-Aufrufe, von denen keiner gegen `mandate.allowedPaths / allowedCapabilities` geprüft wird. Diese Lücke ist identisch mit der Lücke, die diese Sitzung selbst hat (§1) — ein GitHub-Actions-interner Host hätte sie zusätzlich **ohne** den einzigen Kompensationsfaktor, den die interaktive Sitzung heute bietet: einen Menschen, der in Echtzeit zusieht.

Ein `claude-code-cli`-Aufrufer entlang Weg 2 wäre daher — unverändert gegenüber der bestehenden Modell-B-Bewertung — erst zulässig, nachdem mindestens Anforderung 1 aus `GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md` §4 (Tool-Call-Vermittlungsschicht) real existiert und getestet ist. Das ist eine eigene, deutlich größere Architekturaufgabe, kein Bestandteil dieses Dokuments und keine I1-Aufgabe.

3. Bewertung gegen ProviderCutoverEvidence

Weg	realCallerVerified erreichbar?	Bedient tatsächlich claude-code-cli -Profil?	Aufwand
1 — Content-Generator in Model-A-Work-Package	Ja, aber für ein anderes , enger gefasstes Profil (reine Textgenerierung ohne Tool-Zugriff)	Nein — <code>claude-code-cli</code> selbst bleibt unbedient	Klein, mit bestehender ADR-0074-Infrastruktur sofort machbar, falls Owner ein solches Profil separat definieren möchte
2 — Echter agentischer Host	Ja, aber erst nach einer eigenständigen, großen Sicherheitsarchitektur-Erweiterung	Ja — das ist die einzige Weg, die das Profil wörtlich bedient	Groß, eigenes ADR erforderlich, aktuell nicht sicherheitsarchitektonisch tragfähig (siehe Referenzdokument §3)

Ergebnis: Für das `claude-code-cli`-Profil, wie es heute in `providerProfile.ts` definiert ist, gibt es aktuell keinen sicherheitsarchitektonisch vertretbaren Weg zu einem realen Aufrufer. Weg 1 löst ein anderes Problem (Content-Generierung), Weg 2 ist durch eine bereits dokumentierte, ungelöste strukturelle Lücke blockiert.

4. Empfehlung

- `claude-code-cli` bleibt für privilegierten Cutover **BLOCKED** — das ist der korrekte, sicherheitsarchitektonisch begründete Zustand, keine offene Aufgabe, die „einfach noch erledigt werden muss“.
- Falls der Owner reine LLM-Content-Generierung innerhalb von Model-A-Work-Packages will (Weg 1), wäre der richtige nächste Schritt ein neues, eng gefasstes Provider-Profil (z. B. `anthropic-api-content-generation`, keine mutierende Capability, analog zu `google-ai-studio`) — nicht eine Erweiterung von `claude-code-cli`. Das wäre eine separate, klein scoped Owner-Anfrage, kein Bestandteil dieses Dokuments.
- Ein echter `claude-code-cli`-Aufrufer (Weg 2) bleibt an die in `GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md` §4 aufgeführten 7 Anforderungen gebunden, speziell Anforderung 1 (Tool-Call-Vermittlungsschicht) als Blocker. Diese Sitzung empfiehlt **nicht**, diese Architekturarbeit im Rahmen von I1 zu beginnen — das wäre ein eigenständiges, vom Owner zu priorisierendes Vorhaben mit eigenem ADR.

5. Auswirkung auf I1 / Exit Gate

Keine. Exit-Gate-Punkt 2 (`docs/runbooks/M8_AGENT_CUTOVER.md`) bleibt unverändert **PARTIAL**. `ProviderCutoverEvidence` für `claude-code-cli` bleibt unverändert **BLOCKED** (`docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md` §3.2). Dieses Dokument liefert ausschließlich eine Design-Klarstellung, keine neue Evidence, keinen neuen Code.

Verwandte Dokumente

- `docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md` (I1, Option B)
- `docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md`

- `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` §1.2
- `docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md` (Modell A/B)
- `src/platform/Security/providerProfile.ts`

CAPITAL-AI Observability & Telemetry — O1 Baseline

Quelle: docs/architecture/OBSERVABILITY_TELEMETRY_O1_BASELINE.md

CAPITAL-AI Observability & Telemetry — O1 Baseline

Stand: 2026-08-10

Status: Implementation Baseline

Architecture Decision: ADR-0056

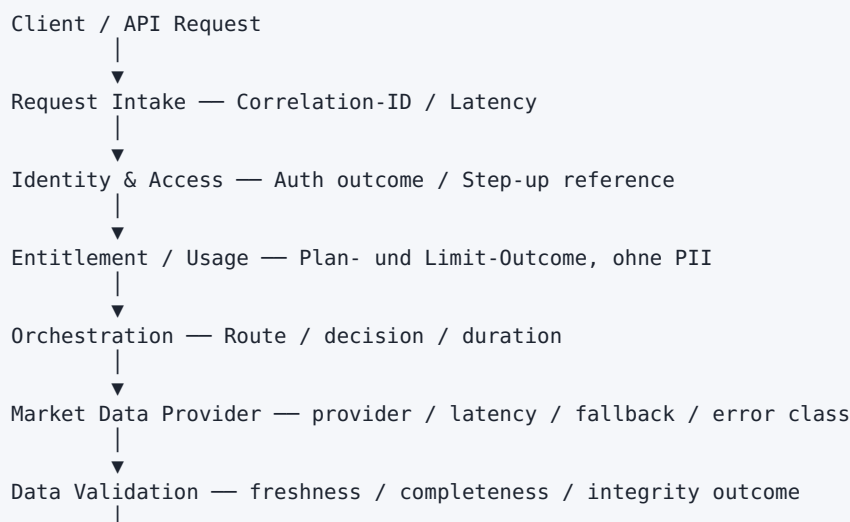
1. Ausgangslage

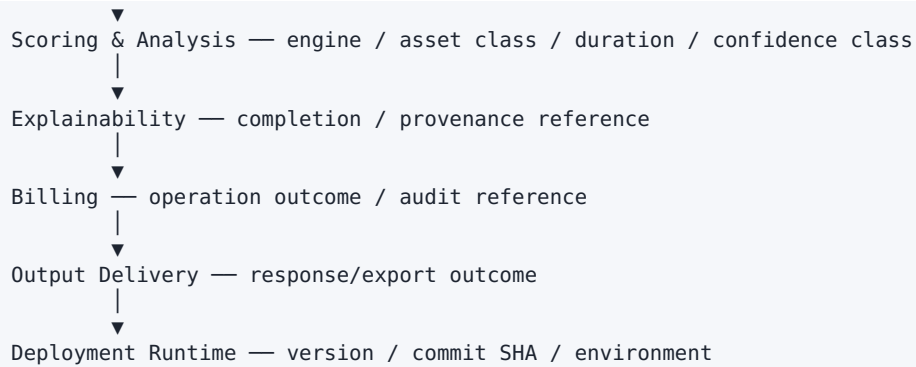
Die Codebasis besitzt bereits wichtige Observability-Einzelbausteine, jedoch keine einheitliche Telemetry-Authority:

- `server/logger.ts` : JSON-Logging, Log-Level, Request-ID und Deployment-Identität;
- IAM/Step-Up: Security-/Audit-Evidence;
- `src/services/screeningSla.ts` : Screening-SLA-/Betriebslogik;
- `src/services/marketDataProviderRouter.ts` : Provider Routing und Failure Domains;
- `src/platform/Compliance/screeningGovernanceEvidence.ts` : Compliance-/Screening-Evidence;
- `src/platform/Supervisor/supervisor.ts` : Supervisor-Auswertung;
- `src/platform/EventMesh` : Domain-Event-Infrastruktur;
- GitHub Actions → Render Deploy Hook + `server/deploymentIdentity.ts` : Production Deployment Evidence.

Gleichzeitig existieren noch zahlreiche direkte `console.*`-Ausgaben. `src/platform/Telemetry` war bisher ohne Code als `unspecified` registriert.

2. Zielbild





Operational Telemetry läuft quer durch diese Kette. Audit Evidence bleibt separat.

3. O1 Contract

Jeder kanonische Telemetry Record besitzt:

- Schema-Version;
- UTC Timestamp;
- Signaltyp `log | metric | trace`;
- Severity;
- Wertschöpfungsstufe;
- kanonischen Event-Namen;
- Outcome;
- optionale Duration, Provider und Assetklasse;
- Service-/Environment-/Version-/Commit-Kontext;
- optionale Request-/Trace-/Span-ID;
- redigierte Attributes;
- optional nur eine Referenz auf separate Audit Evidence.

4. Datenschutz und FinTech-Sicherheitsgrenze

Operational Telemetry darf keine Secrets, Auth Header, Cookies, API Keys, Passwörter, Stripe Secret Keys, Service-Role-Tokens oder direkte PII enthalten. Der generische HTTP-Hook zeichnet keine Bodies und keine Query-Strings auf.

Dies ist bewusst strenger als ein normales Application Log, da Provider-Requests, Portfolio-/Asset-Kontext, Billing und IAM in einer FinTech-Anwendung besonders hohe Leakage-Risiken besitzen.

5. Bestehende Authorities bleiben erhalten

Domäne	Authority	O1-Verhalten
Logging	<code>server/logger.ts</code>	bleibt zentrale Runtime-Logger-Authority

Domäne	Authority	O1-Verhalten
IAM Audit	Security/IAM + DB Audit	nicht in Telemetry replizieren
Compliance Evidence	Compliance Platform	nur referenzieren
Event Lifecycle	EventMesh	nicht durch Telemetry-Events ersetzen
Traceability	Documentary/Traceability	Architektur-/Coverage-Evidence
Deployment	GitHub CI + Render Hook + deploymentIdentity	Commit/Version in Telemetry-Kontext übernehmen
Provider Routing	marketDataProviderRouter	O3/O4 instrumentieren
Screening SLA	screeningSla	O3 in Metriken überführen

6. O1 Codeänderungen

- `src/platform/Telemetry/contracts.ts`
- `src/platform/Telemetry/redaction.ts`
- `src/platform/Telemetry/index.ts`
- `tests/unit/telemetryContract.test.ts`
- Integration in `server/logger.ts`
- Aktualisierung von README und Manifest
- ADR-0056

7. Messbarer Exit-Status

Nach O1 soll der Observability-/Telemetry-Reifegrad von ca. 45 % auf ca. 55 % steigen. Dafür werden noch keine externen Dashboards oder Collector benötigt. Der Wert entsteht durch einen belastbaren Contract, Datenminimierung, Runtime-Korrelation und eine explizite FinTech-Wertschöpfungszuordnung.

8. Nächster Workstream O2

O2 migriert die wichtigsten verbleibenden `console.*`-Pfade risikobasiert auf `createLogger` und führt eine Error Taxonomy ein. Priorität:

1. Provider / Market Data;
2. Orchestrator / Decision Engine;
3. Stripe / Billing;
4. Mailer / Password Reset;
5. DB / Supabase;
6. verbleibende Security-/Runtime-Pfade.

Keine Big-Bang-Migration: pro PR nur klar abgegrenzte Wertschöpfungssegmente mit vorhandener Test-Evidence.

CAPITAL-AI — Open Pull Request Consolidation Evidence

Quelle: docs/architecture/OPEN_PR_CONSOLIDATION_2026-08-04.md

CAPITAL-AI — Open Pull Request Consolidation Evidence

Date: 2026-08-04

Repository: SvenKulessa/Finance

Integration branch: integration/consolidated-open-prs-2026-08-04

Base after direct CSP merge: main@6bcab71c16fe5691688f2d58b4daa30aa51eb716

Release version: 0.6.0

1. Purpose

This document records the conflict-safe consolidation of the previously open Pull Requests into one reviewable integration branch. The consolidation preserves source history, prevents duplicate-file overwrite, resolves document-number collisions and separates GitHub code integration from Supabase production mutation.

2. Directly merged prerequisite

PR #88 (fix: remediate ADR-0035 CSP runtime and safe rollout) was directly merged into main on explicit owner instruction with expected-head protection.

- source head: 73e64892547da8ab39c3ca2eb7e4b02e73e712cd ;
- resulting main merge commit: 6bcab71c16fe5691688f2d58b4daa30aa51eb716 ;
- the consolidation branch was created from this exact commit.

3. Source PR classification

PR	Classification	Integration decision
#78	Source	Render runtime remediation merged into the integration branch.
#80	Source	Stateless Render web-tier audit merged as documentation/evidence.
#81	Exact duplicate	Closed without merge because it had the same head SHA and same ten changed files as #85.
#83	Source	Server modularization scaffold merged as an isolated source set.
#84	Source with shared-file conflict	Version Manager convention validator merged; shared governance workflow later reconciled with #86.

PR	Classification	Integration decision
#85	Canonical successor to #81	Binance landing analysis and ESS-0015 API inventory merged.
#86	Source with shared-file conflict	Six conflict-free files carried over; <code>pr-governance.yml</code> semantically reconciled with #84; original PR closed as superseded.
#87	Source with identifier collision	Content preserved under ESS-0016 and ADR-0041; original colliding ESS-0015/ADR-0038 assignments not merged.
#88	Direct prerequisite	Merged directly to <code>main</code> before consolidation.

4. Identical-file and overlap controls

4.1 PR #81 versus PR #85

Both PRs pointed to the exact source head:

```
2325de7fd855d4a32b949e8b3490e2c34eaf2542
```

Both changed the same ten files. Only PR #85 was integrated. PR #81 was closed as an exact duplicate, preventing double application and preserving the canonical history in one source PR.

4.2 PR #84 versus PR #86

Both changed:

```
.github/workflows/pr-governance.yml
```

The final workflow preserves both intended responsibilities:

- dependency installation and production vulnerability audit;
- TypeScript, tests, production build and deployment-readiness as blocking technical gates;
- Version Manager repository-convention validation as advisory;
- changed-workflow security against trusted `main`;
- immutable Action SHAs and `persist-credentials: false`;
- production drift and optional work claims as advisory process evidence rather than build authorization;
- explicit human authorization for PR creation and protected merges.

No complete version of one PR's workflow was allowed to overwrite the other.

4.3 PR #85 versus PR #87 identifier collision

PR #85 already assigned:

```
ESS-0015 = API Interface Inventory & QA Governance  
ADR-0038 = Binance Landing AI Quick Analysis
```

PR #87 attempted to use the same identifiers for a different market-data architecture. Its content was therefore migrated without semantic loss to:

```
ESS-0016 = Enterprise Market Data Provider & MCP Governance  
ADR-0041 = Enterprise Market Data Provider & MCP Architecture
```

The ESS registry now advances the next free number to `ESS-0017`.

5. Version integrity

The integration branch retains one release identity:

```
package.json      name=capital-ai version=0.6.0  
package-lock.json name=capital-ai version=0.6.0
```

PR #84 adds repository validation scripts but does not change the release version. No dependency declaration was changed without a corresponding lock-file state.

6. Supabase boundary

The connected production project is `AIFINANCIAL` (`ryzywoktpmyhwzxmstyu` , `eu-west-1`). Its migration history was inspected read-only through the latest recorded migration:

```
20260802172425 harden_stripe_function_search_paths
```

The consolidated GitHub changed-file set contains no Supabase migration, schema or Edge Function file. Therefore this consolidation performs:

- no production DDL;
- no production DML;
- no branch merge in Supabase;
- no Edge Function deployment;
- no authentication-policy mutation;
- no secret or publishable-key mutation.

Supabase findings contained in the API QA documentation remain evidence and follow-up work, not implicitly applied production changes.

7. Code-loss prevention rules applied

1. Source PRs were retargeted to a dedicated integration branch rather than merged independently into `main`.

2. Every retargeted merge used the source PR's immutable expected head SHA.
3. Exact duplicates were closed rather than applied twice.
4. Shared files were semantically reconciled instead of accepting last-writer-wins behavior.
5. ESS/ADR collisions were renumbered and registered before the final PR.
6. `package.json` and `package-lock.json` remain aligned at version `0.6.0`.
7. Supabase production state remained read-only.
8. The final integration branch is ahead of, and not behind, its `main` base.

8. Final validation gate

The final consolidation Pull Request must remain a Draft until the integration head has been evaluated by:

- repository CI;
- TypeScript/type checking;
- the complete unit-test suite;
- production build;
- deployment-readiness checks;
- protected Google Marketing/CSP checks where triggered;
- changed-workflow security;
- repository-convention advisory output;
- human/CODEOWNER review for protected architecture and workflow changes.

A green build is technical evidence only and does not itself authorize merge or deployment.

Phase 3.3 — AI Analysis Route Extraction Note

Quelle: docs/architecture/PHASE-3.3-ai-analysis-extraction-note.md

Phase 3.3 — AI Analysis Route Extraction Note

Status: Implementing

Date: 2026-08-08

Parent decision: [ADR-0083 — Server Runtime Architecture Consolidation](#)

Formerly titled `ADR-0014 Phase 3.3`. The ADR-0014 number is reserved for Documentation Governance Validator. Parent is ADR-0083.

Scope

This note records the Phase 3.3 implementation boundary under ADR-0083. It does not supersede ADR-0083.

The following compatibility-owned endpoints are extracted into canonical route factories under `server/routes/`:

- `GET /api/market-sentiment`
- `POST /api/market-sentiment/simulate-shock`
- `POST /api/portfolio-review`

Provider invariants

`GET /api/market-sentiment` remains Gemini-specific because it uses Google Search Grounding. It MUST NOT be normalized into the generic multi-provider fallback chain unless the grounding architecture itself is changed by a separate decision.

The shock simulation and portfolio review remain structured-reasoning workloads on the established Anthropic -> OpenAI -> Gemini fallback chain through `generateStructuredWithFallback`.

Parallel-work protection

This extraction does not modify `server.application.ts`, Render deployment configuration, Secret File validation, Stripe ingress, runtime artifact guards or scoring services. The later cutover that removes the equivalent inline handlers from `server.application.ts` must be performed from a freshly verified `main` head.

Next step

After this additive extraction passes TypeScript, tests, production build and deployment-readiness gates, mount the new routers through the canonical application route composi-

on and remove only the equivalent inline compatibility handlers. Then continue with market-data/provider adapter extraction.

Phase 3.4 — Market-data adapter extraction (v2 note)

Quelle: docs/architecture/PHASE-3.4-market-data-adapters-v2.md

Phase 3.4 — Market-data adapter extraction (v2 note)

Status: In progress

Date: 2026-08-08

Parent decision: [ADR-0083 — Server Runtime Architecture Consolidation](#)

Formerly titled `ADR-0014 Phase 3.4 (v2)`. Parent is ADR-0083.

Phase 3.4 moves provider-facing HTTP adapters out of `server.application.ts` without changing underlying provider implementations or scoring semantics.

The first extracted adapter is `/api/alpha-vantage-quote`, now owned by `server/routes/alpha-VantageRoutes.ts` and mounted via `server/routes/registerMarketDataAdapters.ts`.

Authoritative provider modules remain unchanged: `server/stockFundamentals.ts` for Alpha Vantage fundamentals, `server/fmpIndices.ts` for FMP index data, and `src/lib/assetRegistry.ts` for registry/history behavior. R-001 scoring services remain outside this boundary.

The adapter layer may normalize external provider responses but must not own scoring, runtime-secret validation, Stripe ingress, startup lifecycle, or runtime artifact governance.

Preserved invariants include API-key redaction in logs, explicit Alpha Vantage rate-limit/error behavior, separate crypto/traditional quote modes, R-001 scoring provenance, R-002 runtime immutability, R-003 Stripe ownership, and PR #109/#112 secret validation.

Next: extract `fetchLiveMarketData()` and its provider coordination behind narrowly tested service boundaries before removing any inline compatibility block.

Phase 3.4 — Market-data adapter extraction

Quelle: docs/architecture/PHASE-3.4-market-data-adapters.md

Phase 3.4 — Market-data adapter extraction

Status: In progress

Date: 2026-08-08

Parent decision: [ADR-0083 — Server Runtime Architecture Consolidation](#)

Formerly titled `ADR-0014 Phase 3.4`. The ADR-0014 number is reserved for Documentation Governance Validator. Parent is ADR-0083.

Scope

Phase 3.4 continues the monotonic decomposition of `server.application.ts` by moving provider-facing HTTP adapters into canonical server modules without changing the underlying provider implementations or scoring semantics.

Decision

The first adapter extracted is `/api/alpha-vantage-quote`, now owned by `server/routes/alpha-VantageRoutes.ts`.

Existing provider modules remain authoritative and are not duplicated:

- `server/stockFundamentals.ts` owns Alpha Vantage stock-fundamental caching/fetching;
- `server/fmpIndices.ts` owns FMP index quote/history caching/fetching;
- `src/lib/assetRegistry.ts` remains the registry/history authority used by other routes;
- R-001 scoring services remain outside the provider-adapter boundary.

The HTTP adapter layer may translate provider responses into CAPITAL-AI API responses, but it must not absorb scoring logic, runtime-secret validation, Stripe ingress, startup lifecycle or runtime artifact ownership.

Preserved invariants

- API keys are never logged; request URLs redact provider credentials.
- Alpha Vantage rate-limit and provider error semantics remain explicit.
- Crypto and traditional-asset quote request modes remain distinct.
- R-001 scoring provenance is untouched.
- R-002 runtime immutability is untouched.
- R-003 Stripe ownership is untouched.
- PR #109/#112 runtime-secret validation remains untouched.

Next extraction

After this additive adapter is green, continue with the larger market-data aggregation functions in `server.application.ts` (`fetchLiveMarketData()` and dependent provider coordination) behind narrowly tested service boundaries before any inline block is removed.

Phase 3.4.4 — Crypto/Stooq Provider Stages

Quelle: docs/architecture/PHASE-3.4.4-crypto-stooq-provider-stages.md

Phase 3.4.4 — Crypto/Stooq Provider Stages

Status: Implementing

Parent decision: [ADR-0083 — Server Runtime Architecture Consolidation](#)

Formerly titled `ADR-0014 Phase 3.4.4`. Parent is ADR-0083.

This phase wires the already extracted crypto provider chain behind the canonical `Market-DataProviderStage` contract and extracts Stooq into the same stage model.

Invariants:

- Crypto provider priority remains Binance -> Kraken -> Coinbase -> explicit static fallback.
- Stooq remains the authoritative HTTP provider for stocks, forex and mapped commodities in this compatibility path.
- Missing Stooq volume is not synthesized; existing fallback metadata may be retained, but the provider stage does not fabricate new market evidence.
- FMP index coordination remains a separate stage and separate change.
- R-001 score provenance, R-002 runtime immutability, R-003 Stripe ownership and the temporary TOTP secret diagnostics from PR #118 are outside this workstream.
- `server.application.ts` is not modified in this phase. Its compatibility cutover remains a separately validated change after all provider stages exist.

Phase 3.4.6 — Market-Data Compatibility Facade

Quelle: docs/architecture/PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md

Phase 3.4.6 — Market-Data Compatibility Facade

Status: Accepted for cutover preparation

Parent decision: [ADR-0083 — Server Runtime Architecture Consolidation](#)

Formerly titled `ADR-0014 Phase 3.4.6`. Parent is ADR-0083.

Context

The legacy `fetchLiveMarketData()` in `server.application.ts` still combines provider I/O, registry fallback completion, scoring/enrichment and best-effort side effects. The provider implementations have already been extracted behind `MarketDataProviderStage`, but replacing the legacy function directly would risk losing enrichment, snapshot validation and alert evaluation semantics.

Decision

Introduce `server/marketData/marketDataCompatibilityFacade.ts` as the final composition boundary before the shared-file cutover.

The facade:

- delegates provider execution to the canonical market-data coordinator;
- uses the canonical provider stage factory unless stages are explicitly injected for tests;
- appends registry assets missing from provider results as explicit `dataSource: fallback` records;
- delegates scoring/domain enrichment through an injected `enrichAsset` function;
- keeps score snapshot persistence and alert evaluation best-effort;
- propagates provider-stage failure reporting without failing the complete refresh.

Protected invariants

- R-001 scoring provenance remains owned by existing scoring services and the injected enrichment function.
- R-002 runtime immutability is unchanged.
- R-003 Stripe event ownership is unchanged.
- No Render, secret-validation or startup security behavior is modified.
- Registry/bootstrap data is never relabeled as verified live market evidence.

Cutover gate

After CI and Technical Validation are green, `server.application.ts` may replace the provider/fallback/enrichment composition inside `fetchLiveMarketData()` with one call to `runMarketDataCompatibilityRefresh(...)`. Cache/coalescing and AssetRegistry synchronization remain separate behavior-preserving concerns until their own extraction.

ADR-0014 Phase 3.4.3 — Crypto Provider Chain Extraction

Quelle: docs/architecture/phase3-4-3-crypto-provider-extraction.md

ADR-0014 Phase 3.4.3 — Crypto Provider Chain Extraction

Status: In Progress Date: 2026-08-08

Scope

Extract the concrete crypto market-data provider chain from `server.application.ts` without changing scoring, provenance or runtime behavior.

Canonical provider order

1. Binance public 24h ticker API
2. Kraken public ticker API
3. Coinbase public spot API
4. Static fallback assets with explicit `dataSource: 'fallback'`

Invariants

- Provider failures are fail-open and advance to the next provider.
- Binance remains preferred when it returns BTC or ETH live data.
- Kraken is the second source.
- Coinbase is the final live source and MUST NOT invent 24h change or volume values; unknown values remain zero because that endpoint does not provide them.
- Static fallback data MUST remain explicitly labelled `fallback`.
- This extraction does not alter R-001 scoring provenance, R-002 runtime immutability, R-003 Stripe event ownership, Render startup, runtime-secret validation or FMP/Stooq provider behavior.

Cutover rule

`server.application.ts` is not modified in this additive PR. Wiring the extracted provider stage into `marketDataCoordinator` and deleting the equivalent inline implementation is a separate overlap-checked cutover.

CAPITAL-AI — Pull Request & Multi-Agent Governance

Quelle: docs/architecture/PR_MULTI_AGENT_GOVERNANCE.md

CAPITAL-AI — Pull Request & Multi-Agent Governance

Status: DRAFT — implementation in progress

Authority: Platform Director / Release / Security & Compliance

Purpose: Establish a single-writer, PR-first lifecycle for concurrent AI-agent development.

This file is intentionally created as the first branch commit so that the governance change itself is developed through a Pull Request rather than direct writes to `main`.

The complete normative content is added in this Pull Request together with the PR template, agent lease registry, production-state preflight and CI enforcement.

CAPITAL-AI — Render Production Configuration Audit

Quelle: docs/architecture/RENDER_PRODUCTION_CONFIGURATION_AUDIT.md

CAPITAL-AI — Render Production Configuration Audit

Audit ID: RENDER-AUDIT-0001

Audit date: 2026-08-03

Status: COMPLETE FOR AVAILABLE EVIDENCE / DASHBOARD EVIDENCE PENDING

Priority: HIGH

System: CAPITAL-AI / `capital-ai.online`

Repository: `SvenKulessa/Finance`

Related ADR: ADR-0037

Related runbook: `docs/runbooks/DEPLOYMENT_ROLLBACK_UND_BACKUP.md`

1. Zweck

Dieses Dokument beschreibt die Render-Produktionsarchitektur von CAPITAL-AI so vollständig, dass zukünftige Änderungen, Deployments, Rollbacks und Modulerweiterungen nicht mehr von implizitem Wissen im Render-Dashboard abhängen.

Ziel ist eine belastbare Trennung zwischen:

1. **Repository-/Infrastructure-as-Code-Wahrheit** (`render.yaml` , Dockerfile, Server-Code, CI);
2. **Render-Plattform-Wahrheit** (Service-, Environment-, Deploy-, Skalierungs- und Security-Einstellungen);
3. **Runtime-Wahrheit** (welcher Build/Commit läuft tatsächlich, welche Abhängigkeiten sind betriebsbereit);
4. **externen Provider-Abhängigkeiten** (Supabase, Stripe, SMTP, AI-/Market-Data-Provider);
5. **operativer Wahrheit** (Monitoring, Rollback, Incident Response, Secret Rotation).

Damit soll verhindert werden, dass ein Agent, ein späterer Entwickler oder ein Modell eine lokale Einstellung für die vollständige Produktionsarchitektur hält und dadurch unabsichtlich einen partiellen, nicht reproduzierbaren oder unsicheren Zustand erzeugt.

2. Audit-Grenzen und Beweisstatus

Für diese Session stand **kein Render-Connector/Plugin** zur Verfügung. Das Render-Dashboard konnte daher nicht direkt ausgelesen werden. Es werden keinerlei Dashboard-Werte erfunden.

Jede Aussage ist deshalb mit einem Evidence-Status zu lesen:

Status	Bedeutung
VERIFIED_REPO	Direkt aus dem aktuellen <code>main</code> -Repository belegt.
VERIFIED_PROVIDER	Gegen aktuelle offizielle Render-Dokumentation geprüft.
UNVERIFIED_DASHBOARD	Nur im Render-Dashboard/API verifizierbar; aktuell nicht ausgelesen.
INFERRED	Aus mehreren belegten Komponenten ableitbar, aber nicht direkt aus Render bestätigt.
RISK	Architektur-/Security-Lücke, die aus belegten Fakten entsteht.

Nicht verifizierbar in dieser Session

Insbesondere folgende aktuelle Dashboard-Werte konnten nicht direkt aus Render gelesen werden:

- Workspace-/Plan-Tier;
- Service-ID;
- tatsächliche Region;
- aktueller Instance Type;
- aktuelle Anzahl Instanzen / Autoscaling;
- Projekt- und Environment-Zuordnung;
- Protected-Environment-Status;
- Cross-Environment-Network-Isolation;
- Auto-Deploy-Einstellung;
- Overlapping Deploy Policy;
- Build Filters;
- Service Preview / Preview Environment Einstellungen;
- Docker Command Override;
- Root Directory / Dockerfile Path Overrides;
- Persistent Disk;
- `onrender.com` -Subdomain Policy;
- Custom-Domain-Liste im Dashboard;
- Maintenance Mode;
- Notification-/Webhook-Konfiguration;
- Environment Groups und Secret Files;
- manuell hinzugefügte Environment-Variablen, die nicht in `render.yaml` erscheinen.

Diese Werte werden in Abschnitt 12 als verpflichtende Production-Evidence-Checkliste geführt.

3. Verifizierter Ist-Zustand

3.1 Render Blueprint

`render.yaml` definiert aktuell genau einen Service:

```
services:
- type: web
  name: capital-ai
  runtime: docker
  healthCheckPath: /healthz
```

Evidence: `render.yaml` (`VERIFIED_REPO`).

Damit sind folgende Eigenschaften repositoryseitig belegt:

- öffentlicher Render Web Service;
- Servicename `capital-ai`;
- Docker Runtime;
- HTTP Health Check unter `/healthz`;
- Environment-Variable-Namen als `sync: false`-Placeholder.

Nicht im Blueprint festgelegt sind unter anderem Region, Plan, Branch, Auto-Deploy-Policy, Domain Policy, Shutdown Delay, Skalierung und Environment Protection.

3.2 Secrets und Runtime-Konfiguration

Der Blueprint enthält zahlreiche `sync: false` Variablen. Das ist für Secret-Werte grundsätzlich korrekt: Render hält den Wert außerhalb des Repositorys.

Belegte Gruppen:

- Supabase;
- Stripe;
- Gemini / Anthropic / OpenAI;
- Market-Data-Provider;
- SMTP;
- IAM/TOTP;
- Metrics;
- Social-Media OAuth Clients;
- Google Analytics Measurement ID.

`sync: false` bedeutet jedoch nicht, dass der Wert heute tatsächlich gesetzt oder korrekt ist. Bei bestehenden Blueprints werden neue `sync: false`-Variablen nicht automatisch befüllt; sie müssen im Dashboard gesetzt werden.

3.3 Docker Build

Der Produktionscontainer verwendet eine Multi-Stage-Struktur:

```
node:22-alpine builder
-> npm ci
-> Vite/esbuild production build

node:22-alpine runner
-> npm ci --only=production
-> non-root user capitalai
-> /app/uploads + /app/docs
-> CMD npm run start
```

Positive Punkte:

- reproduzierbarer Lockfile-Build mit `npm ci`;
- Development Dependencies werden nicht in den Runtime-Layer übernommen;
- Produktionsprozess läuft als Non-Root-User;
- `.env`, `.git` und `.github` werden über `.dockerignore` nicht in den Build-Kontext übernommen;
- Docker enthält einen lokalen Healthcheck.

3.4 HTTP Server

Der Express-Prozess bindet aktuell fest auf:

```
const PORT = 3000;
app.listen(PORT, "0.0.0.0", ...)
```

Das `0.0.0.0`-Binding ist für Render korrekt. Die feste Portnummer entspricht jedoch nicht dem empfohlenen Render-Vertrag (`process.env.PORT`, standardmäßig 10000). Render kann andere Ports häufig automatisch erkennen, aber diese Erkennung sollte nicht Teil der Architekturvertrauensbasis sein.

3.5 Render Health Check

`/healthz` antwortet derzeit mit:

- `status: ok`;
- Zeitstempel;
- Prozess-Uptime;
- `configured` Flags für Supabase, Gemini, Anthropic und OpenAI.

Der Endpoint führt bewusst keine externen Netzwerkprüfungen aus.

Das bedeutet:

- sehr gute Liveness-Eigenschaft;
- schneller Healthcheck;
- aber keine vollständige Readiness-Aussage für kritische Produktionsabhängigkeiten.

`runIamSchemaHealthCheck()` wird beim Serverstart ausgeführt, blockiert den Start bei Fehlern aber bewusst nicht. Die Anwendung kann deshalb Render-seitig `healthy` werden, obwohl bestimmte privilegierte IAM-Funktionen fail-closed nicht verfügbar sind.

3.6 Deployment / CI

Aktuelle GitHub-CI auf `main` führt aus:

```
npm ci
npm audit --omit=dev --audit-level=high
npm run lint
npm test
npm run build
npm run predeploy:check
```

Das ist eine gute technische Gate-Basis.

Ob Render aktuell mit **On Commit**, **After CI Checks Pass** oder **Off** arbeitet, ist jedoch nicht im Blueprint definiert und damit `UNVERIFIED_DASHBOARD`.

3.7 Deployment Readiness Gate

`scripts/automation/verifyDeploymentReadiness.ts` prüft unter anderem:

- Health Check im Blueprint;
- Environment-Variablen-Abdeckung;
- Migrationen;
- Lockfile / Dependency Policy / SBOM;
- Traceability;
- RAG Evidence;
- Prompt Registry.

Wichtig: Der Env-Scanner erkennt nur statische Aufrufe wie:

```
getCleanEnv('KEY')
```

Dynamische Aufrufe werden nicht vollständig erkannt.

Beispiel in `server/stripe.ts`:

```
getStripeVar('STRIPE_PRICE_ID_STARTER_YEARLY')
getStripeVar('STRIPE_PRICE_ID_PRO_YEARLY')
```

`getStripeVar()` ruft intern `getCleanEnv(key)` mit einer dynamischen Variable auf. Damit können Environment-Schlüssel vom aktuellen Regex-Gate übersehen werden.

3.8 Dateisystem und lokale Fallback-Persistenz

Render-Webservices besitzen standardmäßig ein **ephemeres Dateisystem**.

Der Code schreibt gleichzeitig lokale Fallback-Daten nach:

```
/app/uploads/subscriptions.json
```

und weitere lokale Credit-/Fallback-Zustände unter `uploads/`.

`render.yaml` definiert keinen Persistent Disk. Ob im Dashboard manuell ein Disk angebunden ist, ist aktuell nicht verifiziert.

Dies ist für Billing-/Entitlement-Daten architektonisch kritisch: Ein lokaler Fallback kann scheinbar erfolgreich geschrieben werden und bei Restart/Deploy anschließend verschwinden.

Für CAPITAL-AI sollte persistenter Business State **nicht** durch einen Render Disk gelöst werden, weil ein Disk horizontale Skalierung verhindert und Zero-Downtime-Deployments abschaltet. Persistenter Subscription-/Credit-State gehört in Supabase/Postgres; lokale Dateien dürfen höchstens Cache/diagnostische Artefakte darstellen.

3.9 Background Work im Web-Prozess

Beim Start des Webservers werden zusätzlich ausgeführt:

- Recursive Document Hygiene File Watcher;
- Market-Data Pre-Caching;
- 60-Sekunden Market-Data Refresh;
- Daily Snapshot Recording;
- Alert Evaluation.

Damit ist der Web-Prozess derzeit nicht vollständig stateless.

Bei mehreren Render-Instanzen würden diese Jobs auf **jeder Instanz** laufen. Das kann verursachen:

- doppelte Provider-Abfragen;
- doppelte Alert-Auswertung;
- konkurrierende Snapshot-Writes;
- Rate-Limit-Verbrauch;
- inkonsistente In-Memory-Caches;
- schwer nachvollziehbare Race Conditions.

Die aktuelle Anwendung darf deshalb nicht einfach durch Erhöhen von `numInstances` oder Aktivieren von Autoscaling horizontal skaliert werden, bevor diese Background-Verantwortlichkeiten getrennt wurden.

3.10 Shutdown-Verhalten

Render sendet bei Zero-Downtime-Deploys `SIGTERM` an die alte Instanz und wartet standardmäßig bis zu 30 Sekunden auf einen geordneten Shutdown.

Im aktuellen Repository wurde kein expliziter `SIGTERM`-Handler gefunden.

Dadurch fehlen kontrollierte Schritte wie:

- HTTP Server `close()` / Request Drain;
- Stoppen von Intervallen/Watchern;
- Abschluss oder Abbruch laufender Jobs;
- Schließen externer Verbindungen;
- definiertes Exit-Verhalten.

3.11 Rollback-Dokumentation

Das bestehende Runbook beschreibt den Render-Dashboard-Rollback teilweise veraltet als erneuten Build des historischen Commits.

Aktuelle Render-Dokumentation beschreibt dagegen die Wiederverwendung des Build-Artefakts eines vorhandenen erfolgreichen Deployments. Zusätzlich deaktiviert ein Dashboard-Rollback automatische Deploys als Schutz gegen sofortige Wiedereinführung des fehlerhaften Commits.

Dieser Dokumentationsdrift ist im Incident-Fall relevant und muss korrigiert werden.

4. Verifizierte Prozessketten

4.1 Source-to-Production

```

GitHub repository
  |
  v
GitHub CI
- dependencies
- SCA
- typecheck
- tests
- build
- deployment readiness
  |
  v
Render Git integration          [Auto-Deploy Policy aktuell UNVERIFIED]
  |
  v
Docker build
  |
  v
new Render instance
  |
  v
GET /healthz
  |
  +-- pass -> traffic switch
  |
  +-- fail -> old deploy remains live
  |
  v
old instance receives SIGTERM
  
```

4.2 Runtime Request Chain

```

Internet
|
  
```

```

v
Render / Cloudflare edge
- DDoS protection
- HTTP -> HTTPS
- TLS termination
|
v
Render internal proxy
|
v
Express 0.0.0.0:3000
|
+-- requestContext / security headers
+-- metrics
+-- CORS
+-- rate limits
+-- API routers
+-- React SPA/static assets

```

4.3 Configuration Chain

```
render.yaml names
|
+--> sync:false placeholders
|
|      v
|      Render Dashboard secret values
|
+--> Docker build environment
|
|      +--> explicitly declared VITE_* ARG values
|
+--> Runtime environment
|
|      v
|      getCleanEnv()
```

4.4 Persistence Chain

```
Primary durable state
-> Supabase/Postgres

Temporary process state
-> memory cache

Current fallback files
-> /app/uploads/*
-> Render filesystem (persistence UNVERIFIED; default ephemeral)
```

Normativer Zielzustand:

```
Business-critical state -> Supabase/Postgres only
Cache                   -> memory / dedicated cache
Generated export        -> object storage / bounded transient file
No entitlement truth     -> local Render filesystem
```

5. Priorisierte Findings

ID	Priorität	Finding	Status
RND-F-001	P0	Business-/Entitlement-Fallback schreibt auf po-	OPEN

ID	Priorität	Finding	Status
		tenziell ephemeres Render-Dateisystem.	
RND-F-002	P0	Horizontale Skalierung ist wegen eingebetteter Background Jobs derzeit nicht safe.	OPEN
RND-F-003	P1	Render Port Contract wird nicht verwendet; Port 3000 ist hartcodiert.	OPEN
RND-F-004	P1	<code>render.yaml</code> ist keine vollständige Service-Source-of-Truth; Dashboard Drift bleibt möglich.	OPEN
RND-F-005	P1	Auto-Deploy ist nicht als <code>checksPass</code> codiert/verifiziert.	UNVERIFIED
RND-F-006	P1	Kein explizites SIGTERM/Graceful-Shutdown-Verhalten.	OPEN
RND-F-007	P1	Health Check misst Liveness, aber nicht definierte Production Readiness.	OPEN
RND-F-008	P1	Environment-Inventar-Gate kann dynamische Env-Key-Aufrufe übersehen.	OPEN
RND-F-009	P1	Region/Plan/Instance Count/Scaling fehlen aus IaC und sind nicht reproduzierbar dokumentiert.	UNVERIFIED
RND-F-010	P1	Bestehendes Rollback-Runbook enthält veraltete Render-Rollback-Semantik.	OPEN
RND-F-011	P2	Secret-Diagnostik loggt Key-Prefixe/Längen und sollte minimiert werden.	OPEN
RND-F-012	P2	Render Notification-/Unhealthy-Alerting ist nicht dokumentiert/verifiziert.	UNVERIFIED
RND-F-013	P2	<code>onrender.com</code> -Subdomain-Policy und Domain Source-of-Truth sind nicht dokumentiert.	UNVERIFIED
RND-F-014	P2	Docker Base Image verwendet einen bewegli-	OPEN

ID	Priorität	Finding	Status
		chen Tag statt vollständig reproduzierbarem Digest.	
RND-F-015	P2	Public <code>/healthz</code> veröffentlicht Provider-Konfigurationsflags.	OPEN
RND-F-016	P2	External uptime probe / user-path smoke monitoring nicht dokumentiert.	UNVERIFIED

6. Findings im Detail

RND-F-001 — P0 — Ephemeral Business-State Fallback

`server/db.ts` schreibt Subscription-Fallbacks lokal unter `uploads/subscriptions.json` und betrachtet die lokale Speicherung bei Remote-Problemen als Fallback.

Render dokumentiert, dass das Dateisystem standardmäßig bei Deploy/Restart verloren geht.

Risiko:

- Abo-/Credit-Zustand kann nach erfolgreichem Request später verschwinden;
- Webhook-/Billing-Zustand kann zwischen Supabase und lokaler Datei divergieren;
- Restart oder Deploy wird unbeabsichtigt zu einem Daten-Lifecycle-Event.

Entscheidung:

Lokale Dateien dürfen niemals authoritative Subscription-, Billing-, IAM- oder Credit-Daten sein.

Empfehlung:

- Supabase bleibt einzige durable Source of Truth;
- bei Supabase-Schreibfehlern fail closed oder durable Queue/Retry verwenden;
- kein Render Disk als Ersatz für die Datenbank einführen, solange horizontale Skalierung/Zero-Downtime erforderlich ist.

RND-F-002 — P0 — Web- und Worker-Verantwortung vermischt

Der Webprozess führt Scheduler-/Background-Verantwortung aus.

Risiko bei Scaling: Jeder neue Web-Replica vervielfacht die Jobs.

Ziel:

Render Web Service
-> HTTP/API only

Render Background Worker / controlled scheduler
-> recurring market refresh coordination

```
-> alert evaluation
-> documentary background work where appropriate

Supabase / distributed lock
-> idempotency / leader coordination
```

Bis diese Trennung abgeschlossen ist:

`numInstances = 1` und Autoscaling dürfen nicht ohne Architekturentscheidung aktiviert werden.

RND-F-003 — P1 — Hardcoded Port

Aktuell:

```
const PORT = 3000;
```

Ziel:

```
const PORT = Number(process.env.PORT || 3000);
```

und Docker-/Healthcheck müssen denselben Portvertrag nutzen.

RND-F-004 — P1 — Unvollständige Infrastructure as Code

Aktuell codiert der Blueprint nur einen Bruchteil des Services.

Für reproduzierbare Production müssen mindestens bewusst festgelegt oder als `dashboard-managed` dokumentiert werden:

- `region`;
- `plan`;
- `branch`;
- `autoDeployTrigger`;
- `healthCheckPath`;
- `maxShutdownDelaySeconds`;
- `numInstances` / `scaling`;
- `domains`;
- `renderSubdomainPolicy`;
- `dockerfilePath` / `dockerContext` soweit relevant;
- `buildFilter`;
- Environment-/Project-Zuordnung;
- Environment Protection / Network Isolation;
- Preview Policy;
- Environment Groups.

RND-F-005 — P1 — Deploy darf CI nicht umgehen

Für Production wird empfohlen:

`autoDeployTrigger`: checksPass

Dadurch deployt Render erst nach erfolgreichen GitHub Checks. Dieser Wert muss im Dashboard verifiziert und anschließend in der IaC-Source-of-Truth festgeschrieben werden.

Wichtig: `neutral` und `skipped` gelten bei Render ebenfalls als passender Check-Ausgang. Kritische Gates dürfen deshalb nicht über Workflow-Bedingungen still `skipped` werden.

RND-F-006 — P1 — Graceful Shutdown

CAPITAL-AI benötigt einen zentralen Shutdown Coordinator.

Minimaler Zielablauf:

```
SIGTERM
-> mark not-ready
-> stop accepting new work
-> server.close()
-> clear intervals/watchers
-> allow in-flight HTTP requests to finish
-> finish/abort bounded jobs
-> flush logs/audit
-> exit(0)
```

`maxShutdownDelaySeconds` muss anschließend an das getestete Drain-Fenster angepasst werden.

RND-F-007 — P1 — Readiness Contract fehlt

`/healthz` ist ein guter Prozess-Liveness-Endpoint, aber aktuell kein vollständiger Readiness Gate.

Ziel ist eine explizite Dependency Classification:

Abhängigkeit	Startup-kritisch	Request-kritisch	Health-Verhalten
Express/runtime	Ja	Ja	hard fail
IAM Schema	für Admin/IAM ja	ja	degraded/not-ready nach Policy
Supabase	für auth/billing durable state ja	vielfach	cached bounded readiness
Stripe	nein beim Boot	Billing only	feature degraded
AI Provider	nein	AI features	feature degraded
Market Data	nein	Screening quality	feature degraded/fallback
SMTP	nein	mail flows	feature degraded

Nicht jede externe API darf Render-Healthchecks direkt blockieren. Entscheidend ist ein dokumentierter, gecachter Readiness-Zustand statt ungeplanter Netzwerkaufrufe pro Health Request.

RND-F-008 — P1 — Environment Contract ist unvollständig prüfbar

Das Readiness-Skript erkennt dynamisch gebildete Schlüssel nicht zuverlässig.

Besonders relevant:

```
STRIPE_PRICE_ID_STARTER_MONTHLY
STRIPE_PRICE_ID_STARTER_YEARLY
STRIPE_PRICE_ID_PRO_MONTHLY
STRIPE_PRICE_ID_PRO_YEARLY
STRIPE_ID_FOUNDER / STRIPE_PRICE_ID_FOUNDER
STRIPE_PRICE_ID_EXPORT_PDF
```

Der aktuelle Blueprint enthält nur die Basis-Price-IDs. Ob die erweiterten IDs im Dashboard existieren, muss verifiziert werden.

Ziel ist ein typisiertes, zentrales `production-env-contract`-Manifest, aus dem:

- `render.yaml` ;
- Predeploy Validation;
- Dokumentation;
- Production Handoff

konsistent abgeleitet werden.

RND-F-009 — P1 — Region/Compute nicht dokumentiert

Render verwendet bei neuen Services ohne `region` standardmäßig Oregon. Region kann nach Erstellung nicht geändert werden.

Da CAPITAL-AI primär EU-Kontext besitzt und Supabase in EU betrieben wird, muss die aktuelle Render-Region zwingend dokumentiert werden. Ein Neuaufbau aus dem aktuellen Blueprint könnte sonst einen anderen geografischen Zustand erzeugen.

Keine Region wird in diesem Audit erfunden. Zielregion ist erst nach Datenfluss-/Latenz-/Compliance-Entscheidung festzuschreiben.

RND-F-010 — P1 — Rollback-Wissen ist veraltet

Das bestehende Runbook muss an die aktuelle Render-Rollback-Semantik angepasst werden:

- Render kann vorhandene Build-Artefakte wiederverwenden;
- Dashboard-Rollback deaktiviert Autodeploys;
- Rollback stellt nicht automatisch alle aktuellen Service-Level-Konfigurationen zurück;
- Environment Groups besitzen besondere Rollback-Semantik;
- Disk-State wird nicht durch Code-Rollback zurückgesetzt.

RND-F-011 — P2 — Secret Diagnostics

Der Server loggt für Stripe-Schlüssel `Configured`, Länge und Präfixe.

Auch wenn keine vollständigen Secrets ausgegeben werden, sollte Production Logging keine Secret-Form-Metadaten benötigen.

Ziel:

```
stripe.secretKey.configured = true|false  
stripe.webhookSecret.configured = true|false
```

ohne Key-Fragmente.

RND-F-012 — P2 — Render Notifications

Mindestens Failure Notifications für folgende Render-Ereignisse sollten aktiv und dokumentiert sein:

- Build/Deploy failure;
- unhealthy service;
- Docker image pull failure;
- optional erfolgreiche Production Deploys.

Ziel: E-Mail und/oder Slack/Webhook mit Incident-Prozess verknüpfen.

RND-F-013 — P2 — Alternate Host Surface

Render erlaubt bei Custom Domains standardmäßig weiterhin die `onrender.com`-Subdomain, sofern sie nicht deaktiviert wird.

Zu prüfen:

- wird die Render-Subdomain operativ benötigt?
- wenn nein: `renderSubdomainPolicy: disabled` erwägen;
- Custom Domains (`capital-ai.online`, gewünschte `www`-Policy) als IaC dokumentieren.

RND-F-014 — P2 — Base Image Reproducibility

`node:22-alpine` ist ein beweglicher Tag.

Für reproduzierbare Supply Chain sollte das Image nach getestetem Upgrade-Prozess auf einen kontrollierten Digest bzw. eine ausreichend immutable Referenz gepinnt werden. Updates erfolgen dann bewusst über Dependency-/Container-Renovation statt unsichtbar zwischen Builds.

RND-F-015 — P2 — Health Endpoint Information Disclosure

`/healthz` zeigt öffentlich, welche Provider konfiguriert sind.

Diese Information ist nicht hochsensitiv, aber für Liveness nicht notwendig. Ein minimaler öffentlicher Health Contract reduziert Fingerprinting.

RND-F-016 — P2 — External Probe

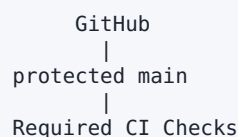
Render Health Checks prüfen die Instanz intern. Ein externer Probe sollte zusätzlich einen echten Benutzerpfad simulieren und Alerting auslösen.

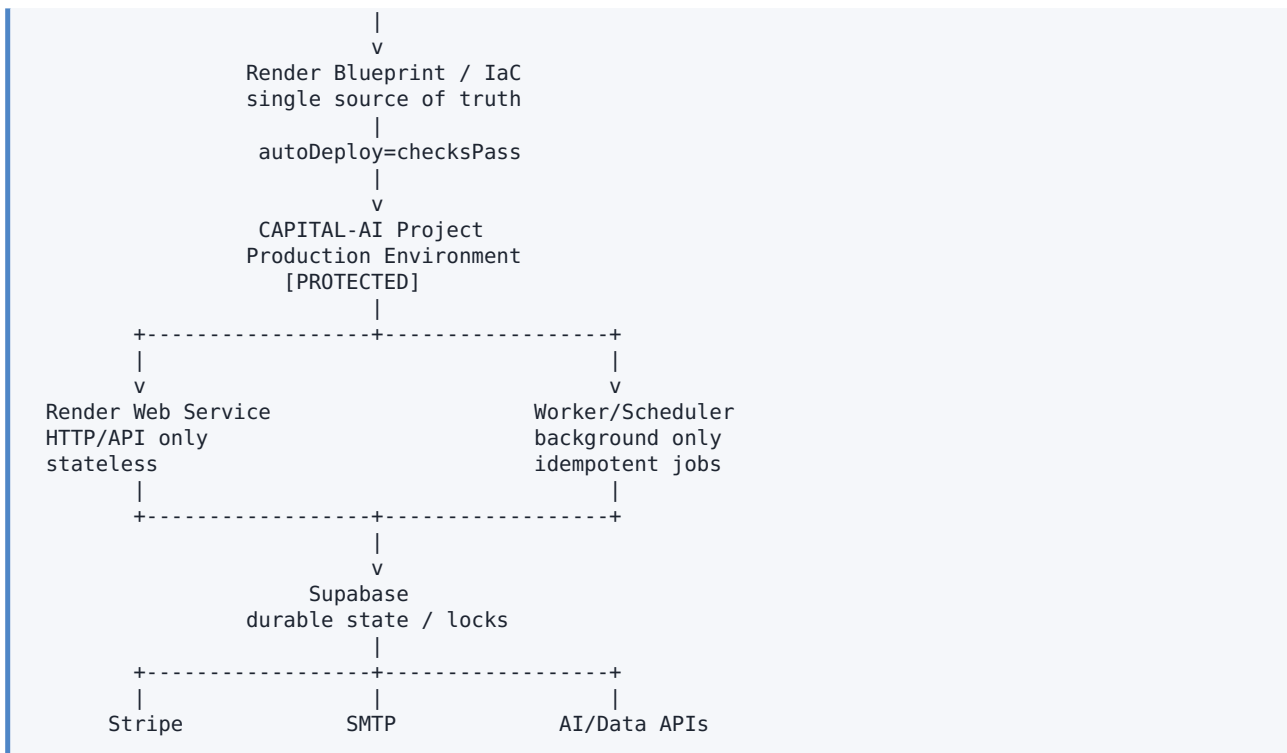
Dabei sollte die Render-/Cloudflare Request-ID (`CF-Ray`) bzw. die CAPITAL-AI Correlation-ID mitgeführt werden, damit externe Ausfälle zu Logs korreliert werden können.

7. Security- und Reliability-Bewertung

Control	Reifegrad	Bewertung
TLS / HTTPS	Hoch	Render verwaltet TLS und HTTP→HTTPS.
DDoS	Hoch	Render stellt Cloudflare-basierten DDoS-Schutz bereit.
Container User	Hoch	Non-root Runtime vorhanden.
Health Check	Mittel/Hoch	HTTP Health vorhanden, Readiness-Semantik zu schwach.
CI Gate	Hoch im Repo	Render-Verknüpfung <code>checksPass</code> nicht verifiziert.
IaC Coverage	Niedrig/Mittel	Blueprint beschreibt nur Teil der Produktion.
Secret Storage	Mittel/Hoch	<code>sync: false</code> korrekt, tatsächliche Werte/Groups nicht verifiziert.
Secret Inventory	Mittel	Dynamische Env Keys können Gate umgehen.
Graceful Shutdown	Niedrig	kein expliziter SIGTERM Coordinator.
Horizontal Scaling	Niedrig	Background Jobs verhindern sicheren Scale-out.
Persistence	Kritisch	lokale Business-Fallbacks + standardmäßig ephemeres FS.
Rollback Knowledge	Mittel	Runbook teilweise veraltet.
Production Environment Protection	Unbekannt	Dashboard-only Evidence fehlt.
Notifications	Unbekannt	Dashboard-only Evidence fehlt.
External Monitoring	Unbekannt	nicht dokumentiert.

8. Empfohlene Zielarchitektur





Render Web Service Contract

```

branch: main
auto deploy: after CI checks pass
PORT: Render-provided
health: explicit readiness policy
shutdown: SIGTERM drain
filesystem: disposable
business state: none
background schedulers: none
  
```

Environment Contract

```

Project: CAPITAL-AI
  Environment: Production [protected]
    Env Group: capital-ai-prod-public-config
    Env Group: capital-ai-prod-secrets
    Service: capital-ai-web
    Worker(s): dedicated background workloads
  
```

Secret Groups sollen environment-scoped sein, damit Dev/Staging nicht versehentlich Production Credentials verwenden können.

9. Blueprint-Zielbild

Die folgende Struktur ist **Zielbild**, nicht automatisch anzuwenden, solange die Dashboard-Werte nicht verifiziert sind:

```

services:
- type: web
  name: capital-ai
  runtime: docker
  repo: <verified repository URL>
  branch: main
  
```

```

region: <VERIFIED CURRENT / APPROVED TARGET>
plan: <VERIFIED CURRENT>
autoDeployTrigger: checksPass
healthCheckPath: /readyz
maxShutdownDelaySeconds: <tested drain window>
numInstances: 1 # until background work is separated
domains:
  - capital-ai.online
renderSubdomainPolicy: <verified decision>
dockerfilePath: ./Dockerfile
envVars:
  - fromGroup: capital-ai-prod-secrets
  - fromGroup: capital-ai-prod-config

```

Keine dieser Dashboard-abhängigen Angaben darf ohne Verifikation blind in `render.yaml` übernommen werden.

10. Production Configuration Source-of-Truth Policy

Jede Render-Einstellung muss künftig genau einer Kategorie angehören:

A. IaC-managed

Muss in `render.yaml` stehen und darf nicht dauerhaft nur im Dashboard existieren.

Beispiele:

- Service type/name/runtime;
- repo/branch;
- region/plan;
- autoDeployTrigger;
- healthCheckPath;
- shutdown delay;
- scaling policy;
- domain policy;
- build filters;
- project/environment topology.

B. Secret-managed

Wert bleibt bewusst ausschließlich in Render.

Repository enthält nur:

- kanonischen Variablennamen;
- Klassifikation `secret|public-config` ;
- Owner;
- Rotation Policy;
- abhängige Module;
- required/optional;

- Build-time/runtime scope.

C. Operational-managed

Dashboard-/Workspace-Einstellungen, die nicht vollständig per Blueprint modelliert werden, erhalten verpflichtende Evidence im Architektur-Dokument.

Beispiele:

- Notification destination;
- Workspace roles;
- Render GitHub App scope;
- Incident contacts.

11. Keine automatische Produktionsänderung aus diesem Audit

Dieses Audit verändert absichtlich **keine** Render-Dashboard-Einstellung.

Folgende Änderungen sind Production Control Plane Changes und benötigen den etablierten CAPITAL-AI-Handoff:

- Region/Plan/Scaling;
- Environment Protection;
- Auto-Deploy;
- Build Filters;
- Domain/Subdomain Policy;
- Persistent Disk;
- Environment Groups/Secrets;
- Notifications/Webhooks;
- Maintenance Mode;
- Render API/Deploy Hooks.

ADR-0037 definiert hierfür die Freigabe- und Evidence-Anforderungen.

12. Verpflichtende Render-Dashboard-Evidence-Checkliste

Diese Tabelle ist bei direktem Render-Zugriff vollständig auszufüllen.

Bereich	Soll-Evidence	Aktuell
Workspace	Workspace name + plan	UNVERIFIED
Service	Service ID + service name	Name repo-verifiziert, ID UNVERIFIED
Source	GitHub repo + branch	UNVERIFIED DASHBOARD

Bereich	Soll-Evidence	Aktuell
Runtime	Docker	VERIFIED_REPO
Region	exakter Render Region Code	UNVERIFIED
Instance	Plan / CPU / RAM	UNVERIFIED
Scaling	manual count / autoscaling targets	UNVERIFIED
Project	Project name	UNVERIFIED
Environment	Production environment assignment	UNVERIFIED
Protection	Protected environment enabled	UNVERIFIED
Network	Cross-environment isolation	UNVERIFIED
Auto Deploy	checksPass	UNVERIFIED
Overlapping Deploy Policy	selected policy	UNVERIFIED
Health	/healthz or approved /readyz	repo= /healthz
Shutdown	max shutdown delay	UNVERIFIED
Docker	Dockerfile path / Docker command override	UNVERIFIED
Root Dir	service root directory	UNVERIFIED
Build Filters	included/ignored paths	UNVERIFIED
Pre-deploy	command, if any	UNVERIFIED
Domains	custom domains + redirect canonicalization	UNVERIFIED
Render subdomain	enabled/disabled	UNVERIFIED
TLS	certificate status	provider-managed, dashboard state UNVERIFIED
Disk	none / mount path / size	render.yaml none; Dashboard UNVERIFIED
Env Groups	names + project scope	UNVERIFIED
Env Vars	key inventory only, never values in docs	PARTIAL
Secret Files	filename inventory only	UNVERIFIED
Notifications	failure/all + destination	UNVERIFIED
Webhooks	event hooks and ownership	UNVERIFIED
Preview	off/manual/automatic + expiry	UNVERIFIED
Maintenance	state + configured URI	UNVERIFIED
GitHub Integration	Render GitHub App repository scope	UNVERIFIED
Deploy	current production commit SHA	

Bereich	Soll-Evidence	Aktuell
		UNVERIFIED in current main runtime

Evidence-Regel

Keine Secret-Werte, API-Tokens, Deploy-Hook-URLs oder Credentials in dieses Dokument kopieren.

Erlaubt sind ausschließlich Namen, IDs/Fingerprints und boolesche Konfigurationszustände, soweit sie selbst kein Secret darstellen.

13. Umsetzungsreihenfolge

Phase 0 — sofort

1. Render Dashboard Evidence erfassen.
2. Prüfen, ob `/app/uploads` persistent ist.
3. Wenn kein Disk vorhanden: lokale Business-State-Fallbacks als Datenverlust-Risiko behandeln.
4. Current Auto-Deploy Policy prüfen.
5. Current Region/Plan/Instance Count dokumentieren.

Phase 1 — Deployment Contract

1. `PORT`-Binding korrigieren.
2. Graceful Shutdown Coordinator implementieren.
3. Liveness/Readiness Contract definieren.
4. Render `autoDeployTrigger: checksPass` normativ festlegen.
5. Service-/Environment-Topology als Blueprint vervollständigen.
6. Rollback Runbook korrigieren.

Phase 2 — Stateless Web Tier

1. Background Jobs aus Web-Prozess extrahieren.
2. Idempotency / distributed locks definieren.
3. lokales Subscription-/Credit-Persistieren entfernen.
4. danach Multi-Instance/Autoscaling neu bewerten.

Phase 3 — Operational Hardening

1. Protected Production Environment;
2. environment-scoped Secret Groups;
3. Notification-/Incident Integration;
4. external uptime/smoke probe;

5. Build image digest policy;
 6. controlled Preview Environment Strategy.
-

14. Definition of Done für ADR-0037

ADR-0037 darf erst als vollständig umgesetzt gelten, wenn:

- alle Dashboard-Evidence-Felder aus Abschnitt 12 belegt sind;
 - Production Port aus `PORT` gelesen wird;
 - Render Auto-Deploy nach CI Checks verifiziert ist;
 - Production Environment Protection verifiziert ist;
 - Region/Plan/Scaling als Source of Truth dokumentiert sind;
 - keine authoritative Business-Daten auf ephemerem FS liegen;
 - Background Work nicht unkontrolliert pro Web-Replica dupliziert wird;
 - SIGTERM/Graceful Shutdown getestet ist;
 - Readiness Contract getestet ist;
 - Rollback-Runbook der aktuellen Render-Semantik entspricht;
 - Environment-Key-Inventar vollständig automatisiert geprüft wird;
 - Notifications und External Probe belegt sind;
 - ein Disaster-Recreation-Test aus dokumentierter IaC + Secret Inventory durchgeführt wurde.
-

15. Offizielle Render-Referenzen — Stand 2026-08-03

- Web Services / Port Binding: <https://render.com/docs/web-services>
- Deploys / Auto Deploy / Graceful Shutdown: <https://render.com/docs/deploys>
- Health Checks: <https://render.com/docs/health-checks>
- Blueprints / IaC: <https://render.com/docs/infrastructure-as-code>
- Blueprint Spec: <https://render.com/docs/blueprint-spec>
- Environment Variables and Secrets: <https://render.com/docs/configure-environment-variables>
- Projects / Protected Environments: <https://render.com/docs/projects>
- Persistent Disks: <https://render.com/docs/disks>
- Scaling: <https://render.com/docs/scaling>
- Rollbacks: <https://render.com/docs/rollbacks>
- Notifications: <https://render.com/docs/notifications>
- Custom Domains: <https://render.com/docs/custom-domains>
- Uptime Best Practices: <https://render.com/docs/uptime-best-practices>

Provider-Anforderungen müssen bei zukünftigen Render-Architekturänderungen erneut gegen die aktuelle Dokumentation geprüft werden.

CAPITAL-AI — Render Runtime Remediation Evidence

Quelle: docs/architecture/RENDER_RUNTIME_REMEDIATION_2026-08-03.md

CAPITAL-AI — Render Runtime Remediation Evidence

Date: 2026-08-03

Environment: Production

Render workspace: AICapital

Render service: Finance (srv-d91o1o9o3t8c73edi55g)

Repository / branch: SvenKulesa/Finance / main

Governing ADR: ADR-0037

Remediation PR: #78

1. Purpose

This document records the verified production findings from the live Render audit and separates:

1. changes already applied directly to Render;
2. code/IaC changes implemented in PR #78;
3. remaining Render control-plane settings that cannot be mutated by the currently connected Render write API and therefore remain explicitly open until applied and re-verified.

No secret values are documented here.

2. Production baseline

Before remediation, the active production deployment was:

- commit: 770f14632086722fa2fe7a7286ce82bfc90c83c ;
- deploy: dep-d9o23hoae00c73fli0ag ;
- service plan: Starter;
- region: Frankfurt;
- runtime: Docker;
- instances: 1;
- runtime port observed by Render: 3000;
- autoDeployTrigger: commit ;
- Render healthCheckPath: empty.

The service was operational and had ample capacity, but the application background market-data refresh failed once per minute with SCORE_NOT_COMPUTABLE for standard crypto assets whose verified historical feature set was unavailable.

3. Production changes already applied

3.1 Canonical Stripe Price-ID contract

The live Stripe account was read and the active CAPITAL-AI prices were matched to the variable names already used by `server/stripe.ts`.

The following canonical Render environment-variable **keys** were populated with their verified live Stripe Price IDs. Values are intentionally omitted from this document:

- `STRIPE_PRICE_ID_STARTER_MONTHLY` ;
- `STRIPE_PRICE_ID_STARTER_YEARLY` ;
- `STRIPE_PRICE_ID_PRO_MONTHLY` ;
- `STRIPE_PRICE_ID_PRO_YEARLY` ;
- `STRIPE_ID_FOUNDER` ;
- `STRIPE_PRICE_ID_EXPORT_PDF` .

Existing environment variables were merged, not replaced. No Stripe product, price, customer, subscription, webhook, Supabase row or IAM state was mutated.

Render automatically triggered deploy `dep-d9o2j7p42hec738ihme0` to pick up the environment-variable update. The deploy completed with status `live` against the unchanged production code commit `770f14632086722fa2fe7a7286ce82bfc90c83c` .

4. PR #78 code remediation

4.1 Market-data refresh fail-loop

The deterministic crypto scoring engine remains fail-closed: it still refuses to compute a score when there are no verified scoring features.

The remediation is applied at the aggregation boundary instead. When `generateCryptoScores()` returns no finite verified feature for a standard crypto asset, the market-data batch may retain an already finite upstream/base score only as `scoreBasis: heuristic` .

Properties of this fallback:

- no synthetic score is generated;
- a missing/non-finite base score still fails closed;
- the result is never labelled `market-data` ;
- one unsupported CMC symbol no longer rejects the complete `Promise.all` market-data refresh batch;
- the No-Demo-Data provenance distinction is preserved.

4.2 Render PORT contract

`server.ts` now resolves the listening port from `PORT` and uses 3000 only when no platform port is supplied. Invalid/non-integer/out-of-range values fail at startup rather than silently binding an unintended port.

4.3 Stripe diagnostics disclosure

Startup diagnostics no longer output:

- secret/key prefixes;
- key lengths;
- partial Price IDs.

The diagnostic now emits boolean configuration-presence fields only, including canonical monthly/yearly price-key coverage.

4.4 Graceful Render shutdown

The HTTP server handle and market-data refresh timer are now retained. On `SIGTERM` / `SIGINT` the process:

1. prevents duplicate shutdown;
2. stops scheduling the periodic market-data refresh;
3. stops accepting new HTTP connections and drains the HTTP server;
4. exits successfully after drain;
5. force-exits after 25 seconds if shutdown cannot complete.

The 25-second application deadline remains below the 30-second Render shutdown window declared in IaC.

4.5 Testable runtime helpers

Pure helpers under `server/runtime/renderRuntimeSafety.ts` cover:

- PORT validation;
- finite-score detection;
- bounded heuristic reuse of an existing base score;
- boolean-only Stripe configuration diagnostics.

Unit tests are included under `tests/unit/renderRuntimeRemediation.test.ts`.

5. Render IaC reconciliation

`render.yaml` is updated to describe the verified Finance production topology instead of the historical `capital-ai` service name:

- service: `Finance`;
- repo: `SvenKulesa/Finance`;

- branch: `main`;
- runtime: Docker;
- region: Frankfurt;
- plan: Starter;
- instances: 1;
- health path target: `/healthz`;
- auto-deploy target: `checksPass`;
- shutdown-delay contract: 30 seconds;
- canonical Stripe billing environment-key inventory.

This is the repository target state. It is not evidence that every control-plane value has already been changed on the existing Render service.

6. Remaining live Render control-plane drift

The connected Render write surface currently supports environment-variable updates and deploy operations, but does not expose a generic service-settings mutation for the existing Docker service. Therefore these verified live values remain open until changed through an authorized Render control-plane operation and re-read afterwards:

Setting	Verified live value	Target	Status
<code>autoDeployTrigger</code>	<code>commit</code>	<code>checksPass</code>	OPEN
<code>healthCheckPath</code>	empty	<code>/healthz</code>	OPEN

They MUST NOT be described as fixed merely because `render.yaml` now expresses the target state.

7. Post-merge production verification

After PR #78 is approved, merged and deployed, production evidence must verify all of the following:

1. deploy commit equals the merged PR #78 commit;
2. startup succeeds on the Render-provided `PORT`;
3. `/healthz` returns 2xx;
4. no key prefixes, key lengths or partial Price IDs appear in startup logs;
5. canonical Stripe configuration-presence flags are true for the intended plans;
6. `Market Data` startup pre-cache completes;
7. at least two consecutive 60-second background refresh cycles complete without `SCORE_NOT_COMPUTABLE`;
8. a controlled redeploy records `Graceful shutdown initiated` and does not produce an unexplained application crash;

9. Render service settings are re-read and the remaining control-plane drift is closed or retained as an explicitly tracked finding.

8. Rollback boundary

The runtime code changes are reverted through the protected GitHub PR/revert process. The Render billing-variable change can be rolled back only by restoring the prior Render environment configuration from trusted operator evidence; secret values must never be copied into repository documentation or PR bodies.

Rollback of governance, health, billing, IAM or deployment controls remains subject to the protected-change rules defined by the CAPITAL-AI architecture.

CAPITAL-AI — Render Production Audit Part 2: Stateless Web Tier & Worker Architecture

Quelle: docs/architecture/RENDER_STATELESS_WEB_TIER_AUDIT_2026-08-03.md

CAPITAL-AI — Render Production Audit Part 2: Stateless Web Tier & Worker Architecture

Audit ID: RENDER-AUDIT-0002
Audit date: 2026-08-03
Status: COMPLETE FOR VERIFIED LIVE / REPOSITORY / DATABASE EVIDENCE
Priority: CRITICAL / HIGH
System: CAPITAL-AI / capital-ai.online
Repository: SvenKulessa/Finance
Production service: Render Finance (srv-d91o1o9o3t8c73edi55g)
Related ADR: ADR-0037 — Render Production Configuration Governance, Stateless Runtime & Reproducible Deployment Contract
Previous audit: docs/architecture/RENDER_PRODUCTION_CONFIGURATION_AUDIT.md

1. Ziel und Scope

Teil 2 prüft die Phase-2-Anforderungen aus ADR-0037:

- 1. Ist der Render-Webservice wirklich stateless?
- 2. Welche Business-, Audit-, Agent-, Documentary- und Runtime-Zustände liegen noch lokal im Container oder Prozessspeicher?
- 3. Welche Background-Jobs laufen im HTTP-Prozess?
- 4. Welche Jobs sind idempotent und welche erzeugen bei paralleler Ausführung doppelte Side Effects?
- 5. Welche Parallelität entsteht bereits durch Render Zero-Downtime Deployments bei numInstances = 1 ?
- 6. Welche Komponenten müssen vor horizontalem Scaling in Worker/Scheduler, Durable State oder Distributed Coordination überführt werden?

Dieser Audit verändert keine Supabase-Daten, keine Render-Skalierung und keine produktiven Business-Daten.

2. Evidence-Klassen

Status	Bedeutung
VERIFIED_RENDER	Direkt im aktiven Render-Service/Deploy/Log verifiziert.

Status	Bedeutung
VERIFIED_REPO	Direkt aus dem aktuellen Repository gelesen.
VERIFIED_SUPABASE	Direkt gegen das aktive Produktionsprojekt gelesen.
VERIFIED_RUNTIME	Live-Verhalten in Produktionslogs belegt.
RISK	Aus verifizierten Komponenten abgeleitetes Risiko.
TARGET	Normativer Zielzustand, noch nicht automatisch implementiert.

Keine Secret-Werte, Tokens oder Credentials werden in diesem Dokument gespeichert.

3. Aktuelle Produktionsbaseline

Render

```
Workspace:      AICapital
Service:        Finance
Service ID:     srv-d91o1o9o3t8c73edi55g
Repository:     SvenKulessa/Finance
Branch:         main
Runtime:        Docker
Region:         Frankfurt
Plan:           Starter
Configured instances: 1
Auto deploy:    yes
Auto deploy trigger: commit
Health check path: nicht im Live-Service gesetzt
```

Aktueller Produktionsstand zum Rebase dieses Audits:

```
Production commit: 24d9daecaf4100ef7dccc3c3c375ac6b0ae8bd43
Render deploy:    dep-d9o3sovk4qsc73b4vl80
Status:           live
Trigger:          new_commit
```

Bemerkung: Auch reine Dokumentations-Commits auf `main` lösen derzeit vollständige Render-Deployments aus. Das bestätigt erneut, dass `autoDeployTrigger=commit` noch nicht auf den ADR-0037-Zielzustand `checksPass` umgestellt wurde.

Supabase

```
Project:      AIFINANCIAL
Project ref:  rzywoktpmyhwzxmstyu
Region:       eu-west-1
Postgres:     17
Status:       ACTIVE_HEALTHY
```

Für diesen Audit relevante durable Tabellen:

- `subscriptions`
- `user_quota`
- `score_snapshots`

- `alert_subscriptions`
- `subscription_confirmations_sent`
- `security_events`
- `audit_logs_iam`
- `iam_access_log`
- `compliance_runs`
- `screening_slo_evidence`

Im aktuellen Produktionsschema fehlen dagegen dedizierte Tabellen für:

- PDF Export Credits / Credit Ledger;
- generische Worker-Leases / Leader Election;
- Alert Delivery Outbox / Delivery Idempotency;
- allgemeine System Events aus `server/systemEvents.ts` ;
- Agent Runtime Registry;
- Documentary Hygiene Runtime State;
- Version Manager Runtime State.

4. Executive Result

Der aktuelle Render-Webservice ist **nicht stateless**.

Die Zustandsbindung besteht gleichzeitig auf drei Ebenen:

- ```
A. Local filesystem state
 subscriptions.json
 pdf_credits.json
 system_events.json
 agents_registry.json
 document_hygiene.json
 version_manager.json
 docs/.history/*
 runtime mutations under docs/*

B. In-process state
 market-data cache
 active market-data promise
 provider cooldowns
 security rate-limit buckets
 Event Mesh registry/delivery log
 SSE client registry
 Documentary watcher/timers

C. Background side effects in HTTP process
 startup market-data prefetch
 60-second market-data refresh
 score snapshot recording
 alert evaluation/email delivery
 Documentary filesystem watcher
```

**Horizontal Scaling bleibt nicht freigabefähig.**

Noch wichtiger: Das Duplicate-Execution-Risiko tritt bereits heute bei normalen Render-Deployments auf.

---

## 5. Live-Beweis: Parallel laufende Background-Jobs während Zero-Downtime Deployment

Beim Deploy von PR #79 wurde der Übergang zwischen alter und neuer Instanz direkt beobachtet.

### Alte Instanz

```
srv-d91010903t8c73edi55g-rlrtb
```

führte um `06:58:19Z` weiterhin den CoinMarketCap-/Market-Data-Refresh aus.

### Neue Instanz

```
srv-d91010903t8c73edi55g-2ql5c
```

war bereits hochgefahren und führte um `06:58:32Z` ihren eigenen Market-Data-Refresh aus.

### Shutdown der alten Instanz

Die alte Instanz erhielt erst um ungefähr `06:58:34Z` SIGTERM.

Damit gilt empirisch:

```
configured numInstances = 1
 !=
exactly one process executes scheduled work at every instant
```

Render Zero-Downtime Deployment bildet selbst bei einer konfigurierten Webinstanz ein temporäres Multi-Process-Concurrency-Fenster.

**Konsequenz:** Nicht-idempotente Background-Arbeit muss duplicate-safe sein, selbst wenn Autoscaling dauerhaft deaktiviert bleibt.

---

## 6. Durable-/Ephemeral-State-Audit

### 6.1 Subscription / Entitlement

`server/db.ts` verwendet zusätzlich zu Supabase:

```
uploads/subscriptions.json
```

`saveSubscription()` schreibt lokal, bevor der Remote-Upsert erfolgt. `getSubscription()` kann bei fehlender Supabase-Verfügbarkeit aus der lokalen Datei lesen.

Supabase besitzt bereits eine produktive `subscriptions`-Tabelle mit:

```
id uuid PK
user_id uuid UNIQUE -> auth.users.id
stripe_subscription_id UNIQUE
tier
status
current_period_end
expires_at
updated_at
```

## RND2-F-001 — P0

**Local Subscription Fallback kann zu veralteter Entitlement-Wahrheit werden.**

Risiken:

- pro Container unterschiedliche Datei;
- Verlust bei Restart/Deploy;
- alter Pro-/Enterprise-Tier kann bei DB-Ausfall weiter ausgeliefert werden;
- Downgrade/Cancellation kann lokal veraltet bleiben;
- nicht-transaktionale zweite Zustandsquelle;
- keine Cache-Kohärenz zwischen parallelen Instanzen.

**Ziel:** Supabase ist einzige Subscription-/Entitlement-Source-of-Truth. Privilegierte Entitlement-Entscheidungen müssen bei fehlender durable Source fail closed oder einen ausdrücklich definierten, integrity-protected und zeitlich begrenzten Cache-Vertrag verwenden. Eine freie JSON-Datei ist nicht ausreichend.

---

## 6.2 PDF Export Credits

Aktuell:

```
uploads/pdf_credits.json
```

wird für Credit-Abfrage, Verbrauch und Stripe-Kauf verwendet.

Verbrauch:

```
read current
-> current - 1
-> write JSON
```

Stripe-PDF-Kauf:

```
checkout/webhook
-> read current
-> current + 3
-> write JSON
```

Im produktiven Supabase-Schema existiert **keine PDF-Credit-Tabelle**.

## RND2-F-002 — P0

**Bezahlte PDF-Credits sind nur auf ephemerem lokalem Dateisystem authoritative.**

Risiken:

- gekaufte Credits können nach Deploy/Restart verschwinden;
- paralleles Read-Modify-Write kann Updates verlieren;
- alte/neue Render-Instanz können unterschiedliche Salden führen;
- Stripe kann Zahlung erfolgreich verbuchen, während das Entitlement später verschwindet;
- Webhook-Retry kann Credits mehrfach addieren;
- keine durable Idempotency per Stripe Event/Session.

Empfohlenes Zielmodell:

```
pdf_credit_accounts
 user_id PK/FK
 balance
 updated_at

pdf_credit_ledger
 id
 user_id
 delta
 source
 stripe_event_id UNIQUE
 created_at
 metadata
```

Credit-Verbrauch muss atomar erfolgen, z. B. durch eine transaktionale Postgres-Funktion oder:

```
UPDATE pdf_credit_accounts
SET balance = balance - 1
WHERE user_id = $1
AND balance > 0
RETURNING balance;
```

## 6.3 System-/Audit-Events

`server/systemEvents.ts` verwendet:

```
uploads/system_events.json
```

Wenn die Datei fehlt, erzeugt der Code vorbefüllte Initial-/Beispielereignisse. Der Enterprise Event Mesh Publish ist nur additiv/best-effort.

## RND2-F-003 — P0

**Security-/Audit-Trail ist ephemeral und enthält synthetisch vorbefüllte Ereignisse.**

### Risiken:

- Audit-Historie verschwindet bei Deploy/Restart;
- Instanzen besitzen unterschiedliche Audit-Trails;
- SSE-Clients sehen nur instanzlokale Ereignisse;
- synthetische Ereignisse sind keine Betriebsbeweise;
- Konflikt mit No Demo Data Policy;
- nicht geeignet als revisionssichere Compliance Evidence.

**Ziel:** append-only/durable Audit Store. Bestehende Supabase Security-/IAM-Tabellen können ergänzt oder durch ein dediziertes `system_audit_events` -Schema erweitert werden. Synthetische Seed-Events müssen aus Production entfernt werden.

---

## 6.4 Agent Registry

`server/systemEvents.ts` führt zusätzlich:

```
uploads/agents_registry.json
```

mit Agentstatus, active task, query count, Modelllabel und Custom-Agent-Registrierung.

### RND2-F-004 — P1

#### Agent Operational Registry ist instanzlokal.

Ein Agent kann in mehreren Prozessen gleichzeitig unterschiedliche Zustände zeigen. Query Counts sind nicht global und verschwinden beim Containerwechsel.

---

## 6.5 Documentary Hygiene

`server/documentHygiene.ts` verwendet:

```
uploads/document_hygiene.json
docs/.history/*
```

Der produktive Webprozess startet einen persistenten Chokidar-Watcher für `/app/docs` und kann unter anderem:

- Markdown-/JSON-/Text-Inhalte verändern;
- Branding anwenden;
- abhängige Dokumente propagieren;
- Review-Tickets anwenden;
- Review-Reverts durchführen;
- Backups unter `docs/.history` erzeugen.

## RND2-F-005 — P0

### Autonomous Documentary kann den deployed Git-Stand außerhalb von Branch/PR-Governance verändern.

Das widerspricht dem neuen Single-Writer-/PR-First-Modell:

```
GitHub protected main
-> immutable build artifact
-> Render container
X runtime mutation must not become a second repository truth
```

Risiken:

- Running Container kann von `main` abweichen;
- keine PR-/CODEOWNER-Prüfung;
- Änderung wird nicht zurück nach Git synchronisiert;
- Änderung verschwindet beim Deploy;
- parallele Container können verschiedene Dokumentstände haben;
- Review History ist nicht revisionssicher.

**Ziel:** Production Documentary erzeugt Vorschläge als Git-Branch/PR oder in einem durable Review Store. Der HTTP-Webprozess darf keinen dauerhaften Git-controlled `docs/`-Watcher besitzen.

---

## 6.6 Version Manager

`src/platform/VersionManager/versionManager.ts` verwendet:

```
uploads/version_manager.json
```

und kann ebenfalls Dokumente unter `docs/` schreiben. Der Default-State referenziert weiterhin `0.5.4`, während die aktuelle Application Runtime `0.6.0` ist.

## RND2-F-006 — P1

### Version State ist lokal, mutable und nicht deployment-authoritative.

Die Produktionsversion muss aus immutable Deployment Identity / Git / Release Metadata abgeleitet werden.

---

## 7. Background-Job-Audit

### 7.1 Market Data Pre-cache

Jeder Webprozess startet beim Boot einen eigenen `fetchLiveMarketData()`-Lauf.

**Risiko:** Jeder Container erzeugt eigene Provider-Requests.

## 7.2 60-Sekunden Market Refresh

Der Webserver registriert einen periodischen Refresh. Prozesslokal sind unter anderem:

- Market Cache;
- aktive Promise für Request-Coalescing;
- CoinMarketCap Cooldown;
- CoinGecko Cooldown.

Request-Coalescing schützt nur innerhalb eines Node-Prozesses.

### RND2-F-007 — P0/P1

**Market Refresh besitzt keine verteilte Lease. Duplicate Execution wurde live beobachtet.**

Das ist nicht nur ein zukünftiges Autoscaling-Risiko, sondern ein aktuelles Deploy-Risiko.

---

## 7.3 Score Snapshots

`recordDailySnapshots()` persistiert in Supabase.

Die Migration definiert:

```
UNIQUE(symbol, snapshot_date)
```

und der Code nutzt Upsert mit Duplicate-Schutz.

### RND2-F-008 — CONTROL PRESENT

**Score-Snapshot-Persistenz ist auf Datenbankzeilenebene idempotent.**

Das verhindert doppelte Tageszeilen, nicht jedoch doppelte Provider- und Scoring-Berechnungen vor dem Upsert.

---

## 7.4 Alert Evaluation / E-Mail

Aktueller Ablauf:

```
SELECT confirmed active alert
-> threshold/cooldown check
-> send email
-> UPDATE last_notified_at
```

Es gibt keinen atomaren Delivery Claim vor dem Mailversand.

### RND2-F-009 — P1

**Alert E-Mail-Side-Effect ist nicht multi-instance-idempotent.**

Zwei Prozesse können dasselbe Abo lesen und beide senden, bevor einer `last_notified_at` aktualisiert.

Empfohlenes Ziel:

```
alert evaluator
-> atomic outbox/delivery claim
-> UNIQUE(alert_id, evaluation_window/event_id)
-> mail worker
-> SENT / FAILED / retry state
```

## 8. Distributed Coordination

### 8.1 Enterprise Event Mesh

Der aktuelle Event Bus ist explizit:

```
in-memory
one bus per process
no shared state across Render workers
```

#### RND2-F-010 — P1 / BY DESIGN

**Event Mesh ist derzeit ein In-Process Dispatcher, kein Distributed Message Bus.**

Bei Worker-Separation darf er nicht stillschweigend als Cross-Service Queue interpretiert werden.

Zukünftige Optionen erfordern eine explizite Architekturentscheidung, beispielsweise:

- Postgres transactional outbox;
- Supabase Queue/pgmq, falls freigegeben;
- Redis-basierte Queue;
- NATS/Kafka/Service Bus in einer späteren Zielplattform.

### 8.2 Security Rate Limiter

Der Security Rate Limiter verwendet eine Node `Map` und dokumentiert selbst seine Single-Instance-Grenze.

#### RND2-F-011 — P1 SECURITY

**Mit mehreren Replikas vervielfacht sich das effektive Rate-Limit-Budget.**

Vor Scale-out müssen mindestens IAM-/Owner-/Billing-/Mail-kritische Limits zentralisiert oder am Gateway/Edge erzwungen werden.

### 8.3 Generic Job Coordination

Im aktuellen produktiven Supabase-Schema wurde keine generische Worker-Lease-, Leader-Election-, Job-Claim- oder Alert-Outbox-Tabelle festgestellt.

#### RND2-F-012 — P1

#### Es fehlt ein generischer durable Coordination Contract für Background Work.

Minimaler Zielvertrag könnte enthalten:

```
worker_leases
 lease_key PK
 holder_id
 acquired_at
 expires_at
 heartbeat_at
 fencing_token
```

Anforderungen:

- atomare Acquisition;
- Lease Expiry;
- eindeutiger Lease Key;
- Holder enthält Deploy-/Service-/Instance-Identität;
- Fencing Token für stale-worker protection, wo Side Effects dies verlangen;
- Correlation-/Job-ID für Retries und Audit.

Für irreversible oder hochwertige externe Side Effects ist ein Outbox-/Idempotency-Ledger robuster als ausschließlich ein Zeit-Lease.

## 9. Zero-Downtime Deployment als eigener Concurrency Domain

Normative Korrektur aus dem Audit:

`numInstances = 1` ist keine ausreichende Singleton-Garantie für Background Work.

Lifecycle:

```
old instance
| background timer active
|
+-----+
| |
| new instance boot |
| | |
| + startup job |
| + timer registration |
| | |
| traffic switch |
| | |
old instance still active ---+
| possible additional timer cycle
|
v
SIGTERM
```

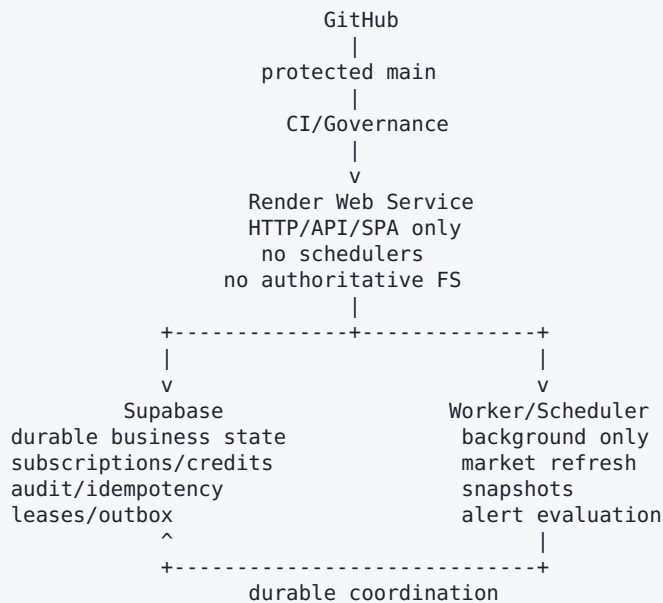
Jeder nicht-idempotente Job benötigt daher mindestens einen der folgenden Mechanismen:

1. externen Lease-/Leader-Vertrag;
2. dedizierten Worker mit eindeutiger Ownership;
3. queue-/outbox-basierten idempotenten Consumer;
4. Kombination aus Lease und Idempotency Ledger.

## 10. Statelessness Matrix

| Komponente          | Current State        | Durable?  | Multi-instance safe? | Ziel                                      |
|---------------------|----------------------|-----------|----------------------|-------------------------------------------|
| HTTP/API/SPA        | Container            | N/A       | grundsätzlich ja     | Web Service                               |
| Market cache        | Memory               | No        | No                   | cache only                                |
| Market scheduler    | Web timer            | No        | <b>No</b>            | Worker + lease                            |
| Provider cooldowns  | Memory               | No        | No                   | worker-owned/shared contract              |
| Score snapshots     | Supabase             | Yes       | <b>Yes row-level</b> | Worker -> Supabase                        |
| Alert subscriptions | Supabase             | Yes       | Storage yes          | Worker/outbox                             |
| Alert send          | Send then update     | Partly    | <b>No</b>            | atomic claim/outbox                       |
| Subscriptions       | Supabase + JSON      | Mixed     | No                   | Supabase only                             |
| PDF credits         | JSON                 | <b>No</b> | <b>No</b>            | durable ledger                            |
| User quota          | Supabase             | Yes       | DB-backed            | retain/verify atomi-city                  |
| Security events     | Supabase where used  | Yes       | DB-backed            | durable audit                             |
| System events       | JSON                 | No        | No                   | durable append-only                       |
| Event Mesh          | Memory               | No        | No                   | in-process only or external transport     |
| Rate limiter        | Memory               | No        | No                   | distributed/edge critical limits          |
| Agent registry      | JSON                 | No        | No                   | durable or explicitly ephemeral telemetry |
| Documentary state   | JSON + docs mutation | No        | No                   | Git PR / durable review                   |
| Version manager     | JSON                 | No        | No                   | deployment/release metadata               |

## 11. Zielarchitektur



### Web Service darf im Zielzustand

- Requests authentifizieren/autorisieren;
- Input validieren;
- synchrone APIs orchestrieren;
- begrenzte nicht-authoritative Memory-Caches führen;
- Business State ausschließlich über durable Stores ändern;
- asynchrone Arbeit als durable Job-/Outbox-Intent publizieren.

### Web Service darf im Zielzustand nicht

- authoritative Business JSON lokal persistieren;
- Git-kontrollierte Dokumente autonom ändern;
- periodische Market Scheduler besitzen;
- nicht-idempotente Alert-/Mail-Scheduler ausführen;
- Release-Versionen in lokalem JSON authoritative verwalten;
- globale Security-Entscheidungen nur aus lokalem Memory ableiten.

### Worker/Scheduler übernimmt

- Market Refresh;
- Alert Evaluation;
- Mail Delivery bei Outbox-Modell;
- Score Snapshot Recording;
- Retry/Backoff;
- Provider Cooldowns;
- Job Lease/Heartbeat;

- Execution Telemetry.

---

## 12. Neue Governance-Beobachtung während des Audits

Während PR #80 offen war, wurde `main` durch parallele Arbeit weiterbewegt. Die reparierte Production-Preflight-Governance erkannte dies korrekt und blockierte den veralteten Audit-Branch, bis dieser auf aktuellen `main` gebracht wurde.

Das ist ein positiver End-to-End-Beweis für PR #79.

Gleichzeitig wurde direkt auf `main` ein neues Dokument angelegt:

```
docs/adr/ADR-0036-server-composition-and-modularization.md
```

Der bestehende Multi-Agent-Governance-Vertrag verwendet jedoch bereits:

```
docs/adr/ADR-0036-pr-first-multi-agent-single-writer-governance.md
```

und `AGENTS.md` verweist normativ auf diese bestehende ADR-0036.

### RND2-F-013 — P1 GOVERNANCE

#### ADR-Nummernkollision durch parallele/direkte Arbeit auf `main`.

Dies ist ein konkretes Beispiel dafür, warum ADR-Registrierung selbst unter Single-Writer-/PR-Governance fallen muss.

Empfohlene Folgemaßnahme außerhalb dieses Audit-PRs:

- ADR-Registry als maschinenprüfbare Source of Truth;
- CI blockiert doppelte numerische ADR-IDs unabhängig vom Dateinamen;
- neue ADR-ID wird erst nach aktuellem `main` reserviert;
- direkte `main`-Writes für `docs/adr/**` durch Branch Protection/CODEOWNER verhindern.

Dieser Audit-PR verändert oder renummeriert den fremden ADR-Commit nicht.

---

## 13. Priorisierte Remediation

### Phase 2A — P0 Durable Business State

1. Supabase PDF-Credit Account + Ledger einführen.
2. Stripe Credit Purchase per Stripe Event/Session idempotent machen.
3. Credit Consumption atomar machen.
4. `pdf_credits.json` aus Production Authority entfernen.
5. lokalen Subscription Write-first-/Fallback-Pfad entfernen.
6. fail-closed Entitlement-Verhalten definieren.

## Phase 2B — P0 Audit/Document Integrity

1. synthetische `system_events.json` Seed-Events entfernen;
2. System-/Audit-Evidence durable persistieren;
3. Production Documentary Filesystem-Mutation deaktivieren;
4. Documentary-Vorschläge in Git PR oder durable Review Queue überführen;
5. lokale Version-Manager-Autorität entfernen.

## Phase 2C — Worker Extraction

1. Market Refresh aus `server.ts` extrahieren;
2. Job Lease / Leader Contract einführen;
3. Score Snapshot in Worker-Pipeline verschieben;
4. Alert Delivery Outbox / Atomic Claim einführen;
5. Alert Evaluation und Versand in Worker verschieben;
6. Worker Shutdown-/Retry-Semantik definieren.

## Phase 2D — Distributed Security / Messaging

1. kritische Rate Limits zentralisieren oder am Gateway erzwingen;
2. Event Mesh Scope explizit als in-process belassen oder External Transport ADR erstellen;
3. Worker-/Job-Telemetrie durable und instance-aware machen;
4. Duplicate-/Stale-Lease-Metriken ergänzen.

## Phase 2E — Scale-out Qualification

Erst danach:

1. Zwei-Instanz-Staging-Test;
2. Deploy während aktiver Worker-Jobs;
3. Beweis: keine doppelten External Side Effects;
4. Beweis: keine Credit-/Subscription-Divergenz;
5. Beweis: Rate Limits unabhängig von Replica Count;
6. Worker Failover nach Lease Expiry testen;
7. erst dann Render Autoscaling neu bewerten.

---

## 14. Immediate Operational Guardrails

Bis Phase 2 abgeschlossen ist:

```
Render Web numInstances MUST remain 1.
Autoscaling MUST remain disabled.
Treat every deploy as a temporary multi-process event.
Do not rely on local PDF credits as durable customer entitlement.
Do not rely on local subscriptions.json as privileged entitlement truth.
```

Do not rely on `system_events.json` as compliance/audit evidence.  
Do not treat runtime Documentary changes as repository truth.  
Do not use the in-memory Event Mesh as a distributed queue.  
Do not assume process-local rate limits remain invariant under scaling.

## 15. Findings Register

| ID         | Priorität     | Finding                                                                  | Status           |
|------------|---------------|--------------------------------------------------------------------------|------------------|
| RND2-F-001 | P0            | Subscription entitlement hat lokale stale/ephemeral Fallback-Autorität.  | OPEN             |
| RND2-F-002 | P0            | Bezahlte PDF-Credits sind local-only und non-atomic.                     | OPEN             |
| RND2-F-003 | P0            | System-/Audit-Events sind ephemeral und enthalten Synthetic Seed Events. | OPEN             |
| RND2-F-004 | P1            | Agent Registry ist instanz-lokal.                                        | OPEN             |
| RND2-F-005 | P0            | Documentary kann Production-Files außerhalb Git/PR verändern.            | OPEN             |
| RND2-F-006 | P1            | Version Manager State ist lokal/mutable/version-drifted.                 | OPEN             |
| RND2-F-007 | P0/P1         | Market Refresh ohne Distributed Lease; Duplicate Execution live belegt.  | CONFIRMED LIVE   |
| RND2-F-008 | PASS/P1       | Score Snapshot DB-Write besitzt Unique-/Idempotency-Control.             | CONTROL PRESENT  |
| RND2-F-009 | P1            | Alert-Send ist nicht duplicate-safe.                                     | OPEN             |
| RND2-F-010 | P1            | Event Mesh ist in-process und kein Distributed Bus.                      | OPEN / BY DESIGN |
| RND2-F-011 | P1 Security   | Rate Limits sind prozess-lokal.                                          | OPEN / BY DESIGN |
| RND2-F-012 | P1            | Kein generischer Worker-Lease-/Outbox-Contract im Produktivschema.       | OPEN             |
| RND2-F-013 | P1 Governance | Doppelte ADR-0036-ID durch parallele/direkte Main-Arbeit.                | CONFIRMED        |

## 16. Definition of Done — ADR-0037 Phase 2

- [ ] Keine authoritative Subscription-/Entitlement-Daten auf Render Local FS.
- [ ] PDF Credit Balance + Ledger durable und transaction-safe.
- [ ] Stripe Credit Purchase idempotent über unveränderliche Provider-ID.
- [ ] Credit Consumption atomar.
- [ ] System-/Audit-Evidence durable.
- [ ] Synthetic Audit Seeds entfernt.
- [ ] Production Documentary mutiert keine Git-kontrollierten Deployment-Dateien.
- [ ] Version Authority kommt aus immutable Deployment Identity.
- [ ] Market Refresh wird nicht von jedem Webprozess ausgeführt.
- [ ] Alert Delivery ist duplicate-safe.
- [ ] Score Snapshot Idempotency bleibt nach Worker Extraction bestehen.
- [ ] Worker Lease/Fencing oder äquivalente Duplicate-Control implementiert.
- [ ] Kritische Rate Limits sind replica-invariant.
- [ ] Event Mesh Transport Scope ist explizit dokumentiert.
- [ ] Zero-Downtime Deploy Test beweist keine doppelten externen Side Effects.
- [ ] Zwei-Instanz-Qualification bestanden, bevor Scale-out aktiviert wird.
- [ ] ADR-ID-Eindeutigkeit wird maschinell geprüft.

---

## 17. Audit Conclusion

ADR-0037 Phase 2 ist **nicht production-complete**.

Der stärkste neue Beweis ist, dass Duplicate Background Execution bereits bei normalen Render Zero-Downtime Deployments auftritt. Stateless-/Worker-Remediation ist daher keine Vorbereitung auf hypothetisches zukünftiges Autoscaling, sondern Voraussetzung für deterministisches Verhalten in der heutigen Produktionsarchitektur.

Empfohlene nächste Engineering-Reihenfolge:

1. Durable PDF Credit Ledger + Subscription fail-closed cleanup
2. Durable Audit/System Event Path + Synthetic Seed Removal
3. Production Documentary filesystem mutation abschalten
4. Worker Lease / Outbox primitives
5. Market Refresh + Alert Execution aus Web extrahieren
6. Distributed critical rate-limit strategy
7. ADR registry uniqueness gate
8. Two-instance qualification

Bis dahin bleibt die sichere Betriebsregel: eine konfigurierte Webinstanz, kein Autoscaling, Deployments als temporäre Multi-Prozess-Phase behandeln.

# CAPITAL-AI Server Modularization

Quelle: docs/architecture/SERVER\_MODULARIZATION.md

## CAPITAL-AI Server Modularization

### Status

**SUPERSEDED** by ADR-0013 and ADR-0014 as of 2026-08-08.

This document records an earlier modularization scaffold under `src/server/`. It is retained temporarily as migration evidence only and is **not** the authoritative production server architecture.

### Canonical runtime

The production architecture is now:

```
server.ts # stable thin process entry
server.application.ts # compatibility composition module during decomposition
server/ # canonical runtime/infrastructure modules
src/routes/ # domain route factories
src/features/ # feature modules
src/services/ # domain services/scoring
src/platform/ # governance/security/platform services
```

See:

- docs/architecture/adr/ADR-0013-server-composition-root-and-modular-bootstrap.md
- docs/architecture/adr/ADR-0014-server-runtime-architecture-consolidation.md

### Historical scaffold

The earlier target described the following `src/server/` structure:

```
src/server/
├── app.ts
├── config/
├── integrations/
├── middleware/
└── routes/
```

The historical document also referenced `lifecycle/startup.ts`, `services/marketData.service.ts`, `scoring.routes.ts`, `sentiment.routes.ts` and `portfolio.routes.ts`. Those paths are not present in the current repository tree. The scaffold therefore must not be treated as a completed production replacement for `server.application.ts`.

### Consolidation rule

No new production dependency may target `src/server/app.ts`. Existing files under `src/server/` are subject to ADR-0014 Phase 2 inventory and retirement. Any unique behavior

found there must first be reconciled against current R-001/R-002/R-003, ADR-0037 and ADR-0040 invariants and then migrated into the canonical runtime tree.

### **Historical objectives retained as design guidance**

The original goals remain valid as guidance: small route modules, explicit infrastructure registration, dependency injection for AI providers, preserved Stripe raw-body ordering, network-independent health checks, deterministic lifecycle ownership and isolated domain services.

Those goals are now implemented incrementally through the canonical architecture rather than by activating the duplicate `src/server/` composition root.

# CAPITAL-AI Vocabulary Governance Migration Roadmap

---

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_MIGRATION\_ROADMAP.md

## CAPITAL-AI Vocabulary Governance Migration Roadmap

Status: COMPLETE / OPERATIONAL

Datum / Date: 2026-08-10

Authority: ESS-0001-CONTRACTS

Related: ESS-0010, ESS-0012, ESS-0017, ESS-0017-CONTRACTS, ADR-0046

CI Cost Governance: docs/governance/GITHUB\_ACTIONS\_BUDGET\_POLICY.md

### Deutsch

#### Zielbild

Naming, Wording, Dokumentation, Traceability und Governance werden als zusammenhängender, ereignisbasierter Teil der CAPITAL-AI Wertschöpfungskette betrieben. Code bleibt Englisch; Enterprise-Dokumentation wird Deutsch und Englisch geführt; menschlich sichtbare Pull-Request-Informationen werden Deutsch geführt.

#### Phase 0 — Baseline und Schutz — COMPLETE

#### Phase 1 — Governance Foundation — COMPLETE

#### Phase 1.5 — Authority- und Nummernraum-Konsolidierung — COMPLETE

ESS-0012 bleibt Documentation Governance; ESS-0017 / ESS-0017-CONTRACTS bleiben Vocabulary Governance; ADR-0046 dokumentiert die Authority-Trennung.

#### Phase 2 — Canonical Vocabulary Registry — COMPLETE

Abschluss durch PR #145.

#### Phase 3 — Safe Rename Gate — COMPLETE

Abschluss durch PR #151 / main CI #649.

#### Phase 4 — Bilingual Documentary Integration — COMPLETE

Abschluss durch PR #152 und erfolgreichen Deploy.

#### Phase 5 — Event-Driven Value Chain — COMPLETE

Abschluss durch PR #153 / main CI #656 / verifiziertes Render-Deployment.

## Phase 6 — Incremental Existing-Code Migration — COMPLETE

PR #154 etablierte Naming-Debt-Inventar und Klassifizierung. PR #155 schloss Phase 6 produktiv ab. REN-0001: Plan -> SubscriptionTier bleibt CONDITIONAL, REN-0002: Screener -> Screening bleibt BLOCKED. Es existierten keine SAFE-Kandidaten; deshalb wurde korrekt keine Rename-Mutation erzwungen. Nicht sichere Kandidaten verbleiben mit automaticMigrationAllowed: false im geschützten Backlog.

## Phase 7 — Continuous Governance — COMPLETE

Abschluss durch PR #156. Implementiert sind Continuous Repository Vocabulary Validation, kostengünstiger CI-Einstiegspunkt, Lifecycle Evidence sowie Regression Protection. Die Canonical Vocabulary Registry bleibt Single Source of Truth; Human Approval, Safe-Rename-Gate, EventMesh und CI-Budgetgrenzen bleiben übergeordnet.

## Migrationsabschluss

Die Vocabulary-Governance-Migration ist abgeschlossen und geht in den operativen Governance-Betrieb über. Weitere Verbesserungen an Documentary Engine, Event-Driven Value Chain und Documentation Hygiene werden in DOCUMENTARY\_EVENT\_VALUE\_CHAIN\_ROADMAP.md geführt.

## English

### Migration sequence

1. Baseline and protection — complete.
2. Governance foundation — complete.
  - 1.5. Authority and namespace reconciliation — complete.
3. Canonical Vocabulary Registry — complete through PR #145.
4. Safe Rename Gate — complete through PR #151 / main CI #649.
5. Bilingual Documentary Integration — complete through PR #152.
6. Event-Driven Value Chain — complete through PR #153 / main CI #656.
7. Incremental Existing-Code Migration — complete through PRs #154 and #155; no unsafe rename was forced.
8. Continuous Governance — complete through PR #156.

### Success criteria

- English-only technical naming for new code.
- DE/EN documentation cannot semantically drift independently.
- Documentation Governance remains ESS-0012; Vocabulary Governance remains ESS-0017.
- Active renames cannot bypass impact validation.
- No non-SAFE rename can be automatically migrated.
- Lifecycle events use the existing EventMesh.

- Automation remains subordinate to protected-change boundaries and CI budget limits.

# CAPITAL-AI Vocabulary Governance — Phase 1.5 Baseline

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_1\_5\_BASELINE.md

## CAPITAL-AI Vocabulary Governance — Phase 1.5 Baseline

Status: Completed

Datum / Date: 2026-08-09

Authority: ESS-0001-CONTRACTS

Scope: ESS namespace reconciliation and documentation baseline before Phase 2

### Deutsch

#### Zweck

Phase 1.5 stellt vor der Implementierung der Canonical Vocabulary Registry einen eindeutigen Governance- und Nummernraumzustand her. Diese Phase nimmt keine aktiven Code-Renames vor und verändert keine Runtime-Abhängigkeiten.

#### Verifizierter Ist-Zustand

1. PR #142 ist in `main` gemerged und die Legacy-Dokumentationsklassifizierung ist damit Teil der aktuellen Baseline.
2. Die verbindliche `.ai/registry/ess-registry.json` führt `ESS-0012` eindeutig als **Documentation Governance** und `ESS-0012-CONTRACTS` als zugehörigen Contract-Teil.
3. `.ai/skills/ESS-0012-Documentation-Governance.md` deklariert `skill.id: ESS-0012` und `own-Contracts: ESS-0012-CONTRACTS`.
4. `.ai/skills/ESS-0012-Contracts.md` definiert ausschließlich Documentation Governance Contracts.
5. `ESS-0013`, `ESS-0014`, `ESS-0015` und `ESS-0016` sind anderweitig belegt.
6. Die frühere Vocabulary-Roadmap referenzierte `ESS-0012`, `ADR-0044` und `ADR-0045` als Vocabulary-Authorities; diese Zuordnung wurde verworfen, weil die IDs bereits belegt sind.
7. `ESS-0017-Vocabulary-Governance.md`, `ESS-0017-CONTRACTS` und `ADR-0046` sind angelegt.
8. Die Vocabulary-Roadmap ist auf die eindeutigen Authorities `ESS-0017`, `ESS-0017-CONTRACTS` und `ADR-0046` umgestellt.
9. Die ESS Registry führt `ESS-0017` und `ESS-0017-CONTRACTS`; `freeNumberSpaceStartsAt` ist auf `ESS-0018` gesetzt.
10. Die Cross-Reference-Prüfung unterscheidet zwischen zulässiger Abhängigkeit zu `ESS-0012` (Documentation Governance) und unzulässiger Verwendung von `ESS-0012` als Vocabulary-Authority.

## Entscheidung für Phase 1.5

- **ESS-0012** bleibt unverändert **Documentation Governance**.
- Vocabulary Governance verwendet **ESS-0017**.
- Die zugehörigen Vocabulary Contracts verwenden **ESS-0017-CONTRACTS**; dieser Contract-Teil belegt keine zusätzliche ESS-Nummer.
- **ADR-0046** definiert die Trennung der Documentation- und Vocabulary-Authorities sowie die Nummernraumkorrektur.
- **ESS-0012 / ESS-0012-CONTRACTS** dürfen im Vocabulary-Scope nur als Documentation-Governance-Abhängigkeit oder Related Authority referenziert werden.
- **ADR-0044** und **ADR-0045** dürfen nicht als Vocabulary-Entscheidungen verwendet werden; historische Hinweise auf die frühere Fehlzuordnung bleiben als Korrektur-Evidence zulässig.
- Phase 2 darf nach Merge dieser Phase auf der eindeutigen Authority-Baseline beginnen.

## Integrationsgrenzen

```
ESS-0001-CONTRACTS
-> ESS-0012 Documentation Governance
-> ESS-0017 Vocabulary Governance
-> Canonical Vocabulary Registry
-> Documentary / Knowledge / Traceability
-> Enterprise Event Mesh
-> Quality / Security / Compliance
-> Supervisor / Platform Director
-> Version Manager / Release
```

**ESS-0012** validiert Dokumentations-Governance. **ESS-0017** definiert Terminologie, Naming und Vocabulary-spezifische Regeln. Globale Enterprise Contracts verbleiben in **ESS-0001-CONTRACTS**.

## Cross-Reference-Prüfung

Geprüft wurden die Phase-1.5-Dokumente, die ESS Registry, der vollständige PR-Diff und repository-weite Suchtreffer für die früher kollidierenden IDs.

Ergebnis:

- **ESS-0017** ist die einzige neue Vocabulary-ESS im Phase-1.5-Scope.
- **ESS-0017-CONTRACTS** ist eindeutig **ESS-0017** zugeordnet.
- **ESS-0012** bleibt Documentation Governance und wird in **ESS-0017** ausschließlich als Abhängigkeit/Related Authority referenziert.
- Für **ADR-0044 + Vocabulary** und **ADR-0045 + Vocabulary** bestehen keine aktiven repository-weiten Treffer als Vocabulary-Authority.
- Vorkommen von **ADR-0044**, **ADR-0045** oder der früheren **ESS-0012**-Vocabulary-Zuordnung innerhalb **ADR-0046**, Roadmap und Baseline sind ausschließlich historische Korrekturhinweise.
- Der PR-Diff führt keine alternative Vocabulary-Authority ein.
- Die Registry weist nach der Änderung **ESS-0018** als nächsten freien Nummernraum aus.

## Schutzregeln

- Code bleibt Englisch.
- Enterprise-Dokumentation wird Deutsch und Englisch geführt.
- Menschlich sichtbare Pull-Request-Informationen bleiben Deutsch.
- Kein aktiver Rename ohne Safe Rename Gate.
- Keine Änderung von Import-/Export-Namen, API-Routen, Schemas, Environment Keys oder Event-Namen in Phase 1.5.
- Keine Big-Bang-Migration.
- Keine parallele EventMesh-, Knowledge- oder Traceability-Infrastruktur.
- Legacy-Dokumente aus PR #142 sind keine kanonische Vocabulary-Quelle, bis ihre Aussagen gegen aktuellen Code und aktuelle Authorities revalidiert wurden.

## Exit-Kriterien Phase 1.5

- [x] PR #142 als Baseline verifiziert.
- [x] ESS-0012-Authority gegen Registry, Skill und Contracts verifiziert.
- [x] belegte ESS-IDs und freier Nummernraum verifiziert.
- [x] kollidierende ADR-0044/ADR-0045-Referenzen identifiziert.
- [x] ESS-0017 Vocabulary Governance angelegt.
- [x] ESS-0017-CONTRACTS angelegt.
- [x] ADR-0046 angelegt.
- [x] Roadmap auf ESS-0017 / ESS-0017-CONTRACTS / ADR-0046 aktualisiert.
- [x] ESS-0017 und ESS-0017-CONTRACTS in `.ai/registry/ess-registry.json` registriert; nächster freier Nummernraum ESS-0018.
- [x] Cross-Reference-Prüfung bestätigt eine eindeutige Vocabulary-Authority unter ESS-0017/ADR-0046.

## English

### Purpose

Phase 1.5 establishes an unambiguous governance and namespace baseline before implementation of the Canonical Vocabulary Registry. It performs no active code renames and changes no runtime dependencies.

### Completed decision

ESS-0012 remains Documentation Governance. Vocabulary Governance is defined by ESS-0017 with ESS-0017-CONTRACTS ; ADR-0046 records the authority and namespace decision. The ESS Registry now allocates ESS-0017 to Vocabulary Governance and advances the next free number to ESS-0018.

Cross-reference validation confirms that ESS-0012 references in Vocabulary artifacts are dependency references to Documentation Governance, not Vocabulary authority assignments. ADR-0044 and ADR-0045 are not used as active Vocabulary decisions.

**Safety boundary**

No imports, exports, APIs, schemas, environment keys, event names, or runtime identifiers are renamed during Phase 1.5. Legacy documentation classified by PR #142 is not a canonical terminology source without current code/authority revalidation.

# CAPITAL-AI Vocabulary Governance — Phase 2 Implementation

---

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_2\_IMPLEMENTATION.md

## CAPITAL-AI Vocabulary Governance — Phase 2 Implementation

Status: Implemented on feature branch

Datum / Date: 2026-08-09

Authority: ESS-0017 / ESS-0017-CONTRACTS

Architecture Decision: ADR-0046

### Deutsch

#### Ziel

Phase 2 implementiert die Canonical Vocabulary Registry als eigenständige Plattformkomponente, ohne bestehende Runtime-Identifizierung umzubenennen und ohne parallele Event-, Knowledge- oder Traceability-Infrastruktur einzuführen.

#### Implementierte Bausteine

- `VocabularyConcept` als typed Concept Contract;
- stabile Concept IDs im Format `VOC-<CATEGORY>-<NNNN>` ;
- englischer `canonicalCodeTerm` ;
- gemeinsame DE/EN Concept Identity;
- Alias- und Forbidden-Term-Verwaltung;
- deterministische Normalisierung und Collision Detection;
- immutable Registry Snapshots;
- öffentliches `IVocabularyRegistry` Interface;
- read-oriented `VocabularyService` ;
- initiale, authority-referenzierte Seed-Concepts;
- Contract-Tests ohne neue Testframework-Abhängigkeit;
- `manifest.json` , `component.yaml` , README und CHANGELOG.

#### Architektur-Integration

Die Komponente liegt unter `src/platform/Vocabulary` und referenziert ESS-0012 ausschließlich als Documentation-Governance-Abhängigkeit. ESS-0013 bleibt EventMesh-Authority.

Phase 2 importiert keine EventMesh-Core-Implementierung und registriert keine neuen Event-Namen.

Knowledge, Documentary und Traceability können die Vocabulary Registry später über öffentliche Interfaces bzw. standardisierte Events konsumieren. Dadurch bleibt die bestehende Wertschöpfungskette erhalten und wird nicht durch eine zweite Infrastruktur fragmentiert.

### Safe-Rename-Grenze

Phase 2 führt keine aktiven Renames aus. Insbesondere werden keine Komponenten, Dateien, Imports/Exports, API-Routen, Schemas, Environment Keys oder bestehenden Event-Namen verändert. Die technische Rename-Impact-Prüfung bleibt Phase 3 vorbehalten.

### Validierung

Vorgesehener Contract-Test:

```
npx tsx src/platform/Vocabulary/Tests/vocabularyRegistry.test.ts
```

Die Tests prüfen Auflösung, Normalisierung, doppelte IDs, Cross-Concept-Alias-Kollisionen, technische Naming-Regeln, Authority-Referenzen, Forbidden Terms und Immutable Snapshots.

### Phase-2-Exit

Phase 2 gilt nach erfolgreicher CI-/Contract-Test-Ausführung als abgeschlossen. Event-driven Lifecycle Integration, Safe Rename Gate und Documentary-Automation bleiben den nachfolgenden Roadmap-Phasen vorbehalten.

## English

### Goal

Phase 2 implements the Canonical Vocabulary Registry as an independent platform component without renaming existing runtime identifiers and without introducing parallel Event, Knowledge or Traceability infrastructure.

### Implemented scope

The implementation provides a typed concept contract, stable language-neutral IDs, English canonical technical terms, shared German/English mappings, alias and forbidden-term governance, deterministic collision detection, immutable registry snapshots, a public registry interface, a read-oriented service, initial authority-backed seed concepts, contract tests and full component metadata.

### Architecture integration

`src/platform/Vocabulary` treats ESS-0012 only as the Documentation Governance dependency and ESS-0013 as the EventMesh authority. Phase 2 does not import EventMesh core implementation and does not register new event names. Downstream lifecycle integration will use public interfaces and canonical events in later phases.

**Safety boundary**

No components, files, imports/exports, API routes, schemas, environment keys or existing event names are renamed in Phase 2. Rename-impact enforcement remains Phase 3.

# CAPITAL-AI Vocabulary Governance — Phase 3 Safe Rename Gate

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_3\_SAFE\_RENAME\_GATE.md

## CAPITAL-AI Vocabulary Governance — Phase 3 Safe Rename Gate

Status: Implemented on feature branch

Datum / Date: 2026-08-10

Authority: ESS-0017 / ESS-0017-CONTRACTS

Architecture Decision: ADR-0046

CI Cost Governance: docs/governance/GITHUB\_ACTIONS\_BUDGET\_POLICY.md

### Deutsch

#### Ziel

Phase 3 führt einen read-only Rename-Impact-Analyzer ein. Er bewertet vorgeschlagene technische Renames, bevor bestehender Anwendungscode verändert wird. Die Analyse ist ein vorgelagertes Governance-Gate und führt selbst keine Umbenennungen durch.

#### Kommando

```
npx tsx scripts/automation/validateRenameImpact.ts --from <SourceTerm> --to <ApprovedCanonicalTerm>
```

Optional kann mit `--root <path>` ein alternativer Repository-Root angegeben werden.

#### Klassifikation

- **SAFE** : keine Runtime-Referenzen und keine Blocker; nachgelagerte technische Gates bleiben trotzdem verpflichtend.
- **CONDITIONAL** : Runtime-Referenzen existieren, aber kein harter Blocker. Eine dependency-aware Migration ist erforderlich.
- **BLOCKED** : mindestens ein harter Governance-/Dependency-Blocker wurde gefunden.

#### Harte Blocker

Der Analyser blockiert insbesondere:

- Zielbegriffe, die nicht als `approved canonicalCodeTerm` in der Canonical Vocabulary Registry registriert sind;
- `forbiddenTerms` aus der Vocabulary Registry;
- technische Zielnamen außerhalb des Identifier-Regex `^[A-Za-z_$][A-Za-z0-9_$]*$`;
- Case-only-Renames, weil Dateisystem- und Deploy-Verhalten zwischen Linux, Windows und macOS divergieren kann;

- sensitive Treffer in Environment-/Config-Referenzen;
- API-/Route-Referenzen;
- Dynamic Imports / Lazy Loading;
- Schema-/Contract-Flächen.

## Referenzscan

Der Analyzer durchsucht textbasierte Repository-Artefakte und ignoriert generierte oder externe Verzeichnisse wie `node_modules`, `dist`, `coverage`, `.git` und Build-Caches. Runtime-Flächen (`src/`, `server.ts`, `scripts/`, `config/`, `.github/`) werden von Dokumentationsflächen (`docs/`, `.ai/`) getrennt bewertet.

## Regex- und Vocabulary-Integration

Der Analyzer escaped Quellbegriffe vor der Regex-Verwendung und verwendet identifizier-aware Boundaries, damit Teilstrings nicht als vollständige technische Referenzen klassifiziert werden. Zielbegriffe werden direkt gegen die in Phase 2 eingeführte Canonical Vocabulary Registry geprüft. Es wird keine zweite Terminologiequelle aufgebaut.

## CI- und Kostenintegration

Phase 3 erzeugt keinen eigenen GitHub-Actions-Workflow für die Rename-Analyse. Der Analyzer ist ein lokaler/read-only Preflight. Für einen PR-Head-SHA wird die technische Evidence aus dem bestehenden einzelnen vollständigen CI-Lauf wiederverwendet. Die Richtlinie `GITHUB_ACTIONS_BUDGET_POLICY.md` gilt verbindlich: keine duplizierte npm/Test/Build-Pipeline, keine Synchronisierung ohne echten main-Drift, kein manueller `Re-run all jobs` zur Statuskosmetik und höchstens ein gezielter Wiederholungslauf nach einer tatsächlichen Fehlerkorrektur.

Bei der Phase-3-Synchronisierung wurde ein verbliebener Verstoß gegen diese Richtlinie erkannt: Der Google-Marketing-Schutzworkflow duplizierte `npm ci`, Typecheck und Production Build auch bei einer reinen `package.json`-Scriptänderung. Die Korrektur verlagert den nach dem Build erforderlichen CSP-Delivery-Test in den einzigen CI-Hauptlauf und reduziert den spezialisierten Marketing-Guard auf eine kostengünstige statische Schutzprüfung. Gleichzeitig wird der Docker-Image-Build auf Pull Requests gemäß Budgetrichtlinie nur noch bei Docker-/Runtime-/Dependency-/Deployment-relevanten Änderungen ausgeführt; auf `main` bleibt er vor Production-Deploy verpflichtend.

## Nachgelagerte Gates

Auch ein `SAFE`-Ergebnis ersetzt keine technische Validierung. Vor einem tatsächlichen Rename bleiben mindestens erforderlich:

1. `npm run lint`
2. `npm test`
3. `npm run build`
4. `npm run predeploy:check`

Diese Gates werden nicht als separater Phase-3-Workflow dupliziert, sondern über die bestehende CI-Evidence genutzt.

Phase 3 erzeugt ausschließlich Evidence. Aktive Renames sind weiterhin Phase 6 vorbehalten.

### Architektur- und Deploy-Grenze

Diese Phase ändert keine bestehenden Komponenten-, Datei-, Import-/Export-, API-, Schema-, Environment- oder Event-Namen. Die einzigen Workflow-Anpassungen konsolidieren bereits vorhandene technische Evidence und setzen die verbindliche 15-EUR-CI-Kostenrichtlinie um; sie verändern nicht den Render-Deploy-Hook oder dessen fail-closed main-Grenze.

## English

### Goal

Phase 3 introduces a read-only rename impact analyzer. It evaluates proposed technical renames before existing application code is changed. The analyzer is a governance gate and never performs a rename itself.

### Classification

**SAFE** means no runtime references and no hard blockers were detected. **CONDITIONAL** means runtime references require a dependency-aware migration. **BLOCKED** means at least one hard governance or dependency blocker exists.

### Integration

Target names are validated against the Phase 2 Canonical Vocabulary Registry. The analyzer scans repository references, separates runtime and documentation surfaces, applies identifier-aware regex matching, blocks case-only renames and sensitive dependency surfaces, and preserves the existing lint/test/build/predeploy gates.

### CI cost boundary

Phase 3 adds no independent rename-analysis workflow. Existing CI evidence is reused under `GITHUB_ACTIONS_BUDGET_POLICY.md`. The previously duplicated Google Marketing install/typecheck/build path is consolidated into the single main CI path, and PR Docker builds become conditional on Docker/runtime/dependency/deployment relevance while remaining mandatory on main before production deployment.

### Safety boundary

Phase 3 produces evidence only. No active component, file, import/export, API, schema, environment or event identifiers are renamed. Actual incremental migrations remain Phase 6 work.

# CAPITAL-AI Vocabulary Governance — Phase 4 Bilingual Documentary Integration

---

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_4\_BILINGUAL\_DOCUMENTARY.md

## CAPITAL-AI Vocabulary Governance — Phase 4 Bilingual Documentary Integration

Status: In Progress

Datum / Date: 2026-08-10

Baseline: `main@a41c85bc24d4ada79bb9c47c5f01693bbcb2857e`

Authorities: ESS-0001-CONTRACTS, ESS-0010, ESS-0012, ESS-0017, ESS-0017-CONTRACTS, ADR-0046

### Deutsch

#### Zweck

Phase 4 verbindet die Canonical Vocabulary Registry mit der Documentary-Schicht, ohne eine noch nicht vorhandene vollständige Documentary Engine vorzutauschen. Die bestehende Documentary-Zielstruktur bleibt erhalten; implementiert wird zunächst ein ausführbarer Bilingual-Contract-Layer.

#### Verifizierte Baseline

Der Main-Commit `a41c85bc24d4ada79bb9c47c5f01693bbcb2857e` wurde durch CI #649 vollständig verifiziert. TypeScript, Unit-Tests, Production Build, CSP Delivery, Deployment Readiness, Docker Build, Runtime-Metadaten und der Render-Deploy-Gate waren erfolgreich.

#### Implementationsgrenze

`src/platform/Documentary/README.md` dokumentiert aktuell, dass die vollständige Documentary Engine noch nicht implementiert ist. Phase 4 führt deshalb nur die kleinste belastbare Integration ein:

- `BilingualDocumentReference` als sprachgebundene Sicht auf eine sprachneutrale Concept-ID;
- `BilingualDocumentPair` als gekoppeltes DE/EN-Paar derselben Concept-ID;
- Projektion ausschließlich aus einem `approved` Vocabulary Concept;
- identischer `canonicalCodeTerm` für DE und EN;
- identische ESS-, ADR- und Traceability-Referenzen für beide Sprachen;
- fail-closed bei unbekannten Concepts, nicht freigegebenen Concepts oder fehlenden Übersetzungen;
- keine neuen Event-Typen und keine EventMesh-Änderung in Phase 4.

## Architekturfluss



Die Dokumentationssprache ist damit eine Darstellung desselben semantischen Concepts und keine zweite unabhängige Terminologiequelle.

### Nicht Teil dieses PRs

- keine vollständige Documentary Engine;
- keine automatische Legacy-Dokumentmigration;
- keine Event-Erzeugung für fehlende Übersetzungen;
- keine Änderungen an Knowledge Graph, Traceability Store oder EventMesh;
- keine aktiven Code-Renames.

Diese Punkte folgen inkrementell innerhalb Phase 4 bzw. Phase 5.

## English

### Purpose

Phase 4 connects the Canonical Vocabulary Registry to the Documentary layer without pretending that the full Documentary Engine already exists. The first executable integration is a bilingual contract/projection layer.

Both German and English views are derived from the same approved `VocabularyConcept.id`, retain the same canonical code term and language-neutral ESS/ADR/traceability references, and fail closed for unknown concepts, unapproved concepts or missing translations. No new EventMesh events are introduced in this phase.

### Exit direction

Phase 4 can be completed when Documentary consumers consistently derive DE/EN representations from shared concept identities, missing translations are handled through the existing event architecture, and the revalidated legacy-document migration can proceed file-by-file without semantic divergence.

# CAPITAL-AI Vocabulary Governance — Phase 5 Event-Driven Value Chain

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_5\_EVENT\_DRIVEN\_VALUE\_CHAIN.md

## CAPITAL-AI Vocabulary Governance — Phase 5 Event-Driven Value Chain

Status: In Progress

Date: 2026-08-10

Base: verified `main` after Phase 4 / PR #152

Authorities: ESS-0001-CONTRACTS, ESS-0013, ESS-0017, ADR-0018

### Deutsch

#### Ziel

Phase 5 verbindet die Governance-Wertschöpfungskette über den bestehenden Enterprise Event Mesh. Es wird kein zweiter Event-Bus und keine parallele Event-Registry eingeführt.

#### Verifizierte Ausgangslage

- Phase 4 ist gemerged und produktiv deployed.
- Der bestehende EventMesh besitzt einen kanonischen Standard-Event-Katalog.
- `PlatformDecisionEvent`, `SupervisorAlertEvent`, `DocumentationGeneratedEvent`, `KnowledgeUpdatedEvent`, `VersionCalculatedEvent`, `ReleasePreparedEvent` und weitere Lifecycle-Ereignisse sind bereits registriert.
- Platform Director besitzt einen fail-closed Decision-Contract, aber noch keine vollständige autonome Entscheidungslogik.

#### Phase-5-Schritt 1 — Approval Bridge

Der erste ausführbare Schritt implementiert die Grenze zwischen Human-/Owner-Freigabe und autonomer Event-Propagation:

```
Supervisor / Evidence
 ↓
PlatformDecisionRecord
 ↓
APPROVED ?
├─ nein → BLOCKED
└─ ja
 ↓
PlatformDecisionEvent
 ↓
Enterprise Event Mesh
```

`publishApprovedPlatformDecision()` darf ausschließlich bereits genehmigte, immutable `PlatformDecisionRecord`-Instanzen veröffentlichen. Die Funktion erzeugt keine Entscheidung und führt keine automatische Genehmigung durch.

## Invarianten

- `status !== APPROVED` → fail closed.
- `correlationId` ist verpflichtend und wird unverändert propagiert.
- `PlatformDirector` ist der einzige Producer dieses Bridge-Schritts.
- Event-Namen stammen ausschließlich aus dem bestehenden Standard Event Catalog.
- Keine direkte Kopplung von EventMesh an Vocabulary, Documentary oder andere Fachkomponenten.
- Keine Änderungen an Stripe, Supabase, Render oder Datenbankschemas.

## Nächste Schritte innerhalb Phase 5

1. Supervisor-Evidence und Impact-Analysis formal an die Decision-Voraussetzungen anbinden.
2. Approved `PlatformDecisionEvent` als Trigger für idempotente Downstream-Handler registrieren.
3. Vocabulary, Documentary, Knowledge und Traceability über ihre öffentlichen Interfaces anbinden.
4. Quality, Security und Compliance als Validierungsstufe ergänzen.
5. Version Manager und Release nur nach erfolgreichen Gates freigeben.
6. Correlation/Causation über die gesamte Kette nachweisbar erhalten.
7. Event-Handler idempotent und replay-sicher gestalten.

## English

### Goal

Phase 5 connects the governance value chain through the existing Enterprise Event Mesh. No second event bus or parallel event registry is introduced.

### Step 1 — Approval Bridge

The first executable step establishes the boundary between explicit Platform Director approval and autonomous event propagation. `publishApprovedPlatformDecision()` only publishes an already approved immutable `PlatformDecisionRecord`; it never creates or auto-approves a decision.

The original correlation identifier is preserved, non-approved decisions fail closed, and only canonical event names from the existing EventMesh catalog are used.

### Phase 5 completion target

A relevant lifecycle change must propagate deterministically through Supervisor assessment, Platform Director approval boundary, Vocabulary/Knowledge/Documentary/Traceability updates, Quality/Security/Compliance validation, Version Manager and Release validation without bypassing human approval or CI/deployment governance.

# CAPITAL-AI Vocabulary Governance — Phase 6 Incremental Existing-Code Migration

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_6\_INCREMENTAL\_MIGRATION.md

## CAPITAL-AI Vocabulary Governance — Phase 6 Incremental Existing-Code Migration

Status: IMPLEMENTATION COMPLETE — DRAFT REVIEW PENDING

Date: 2026-08-10

Authority: ESS-0017 / ESS-0017-CONTRACTS

Verified foundation: PR #154 / 77cf6ed3143dc39120c55f7c767d4a758bc9735d / main CI #658

### Deutsch

#### Ziel und Ergebnis

Phase 6 migriert Naming-Schulden ausschließlich bei nachgewiesenem `SAFE`. Die vollständige Auswertung des aktuellen Inventars ergibt **0 SAFE-Kandidaten**. Daher wurde kein aktiver Code-Rename durchgeführt.

#### 6.0 Naming Debt Inventory — COMPLETE

PR #154 etablierte `rename-candidates.json` mit stabilen `REN-*`-IDs, Vocabulary Concept-IDs, kanonischen Zielbegriffen und `CLASSIFY_ONLY`.

#### 6.1 Evidence Classification — COMPLETE

Die materialisierte Evidence liegt in `docs/governance/vocabulary/rename-classification-evidence.json` und wird gegen den Live-Analyzer regressionsgeprüft.

- `REN-0001 – Plan -> SubscriptionTier`: `CONDITIONAL`. Exakte `Plan`-Referenzen liegen in Billing-/Entitlement-Runtime-Flächen. Ein dependency-aware Migrationsplan ist erforderlich.
- `REN-0002 – Screener -> Screening`: `BLOCKED`. `src/components/Screener.tsx` enthält exakte `Screener`-Runtime-Referenzen und mit `interface ScreenerProps` zugleich eine Contract-Fläche; der Phase-3-Analyzer meldet `SCHEMA_OR_CONTRACT_REFERENCE`.

#### 6.2 SAFE Migration Batch — COMPLETE, NO-OP

`safeMigrationCount = 0`. Ohne SAFE-Kandidat ist keine Rename-Mutation zulässig. Der No-op ist der korrekte fail-closed Abschluss.

#### 6.3 Dependency Validation — FINAL DRAFT GATE

PR #154 / main CI #658 ist vollständig verifiziert, einschließlich Docker und Render. Der Abschluss-Draft muss TypeScript, Tests, Build, Manifest-Integrität und Deployment Readiness bestehen.

## 6.4 Vocabulary / Documentary / Traceability Synchronisation — COMPLETE

Alle Kandidaten bleiben an freigegebene Vocabulary Concept-IDs gebunden. Da keine Mutation ausgeführt wurde, entstehen keine neuen Documentary-/Traceability-Identitäten und keine DE/EN-Drift.

## 6.5 Conditional / Blocked Backlog — COMPLETE

`docs/governance/vocabulary/rename-backlog.json` enthält alle nicht sicheren Kandidaten. REN-0001 ist `CONDITIONAL`, REN-0002 `BLOCKED`; beide sind `DEFERRED` und `automaticMigrationAllowed: false`. Eine spätere Migration benötigt explizite scopegebundene Freigabe und die jeweils erforderliche Dependency-/Architektur-Evidence.

### Exit-Kriterien

- [x] Inventory vollständig und maschinenlesbar.
- [x] Alle Kandidaten durch den bestehenden Phase-3-Analyzer klassifiziert.
- [x] Evidence wird gegen den Live-Analyzer regressionsgeprüft.
- [x] 0 SAFE-Kandidaten dokumentiert; keine unzulässige Mutation.
- [x] Alle `CONDITIONAL/BLOCKED`-Kandidaten im geschützten Backlog.
- [x] Vocabulary/Documentary/Traceability bleiben konsistent.
- [ ] Finaler Draft-PR-CI-Lauf erfolgreich.
- [ ] Merge und anschließender main-CI-/Render-Gate erfolgreich.

### English

Phase 6 is evidence-first and fail-closed. The current inventory contains zero SAFE candidates, so no code rename is permitted or performed. REN-0001 is `CONDITIONAL` because exact `Plan` references occur on billing/entitlement runtime surfaces. REN-0002 is `BLOCKED` because `src/components/Screeners.tsx` combines exact `Screeners` runtime references with the `interface ScreenersProps` contract surface, producing `SCHEMA_OR_CONTRACT_REFERENCE`. Stored evidence is regression-checked against the live Phase-3 analyzer. Production completion requires successful draft CI, merge, and verified main deployment.

# Phase 7 — Continuous Vocabulary Governance

Quelle: docs/architecture/VOCABULARY\_GOVERNANCE\_PHASE\_7\_CONTINUOUS\_GOVERNANCE.md

## Phase 7 — Continuous Vocabulary Governance

Status: IMPLEMENTED IN DRAFT

Date: 2026-08-10

Authority: ESS-0017 / ESS-0017-CONTRACTS

Foundation: Phase 6 / PR #155 / main 4b1fc07c70626fabfd968301f3fb6773ebf05b7c

### Deutsch

#### Ziel

Phase 7 macht Vocabulary Governance zu einer kontinuierlichen, fail-closed Repository-Funktion. Sie verbindet die bestehende Canonical Vocabulary Registry mit Repository-Validierung, CI, Documentary/Knowledge/Traceability-Evidence und Lifecycle-Ereignissen, ohne eine zweite Authority oder einen zweiten EventMesh einzuführen.

#### 7.0 Continuous Repository Gate

`ContinuousGovernanceValidator` scannt governte Quell-, Konfigurations- und Dokumentationsartefakte deterministisch gegen die `forbiddenTerms` der bestehenden Registry. Phase-6-Evidence-Dateien werden als historische Migrationsnachweise explizit ausgenommen. Findings sind maschinenlesbar und blockieren im Strict Gate.

#### 7.1 CI Integration Contract

`scripts/automation/validateContinuousVocabularyGovernance.ts` stellt einen dedizierten, kostengünstigen Validator bereit. Er ist für die bestehende CI vorgesehen und benötigt keinen zusätzlichen Workflow oder Runner. Die zentrale GitHub-Actions-Budgetrichtlinie bleibt bindend.

#### 7.2 Lifecycle Evidence

`GovernanceLifecycleEvent` definiert einen kleinen, transportneutralen Evidence-Contract für `CONCEPT_REGISTERED` und `CONCEPT_REJECTED`. Der Contract trägt Concept-ID, canonical term, Status, Authority- und Traceability-Referenzen. Er trifft keine autonome Governance-Entscheidung und ersetzt nicht den bestehenden EventMesh.

#### 7.3 Documentary / Knowledge / Traceability

Continuous Governance erzeugt keine unabhängigen semantischen Identitäten. Concept-ID und Authority-Referenzen bleiben die verbindende Identität für Documentary-, Knowledge- und Traceability-Projektionen. Dadurch können diese Consumer später an denselben Lifecycle-Contract angebunden werden, ohne Registry-Duplikation.

## 7.4 Regression Protection

Unit-Tests verifizieren Forbidden-Term-Blocking, canonical-term acceptance und immutable Lifecycle Evidence. Bestehende Phase-3/6-Rename-Gates bleiben unverändert und vorrangig für aktive Migrationen.

## 7.5 Schutzgrenzen

- keine autonome Freigabe neuer Concepts;
- keine automatische Rename-Mutation;
- keine zweite Vocabulary Registry;
- kein zweiter EventMesh;
- keine Mutation an Stripe, Supabase, Render, APIs, DB-Schemas oder ENV Keys;
- keine zusätzliche GitHub-Actions-Vollpipeline;
- Human Approval und bestehende Protected-Change-Grenzen bleiben bindend.

## Exit-Kriterien

- [x] kontinuierlicher Vocabulary-Validator implementiert;
- [x] maschinenlesbarer CI-Einstiegspunkt implementiert;
- [x] Lifecycle-Evidence-Contract implementiert;
- [x] Documentary/Knowledge/Traceability-Identitätsregeln dokumentiert;
- [x] Regressionstests implementiert;
- [ ] Draft-PR-CI erfolgreich;
- [ ] Merge und main-CI-/Docker-/Render-Gate erfolgreich.

## English

Phase 7 turns Vocabulary Governance into a continuous fail-closed repository capability. The canonical registry remains the single authority. A repository validator detects forbidden terminology, a low-cost CI entry point exposes the gate, and a transport-neutral lifecycle evidence contract carries stable concept, authority and traceability identities. No autonomous approval, automatic rename, second registry, second event mesh or external-platform mutation is introduced.

# Work Claim — ADR-0014 Phase 3.4.2

---

Quelle: docs/architecture/workclaims/server-domain-decomposition-phase3-4-2.md

## Work Claim — ADR-0014 Phase 3.4.2

Status: ACTIVE Owner scope: server market-data coordination Base: main after PR #115

### Claimed paths

- `server/marketData/**`
- `tests/server/marketDataCoordinator.contract.test.ts`

### Explicitly protected / not modified

- `server.application.ts`
- `server/validateRuntimeSecrets.ts`
- `server/stockFundamentals.ts`
- `server/fmpIndices.ts`
- `src/lib/assetRegistry.ts`
- crypto/meme/traditional scoring services
- Stripe ingress/durable inbox
- Render/Docker/runtime-artifact guard configuration

### Intent

Extract the orchestration semantics of `fetchLiveMarketData()` before moving provider-specific implementations. Provider order, fail-open fallback behavior, provenance labels and best-effort snapshot persistence must remain behavior-preserving during the later cutover.